

Contents

第十三册简介：大模型数学基础	6
本书定位	6
为什么需要这本书	6
适合读者	6
学完后应具备的能力	6
阅读建议	6
与其他书的关系	7
第十三册：大模型数学基础	7
本书定位	7
章节文件	7
第一章：线性代数	7
0. 本讲资料边界与第二轮精修口径	7
1.1 为什么大模型需要线性代数	7
1.1.1 关键公式与 LLM 线性代数速查	8
1.2 向量：模型中的基本表示单位	10
1.3 内积：相似度的基础	10
1.4 范数：向量和梯度的大小	10
1.5 矩阵：线性变换的参数化形式	11
1.6 矩阵乘法：最重要的是 shape	11
1.7 Attention 中的线性代数	11
1.8 正交和投影	12
1.9 特征值和特征向量	12
1.10 SVD：低秩近似的基础	12
1.11 LoRA 为什么是线性代数问题	13
1.12 Embedding 检索中的线性代数	13
1.13 最小可运行线性代数审计 demo	14
1.14 面试高频问题	16
1.14.1 为什么 attention score 是 T by T	16
1.14.2 Dot product 和 cosine similarity 有什么区别	16
1.14.3 LoRA 为什么省参数	16
1.14.4 为什么要关注 gradient norm	16
1.15 常见误区	16
1.16 小练习	17
1.17 本章总结	17
第二章：概率论	17
0. 本讲资料边界与第二轮精修口径	17
2.1 为什么 LLM 离不开概率论	17
2.1.1 关键公式与 LLM 概率速查	18
2.2 随机变量：把不确定结果形式化	20
2.3 概率分布：模型输出的真正对象	20
2.4 联合概率、条件概率和边缘概率	20
2.5 概率链式法则：自回归语言模型的基础	21
2.6 为什么 next-token prediction 是概率建模	21
2.7 最大似然估计：让训练数据更可能出现	21
2.8 期望：平均意义下的目标	22
2.9 方差：不确定性和噪声大小	22
2.10 贝叶斯公式：根据证据更新信念	22
2.11 独立性和条件独立	23
2.12 采样：从概率分布生成文本	23
2.13 概率高不等于事实真	23
2.14 校准：概率能否反映真实可信度	24

2.15 最小可运行概率审计 demo	24
2.16 面试高频问题	27
2.16.1 为什么语言模型可以只预测下一个 token	27
2.16.2 最大似然和交叉熵是什么关系	27
2.16.3 为什么 batch size 会影响训练稳定性	27
2.16.4 为什么概率高不代表事实正确	27
2.17 常见误区	28
2.18 小练习	28
2.19 本章总结	28
第三章：信息论	28
0. 本讲资料边界与第二轮精修口径	28
3.1 为什么大模型需要信息论	29
3.1.1 关键公式与 LLM 信息论速查	29
3.2 信息量：越意外，信息越多	31
3.3 熵：分布的不确定性	31
3.4 交叉熵：用模型分布解释真实分布的代价	32
3.5 交叉熵和最大似然的关系	32
3.6 KL 散度：两个分布的差异	33
3.7 交叉熵、熵和 KL 的关系	33
3.8 Perplexity：平均每步有多少有效选择	33
3.9 为什么低 loss 不一定代表好模型	34
3.10 RLHF 中的 KL penalty	34
3.11 DPO 和 KL 约束的关系	35
3.12 互信息：知道一个变量能减少多少不确定性	35
3.13 信息论视角理解 RAG	35
3.14 信息论视角理解安全和拒答	36
3.15 最小可运行信息论审计 demo	36
3.16 面试高频问题	38
3.16.1 交叉熵和 KL 有什么关系	38
3.16.2 Perplexity 是什么	38
3.16.3 为什么 RLHF 需要 KL penalty	38
3.16.4 DPO 为什么需要 reference model	38
3.16.5 为什么 perplexity 不能跨 tokenizer 随便比较	38
3.16.6 KL 散度为什么不是普通距离	39
3.16.7 互信息在 RAG 里怎么理解	39
3.17 常见误区	39
3.18 小练习	39
3.19 本章总结	39
第四章：优化基础	40
0. 本讲资料边界与第二轮精修口径	40
4.1 优化在大模型训练中解决什么问题	40
4.1.1 关键公式与 LLM 优化速查	40
4.2 梯度：loss 下降最快的局部方向	42
4.3 一个最小梯度下降例子	43
4.4 Learning rate：优化中最敏感的旋钮	43
4.5 SGD：随机估计全量梯度	44
4.6 Momentum：给梯度下降加惯性	44
4.7 Adam：自适应学习率	45
4.8 AdamW：解耦 weight decay	45
4.9 Warmup：训练早期不要走太猛	46
4.10 Learning rate decay：后期小步收敛	46
4.11 Gradient clipping：限制异常更新	46
4.12 Hessian 和曲率直觉	47
4.13 凸优化和非凸优化直觉	47

4.14 Batch size、gradient accumulation 和优化噪声	48
4.15 Loss spike 的优化视角	48
4.16 最小可运行优化审计 demo	48
4.17 面试高频问题	51
4.17.1 为什么大模型训练常用 warmup	51
4.17.2 Adam 和 AdamW 有什么区别	51
4.17.3 为什么要做 gradient clipping	51
4.17.4 Batch size 和 learning rate 怎么关联	51
4.17.5 Adam 的一阶矩和二阶矩分别是什么	51
4.17.6 训练出现 loss spike 你怎么从优化角度排查	51
4.18 常见误区	51
4.19 小练习	52
4.20 本章总结	52
第五章：矩阵分解与低秩	52
0. 本讲资料边界与第二轮精修口径	52
5.1 为什么大模型需要低秩直觉	53
5.1.1 关键公式与低秩速查	53
5.2 Rank：矩阵真正有多少独立方向	55
5.3 SVD：把矩阵拆成重要方向	55
5.4 为什么低秩近似可行	55
5.5 PCA：寻找数据中最重要的方向	56
5.6 LoRA：低秩更新而不是低秩权重	56
5.7 LoRA 参数量怎么算	57
5.8 LoRA rank 如何选择	58
5.9 LoRA 应该加在哪些模块	58
5.10 QLoRA 和量化的关系	58
5.11 低秩和模型压缩	59
5.12 Embedding 几何和低维可视化	59
5.13 低秩方法的优点和边界	60
5.14 最小可运行低秩审计 demo	60
5.15 面试高频问题	63
5.15.1 LoRA 为什么省参数	63
5.15.2 LoRA 是不是让原权重低秩	63
5.15.3 Rank 越大越好吗	63
5.15.4 PCA 对大模型有什么用	63
5.16 常见误区	63
5.17 小练习	63
5.18 本章总结	64
第六章：统计学习与泛化	64
0. 本讲资料边界与第二轮精修口径	64
6.1 泛化到底是什么	64
6.1.1 关键公式与泛化诊断速查	65
6.2 训练误差、验证误差和泛化误差	66
6.3 Bias-Variance 直觉	67
6.4 Overfitting：模型学到了训练集的细节	67
6.5 Underfitting：模型还没学够	68
6.6 Regularization：限制模型过度贴合训练数据	68
6.7 数据规模和模型规模	68
6.8 大模型为什么参数多但还能泛化	69
6.9 记忆和泛化不是二选一	69
6.10 Benchmark contamination	69
6.11 分布外泛化和组合泛化	70
6.12 泛化和评估设计	70
6.13 最小可运行泛化审计 demo	70

6.14 面试高频问题	74
6.14.1 大模型是在背诵还是泛化	74
6.14.2 如何判断 SFT 过拟合	74
6.14.3 如何缓解过拟合	74
6.14.4 Benchmark contamination 为什么危险	74
6.15 常见误区	74
6.16 小练习	75
6.17 本章总结	75
第七章：贝叶斯与不确定性	75
0. 本讲资料边界与第二轮精修口径	75
7.1 为什么大模型需要不确定性意识	76
7.1.1 关键公式与不确定性速查	76
7.2 先验、似然和后验	78
7.3 语言先验和证据	78
7.4 MAP：先验和数据的折中	78
7.5 不确定性的两类来源	79
7.6 校准：置信度是否可信	79
7.7 如何估计 LLM 的不确定性	79
7.8 幻觉和不确定性	80
7.9 RAG 中的不确定性处理	80
7.10 高风险场景如何决策	80
7.11 LLM-as-a-Judge 的校准	81
7.12 最小可运行贝叶斯与校准审计 demo	81
7.13 面试高频问题	84
7.13.1 贝叶斯公式的直觉是什么	84
7.13.2 LLM 为什么会自信地错	84
7.13.3 如何让模型知道自己不知道	84
7.13.4 如何校准 LLM judge	84
7.14 常见误区	84
7.15 小练习	85
7.16 本章总结	85
第八章：强化学习数学	85
0. 本讲资料边界与第二轮精修口径	85
8.1 为什么 RLHF 需要强化学习	86
8.1.1 关键公式与 RLHF 数学速查	86
8.2 MDP 的基本组成	88
8.3 Policy：模型如何选择动作	88
8.4 Reward 和 Return	88
8.5 Value function：估计未来能拿多少 reward	88
8.6 Advantage：比预期好多少	89
8.7 Policy gradient 的直觉	89
8.8 PPO 的核心直觉	89
8.9 KL penalty 在 RLHF 中的作用	90
8.10 RLHF 完整流程的数学视角	90
8.11 Reward hacking	90
8.12 DPO 和 RLHF 的关系	91
8.13 最小可运行强化学习数学审计 demo	91
8.14 RLHF 和普通 RL 的区别	93
8.15 面试高频问题	94
8.15.1 PPO 在 RLHF 中做什么	94
8.15.2 Value model 有什么用	94
8.15.3 Advantage 是什么	94
8.15.4 为什么 RLHF 不能只看 reward score	94
8.16 常见误区	94

8.17 小练习	94
8.18 本章总结	94
第九章：评估统计与显著性	95
0. 本讲资料边界与第二轮精修口径	95
9.1 为什么模型评估需要统计意识	95
9.1.1 关键公式与评估统计速查	96
9.2 随机变量、均值和方差	97
9.3 标准误和样本量	97
9.4 假设检验和 p-value	97
9.5 置信区间：比单个分数更有用	98
9.6 统计显著和实际显著	98
9.7 Paired evaluation	99
9.8 Bootstrap	99
9.9 最小可运行评估统计审计 demo	99
9.10 A/B test	102
9.11 多重比较问题	103
9.12 Ablation 和 controlled experiment	103
9.13 LLM-as-a-Judge 的统计问题	104
9.14 面试高频问题	104
9.14.1 p-value 是什么	104
9.14.2 为什么要报告置信区间	104
9.14.3 Bootstrap 在大模型评估中怎么用	104
9.14.4 如何判断新模型是否值得上线	104
9.15 常见误区	104
9.16 小练习	105
9.17 本章总结	105
第十章：数学面试题	105
0. 本讲资料边界与第二轮精修口径	105
10.1 面试回答数学题的基本模板	105
10.1.1 关键公式与数学面试总复盘	106
10.2 交叉熵和 KL 的关系是什么	108
10.3 $KL(p q)$ 和 $KL(q p)$ 有什么区别	109
10.4 Perplexity 如何理解	109
10.5 Softmax 为什么常和交叉熵一起用	109
10.6 为什么训练语言模型等价于最大似然估计	110
10.7 Adam 和 SGD 的区别是什么	110
10.8 AdamW 为什么不同于 Adam	111
10.9 学习率 warmup 为什么有用	111
10.10 梯度裁剪为什么能稳定训练	111
10.11 为什么归一化有助于训练	112
10.12 SVD 和低秩近似是什么	112
10.13 LoRA 为什么是低秩	112
10.14 Attention 中点积为什么要除以 $\sqrt{d}$	113
10.15 为什么残差连接有助于深层网络	113
10.16 Bias-variance trade-off 如何理解	113
10.17 为什么正则化能缓解过拟合	114
10.18 Bayes 公式在大模型中有什么用	114
10.19 MLE 和 MAP 有什么区别	114
10.20 DPO loss 如何理解	115
10.21 PPO 中 advantage 是什么	115
10.22 PPO 为什么需要 clipping	115
10.23 统计显著和实际显著有什么区别	116
10.24 p-value 和置信区间怎么解释	116
10.25 Bootstrap 为什么适合大模型评估	116

10.26 为什么 pairwise eval 常比单独打分更稳	117
10.27 如果模型分数提升 0.5%，你如何判断是否可信	117
10.28 如何解释 embedding 相似度	117
10.29 为什么高维空间中的距离会失效	118
10.30 面试中的数学题答题清单	118
10.31 高频快问快答	118
10.31.1 Cross entropy 为什么能训练分类模型	118
10.31.2 KL 为什么非负	118
10.31.3 为什么 log probability 要相加	118
10.31.4 Temperature 对 softmax 有什么影响	119
10.31.5 Label smoothing 有什么作用	119
10.31.6 Weight decay 和 L2 正则一样吗	119
10.31.7 为什么 batch size 会影响训练	119
10.31.8 为什么要做 learning rate decay	119
10.31.9 为什么 checkpoint averaging 或 EMA 有时有效	119
10.31.10 为什么 LLM judge 需要校准	119
10.32 常见追问陷阱	119
10.32.1 “最小化 CE 等价于最小化 KL” 总是对吗	119
10.32.2 “PPL 低就一定模型好” 对吗	119
10.32.3 “LoRA 参数少所以一定不如全参微调” 对吗	119
10.32.4 “p-value 小就一定能上线” 对吗	120
10.32.5 “DPO 一定比 PPO 好” 对吗	120
10.33 最小可运行数学面试复盘 demo	120
10.34 小练习	124
10.35 本章总结	124

第十三册简介：大模型数学基础

本书定位

第十三册系统补齐大模型算法岗需要的数学基础。它不是泛泛学习数学，而是聚焦线性代数、概率论、信息论、优化、低秩、统计学习、贝叶斯、强化学习数学和评估统计如何服务于 LLM。

为什么需要这本书

大模型很多问题背后都有数学直觉：交叉熵和 KL、SVD 和低秩微调、Adam 和优化动态、显著性和 A/B 测试、Reward Model 和偏好学习。如果数学薄弱，面试回答很容易停留在术语层。

适合读者

1. 数学基础不稳但想准备大模型算法岗的人。
2. 想把公式和代码、实验、面试表达联系起来的人。
3. 需要补齐优化、概率、信息论和统计评估的人。

学完后应具备的能力

1. 能解释向量、矩阵、投影、SVD 和低秩近似。
2. 能理解概率建模、MLE、交叉熵、KL 和 perplexity。
3. 能解释梯度下降、Adam、AdamW、学习率调度和优化稳定性。
4. 能理解统计显著性、置信区间和实验设计。

阅读建议

建议按短板选读，不必一次读完。每学一个数学点，都要回到 LLM 训练、对齐、评估或部署中的具体用法。

与其他书的关系

第十三册是第一册、第二册、第五册、第七册和第十册的数学支撑层。

第十三册：大模型数学基础

本书定位

本书系统补齐大模型算法岗需要的数学基础。目标不是泛泛学习数学，而是理解概率、优化、线性代数、信息论和统计学习如何直接服务于 LLM、对齐、推理、评估和安全。

章节文件

1. chapters/01-线性代数.md
2. chapters/02-概率论.md
3. chapters/03-信息论.md
4. chapters/04-优化基础.md
5. chapters/05-矩阵分解与低秩.md
6. chapters/06-统计学习与泛化.md
7. chapters/07-贝叶斯与不确定性.md
8. chapters/08-强化学习数学.md
9. chapters/09-评估统计与显著性.md
10. chapters/10-数学面试题.md

第一章：线性代数

线性代数是理解大模型的第一层数学语言。Embedding 是向量，Linear layer 是矩阵乘法，Attention 是一组矩阵乘法和归一化，LoRA 是低秩矩阵更新，SVD 是理解表示压缩和低秩近似的基础。

很多人觉得自己“知道 Transformer”，但一问 Q、K、V 的 shape、attention score 为什么是 [T, T]、LoRA 为什么省参数、embedding 检索为什么要归一化，就会发现线性代数地基并不稳。

本章目标不是重写一本线性代数教材，而是讲清大模型算法岗最常用的线性代数直觉：向量、矩阵、矩阵乘法、内积、范数、正交、投影、特征值、SVD，以及它们如何出现在 embedding、attention、MLP、LoRA、RAG 和评估里。

0. 本讲资料边界与第二轮精修口径

本章第二轮精修时，重点核对了 MIT OpenCourseWare 18.06 线性代数课程、NumPy linalg 文档、PyTorch torch.linalg 文档、LoRA 论文，以及前序 Transformer、RAG、LoRA、分布式训练和评估章节中的 shape / 参数量口径。

本章只讲大模型算法岗最常用的线性代数，不扩展成完整数学教材：

1. 重点放在向量、矩阵、内积、范数、矩阵乘法 shape、投影、特征值、SVD、低秩近似和它们在 LLM 中的落点。
2. 不展开抽象向量空间、Jordan 标准形、矩阵微积分全套推导、数值线代算法细节或 LAPACK 级实现。
3. 公式必须能服务面试表达：看到公式后能说清变量、shape、直觉和工程场景。
4. 代码 demo 用标准库实现核心计算，避免用框架 API 把线性代数机制藏起来。

1.1 为什么大模型需要线性代数

大模型中的绝大多数中间状态都可以看成向量、矩阵或更高维张量。

一段 token 序列进入模型后，常见 hidden states 的 shape 是：

X: [B, T, d]

其中:

1. B 是 batch size。
2. T 是 sequence length。
3. d 是 hidden size。

每个 token 是一个 d 维向量，一句话是一组 token 向量，一个 batch 是多组句子向量。

Transformer 的主要计算，就是对这些向量反复做：

1. 线性变换。
2. 相似度计算。
3. 加权求和。
4. 归一化。
5. 低秩或高维空间中的表示变换。

如果线性代数理解清楚，很多 LLM 问题会变得自然：

1. Linear layer 参数量为什么是 $d_{in} * d_{out}$?
2. Attention 为什么要做 $Q K^T$?
3. 为什么 attention score 的 shape 是 $[B, H, T, T]$?
4. 为什么 embedding 检索常用 cosine similarity?
5. 为什么 LoRA 用两个小矩阵近似一个大矩阵更新?
6. 为什么 gradient norm 能反映训练稳定性?

1.1.1 关键公式与 LLM 线性代数速查

token hidden state 通常写成：

$$h_i \in \mathbb{R}^d$$

其中 d 是 hidden size， h_i 是第 i 个 token 在某一层的上下文表示。

两个向量的内积：

$$x^\top y = \sum_{j=1}^d x_j y_j$$

L2 范数：

$$\|x\|_2 = \sqrt{\sum_{j=1}^d x_j^2}$$

cosine similarity：

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

如果 x 和 y 都已经 L2 normalize，那么 dot product 和 cosine similarity 等价：

$$\bar{x}^\top \bar{y} = \cos(x, y)$$

矩阵乘法 shape：

$$X \in \mathbb{R}^{B \times d_{in}}, \quad W \in \mathbb{R}^{d_{in} \times d_{out}}, \quad Y = XW + b \in \mathbb{R}^{B \times d_{out}}$$

多头 attention 的核心 shape:

$$Q, K, V \in \mathbb{R}^{B \times H \times T \times d_h}$$

$$S = \frac{QK^\top}{\sqrt{d_h}} \in \mathbb{R}^{B \times H \times T \times T}$$

$$O = \text{softmax}(S)V \in \mathbb{R}^{B \times H \times T \times d_h}$$

这里后两维 $T \times T$ 表示每个 query token 都和每个 key token 计算一次相似度。

向量投影:

$$\text{proj}_u(x) = \frac{x^\top u}{u^\top u} u$$

如果 u 是单位向量, 则 $\text{proj}_u(x) = (x^\top u)u$ 。

特征值和特征向量:

$$Av = \lambda v$$

SVD:

$$W = U\Sigma V^\top$$

rank-r 近似:

$$W_r = U_r \Sigma_r V_r^\top$$

LoRA 低秩更新:

$$\Delta W = AB, \quad A \in \mathbb{R}^{d_{in} \times r}, \quad B \in \mathbb{R}^{r \times d_{out}}$$

LoRA 可训练参数量:

$$N_{LoRA} = r(d_{in} + d_{out})$$

相对全量更新的参数比例:

$$\rho_{LoRA} = \frac{r(d_{in} + d_{out})}{d_{in}d_{out}}$$

线性代数面试题的底层能力不是背公式, 而是看到任意 LLM 结构后能快速说清: 对象是向量还是矩阵, shape 是否能相乘, 哪个维度在做相似度, 哪个维度在做加权求和, 参数量和计算量如何随维度变化。

1.2 向量：模型中的基本表示单位

向量可以理解为一组有方向和大小的数字。

在大模型中，一个 token 的 hidden state 可以写成：

$h_i \in \mathbb{R}^d$

它不是一个人工可解释的标签，而是模型内部对当前 token 在当前上下文中的表示。

例如“苹果”这个词，在“我吃了一个苹果”和“苹果发布了新芯片”中，初始 token embedding 可能相同，但经过 Transformer 层以后，上下文表示会变得不同。因为 self-attention 会让这个 token 从周围 token 中读取信息。

向量有两个重要直觉：

1. 方向：常常对应语义、属性或特征组合。
2. 长度：常常对应尺度、激活强度或模型内部表示大小。

在 embedding 检索中，很多时候我们更关心方向而不是长度。因此常常先把向量归一化，再用 dot product 或 cosine similarity 比较语义相似度。

1.3 内积：相似度的基础

两个向量的内积是对应元素相乘再求和：

$\text{dot}(x, y) = \sum_i x_i * y_i$

内积可以理解为两个向量的匹配程度。

1. 方向相近，内积通常较大。
2. 方向近似垂直，内积接近 0。
3. 方向相反，内积可能为负。

Cosine similarity 会去掉长度影响，只比较方向：

$\text{cos}(x, y) = \text{dot}(x, y) / (\text{norm}(x) * \text{norm}(y))$

在 RAG 和 embedding 检索中，query embedding 和 document embedding 的相似度常用 cosine similarity 或归一化后的 dot product。

面试回答：

Dot product 同时受到方向和长度影响，cosine similarity 会除以向量范数，更关注方向相似性。在 embedding 检索里，我们通常希

1.4 范数：向量和梯度的大小

范数是衡量向量大小的函数。最常见的是 L2 norm：

$\text{norm}_2(x) = \sqrt{\sum_i x_i^2}$

在大模型中，范数经常出现在：

1. Embedding 归一化。
2. Gradient norm 监控。
3. Gradient clipping。
4. Weight decay。
5. LayerNorm 和 RMSNorm。
6. Cosine similarity。

训练时，gradient norm 是非常重要的监控指标。如果 gradient norm 突然变大，可能说明 learning rate 过大、数据 batch 异常、混合精度溢出、loss mask 错误或模型开始不稳定。

面试回答：

范数可以理解为向量的尺度。训练中常看 gradient norm 来判断梯度是否异常，必要时使用 gradient clipping 限制更新幅度。检索

1.5 矩阵：线性变换的参数化形式

矩阵可以理解为一组向量，也可以理解为一个线性变换。

一个 Linear layer 通常写成：

$$Y = X W + b$$

如果：

X: [B, d_in]

W: [d_in, d_out]

b: [d_out]

Y: [B, d_out]

那么这个线性层把每个输入向量从 d_in 维映射到 d_out 维。

Transformer 中的 Q、K、V 就来自三个线性投影：

$$Q = X W_Q$$

$$K = X W_K$$

$$V = X W_V$$

这说明矩阵不是抽象符号，而是模型学习到的表示变换。W_Q 学会把 hidden state 变成“查询向量”，W_K 学会变成“索引向量”，W_V 学会变成“被读取的内容向量”。

1.6 矩阵乘法：最重要的是 shape

矩阵乘法的 shape 规则是：

A: [m, n]

B: [n, p]

$$C = A B$$

C: [m, p]

中间维度 n 必须匹配。

在大模型面试中，shape 推导非常重要。以多头注意力为例：

Q: [B, H, T, d_h]

K: [B, H, T, d_h]

V: [B, H, T, d_h]

计算 attention scores：

$$\text{scores} = Q @ K^T$$

scores: [B, H, T, T]

再加权求和：

$$\text{context} = \text{softmax}(\text{scores}) @ V$$

context: [B, H, T, d_h]

后两维 [T, T] 的含义是：每个 query token 都和每个 key token 计算一个相似度。

面试回答：

Attention 的 shape 可以从矩阵乘法推出来。Q 和 K 的最后一维都是 head_dim，Q 乘 K 的转置后，每个 token 会和所有 token 得到一

1.7 Attention 中的线性代数

Self-attention 的核心可以写成：

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_h}) V$$

这行式子包含三个线性代数动作。

第一，线性投影。

```
Q = X W_Q  
K = X W_K  
V = X W_V
```

第二，内积相似度。

```
scores = Q K^T
```

第三，加权求和。

```
output = softmax(scores) V
```

为什么要除以 $\sqrt{d_h}$ ？

因为维度变大时，随机向量内积的尺度会变大，softmax 容易变得过于尖锐，导致梯度不稳定。除以 $\sqrt{d_h}$ 是为了稳定 attention score 的尺度。

1.8 正交和投影

两个向量正交，表示它们内积为 0：

```
dot(x, y) = 0
```

直观上，它们代表互不重叠的方向。

投影可以理解为：一个向量在某个方向上的分量。如果 u 是单位向量，那么 x 在 u 方向上的投影长度是：

```
dot(x, u)
```

投影在机器学习里很常见。PCA 可以看成寻找一组重要方向，把数据投影到这些方向上，尽量保留最大方差。Embedding 分析、表示压缩和低秩近似也都和投影直觉有关。

面试中不一定会直接问投影公式，但会问类似问题：为什么低维表示可以保留主要信息？为什么 PCA 能降维？为什么某些表示方向能对应模型行为？这些背后都需要投影直觉。

1.9 特征值和特征向量

如果一个矩阵 A 作用在向量 v 上，只改变它的长度，不改变它的方向，就有：

```
A v = lambda v
```

这里 v 是特征向量， λ 是特征值。

直觉上，特征向量是这个线性变换的“稳定方向”，特征值表示这个方向被放大或缩小多少。

在大模型面试中，特征值不一定高频直接出现，但它支撑很多重要概念：

1. PCA 的主方向。
2. Hessian 和优化曲率。
3. 矩阵稳定性。
4. 表示空间中的主要变化方向。

面试回答可以保持直觉：

特征向量是矩阵变换后方向不变的向量，特征值表示这个方向被缩放的倍数。它帮助我们理解一个线性变换在哪些方向上放大、压缩。

1.10 SVD：低秩近似的基础

SVD，也就是奇异值分解，可以把一个矩阵分解为：

```
W = U S V^T
```

直觉上，SVD 把一个矩阵拆成一组从重要到不重要的方向。S 中的奇异值越大，说明对应方向越重要。

如果只保留最大的 r 个奇异值，就得到一个 rank- r 近似：

$$W_{\text{approx}} = U_r S_r V_r^T$$

这说明很多矩阵的信息可能集中在少数重要方向上，可以用低秩矩阵近似。

这和大模型里的 LoRA 有直接联系。LoRA 假设微调时权重更新 ΔW 可以用低秩矩阵表示：

$$\Delta W = A B$$

其中：

A: $[d_{\text{in}}, r]$

B: $[r, d_{\text{out}}]$

$r \ll \min(d_{\text{in}}, d_{\text{out}})$

原本更新一个大矩阵需要 $d_{\text{in}} * d_{\text{out}}$ 个参数，而 LoRA 只需要：

$$r * (d_{\text{in}} + d_{\text{out}})$$

当 r 很小，参数量和显存都会显著下降。

面试回答：

SVD 说明一个矩阵可以分解为若干重要方向的组合，如果主要信息集中在少数方向上，就可以用低秩近似。LoRA 的直觉类似：微调不

1.11 LoRA 为什么是线性代数问题

普通全参微调会直接更新权重：

$$W_{\text{new}} = W + \Delta W$$

LoRA 不直接学习完整的 ΔW ，而是学习低秩分解：

$$\Delta W = A B$$

假设 W 的 shape 是：

$W: [4096, 4096]$

全量更新参数量约为：

$$4096 * 4096 = 16777216$$

如果 LoRA rank 是 8，那么参数量约为：

$$4096 * 8 + 8 * 4096 = 65536$$

这就是 LoRA 省参数的线性代数原因。

但低秩也有边界。rank 太小，表达能力可能不足；rank 太大，参数和显存节省变少，也更容易过拟合。面试中要体现这个 trade-off。

1.12 Embedding 检索中的线性代数

RAG 中的向量检索，本质上也是线性代数。

离线阶段：

document -> embedding vector

在线阶段：

query -> embedding vector

然后比较 query 向量和 document 向量的相似度。

常见相似度：

1. Dot product。
2. Cosine similarity。
3. L2 distance。

如果 embedding 已经归一化, dot product 和 cosine similarity 排序通常等价。

面试回答:

Embedding 检索把文本映射到向量空间, 语义相似的文本应该方向接近。检索时用 dot product、cosine similarity 或 L2 distance

1.13 最小可运行线性代数审计 demo

下面这个 demo 不依赖 NumPy 或 PyTorch, 直接用 Python 标准库检查向量相似度、矩阵乘法 shape、scaled dot-product attention、SVD 能量直觉和 LoRA 参数量。

```
from math import exp, sqrt
```

```
def shape(matrix):  
    return (len(matrix), len(matrix[0]) if matrix else 0)
```

```
def dot(x, y):  
    return sum(a * b for a, b in zip(x, y))
```

```
def l2_norm(x):  
    return sqrt(dot(x, x))
```

```
def cosine(x, y):  
    return dot(x, y) / (l2_norm(x) * l2_norm(y))
```

```
def normalize(x):  
    n = l2_norm(x)  
    return [v / n for v in x]
```

```
def matmul(a, b):  
    rows, inner = shape(a)  
    inner_b, cols = shape(b)  
    assert inner == inner_b, (shape(a), shape(b))  
    return [[sum(a[i][k] * b[k][j] for k in range(inner)) for j in range(cols)] for i in range(rows)]
```

```
def transpose(a):  
    rows, cols = shape(a)  
    return [[a[i][j] for i in range(rows)] for j in range(cols)]
```

```
def softmax(row):  
    m = max(row)  
    values = [exp(v - m) for v in row]  
    total = sum(values)  
    return [v / total for v in values]
```

```

def scaled_dot_product_attention(q, k, v):
    head_dim = len(q[0])
    scores = matmul(q, transpose(k))
    scaled = [[s / sqrt(head_dim) for s in row] for row in scores]
    attn = [softmax(row) for row in scaled]
    return scaled, attn, matmul(attn, v)

def project(x, u):
    scale = dot(x, u) / dot(u, u)
    return [round(scale * value, 3) for value in u]

def lora_params(d_in, d_out, rank):
    full = d_in * d_out
    lora = rank * (d_in + d_out)
    return full, lora, round(lora / full, 6)

x = [1.0, 2.0, 2.0]
y = [2.0, 0.0, 1.0]
x_bar = normalize(x)
y_bar = normalize(y)
l2_sq_normalized = sum((a - b) ** 2 for a, b in zip(x_bar, y_bar))

q = [[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]]
k = [[1.0, 0.0], [0.0, 1.0], [1.0, -1.0]]
v = [[1.0, 2.0], [3.0, 0.0], [0.0, 1.0]]
scores, attn, context = scaled_dot_product_attention(q, k, v)

singular_values = [5.0, 2.0, 0.5]
energy_top2 = sum(s * s for s in singular_values[:2]) / sum(s * s for s in singular_values)
full_params, rank8_params, ratio = lora_params(4096, 4096, 8)

checks = {
    "normalized_dot_equals_cosine": round(dot(x_bar, y_bar), 3) == round(cosine(x, y), 3),
    "normalized_l2_relation": round(l2_sq_normalized, 3) == round(2 - 2 * cosine(x, y), 3),
    "attention_rows_sum_to_one": all(abs(sum(row) - 1.0) < 1e-9 for row in attn),
    "context_shape_ok": shape(context) == (3, 2),
    "lora_saves_params": rank8_params < full_params,
}

print("dot=", round(dot(x, y), 3))
print("norms=", (round(l2_norm(x), 3), round(l2_norm(y), 3)))
print("cosine=", round(cosine(x, y), 3))
print("normalized_l2_sq=", round(l2_sq_normalized, 3))
print("projection_on_x_axis=", project(x, [1.0, 0.0, 0.0]))
print("score_shape=", shape(scores))
print("attention_row_sums=", [round(sum(row), 3) for row in attn])
print("context_shape=", shape(context))
print("first_context=", [round(v, 3) for v in context[0]])
print("svd_energy_top2=", round(energy_top2, 3))
print("lora_params=", {"full": full_params, "rank8": rank8_params, "ratio": ratio})
print("checks=", checks)

```

运行后可以看到：

```
dot= 4.0
norms= (3.0, 2.236)
cosine= 0.596
normalized_l2_sq= 0.807
projection_on_x_axis= [1.0, 0.0, 0.0]
score_shape= (3, 3)
attention_row_sums= [1.0, 1.0, 1.0]
context_shape= (3, 2)
first_context= [0.994, 1.203]
svd_energy_top2= 0.991
lora_params= {'full': 16777216, 'rank8': 65536, 'ratio': 0.003906}
checks= {'normalized_dot_equals_cosine': True, 'normalized_l2_relation': True, 'attention_rows_sum_to_one': True, 'cont
```

这个 demo 要看懂五点：

1. dot product、cosine similarity 和归一化后的 L2 距离是同一组向量几何关系的不同表达。
2. attention score 的 shape 来自 $Q @ K^T$ ，不是凭空记忆。
3. softmax 后每一行 attention weight 都应接近 1，表示一个 query token 对所有 value token 的加权分布。
4. top singular values 的平方和占比可以帮助直觉理解低秩近似保留了多少主要能量。
5. LoRA 参数量从 $d_{in} * d_{out}$ 降到 $r * (d_{in} + d_{out})$ ，rank 越小越省参数，但表达能力也越受限。

1.14 面试高频问题

1.14.1 为什么 attention score 是 T by T

回答：

因为 self-attention 中每个 query token 都要和每个 key token 计算相似度。序列长度是 T，所以一共有 T 个 query 和 T 个 key，得

1.14.2 Dot product 和 cosine similarity 有什么区别

回答：

Dot product 同时受方向和向量长度影响。Cosine similarity 会除以两个向量的范数，只看方向相似性。Embedding 检索中如果向量

1.14.3 LoRA 为什么省参数

回答：

LoRA 不直接学习完整的权重更新矩阵 ΔW ，而是把它写成两个低秩矩阵 A 和 B 的乘积。如果原矩阵是 d_{in} by d_{out} ，全量更新需

1.14.4 为什么要关注 gradient norm

回答：

Gradient norm 反映梯度整体尺度。训练中如果 gradient norm 突然升高，可能说明学习率过大、数据异常、数值溢出或 loss mask 有

1.15 常见误区

1. 只记公式，不会推 shape。
2. 把向量长度和方向混为一谈。
3. 以为 cosine similarity 和 dot product 总是一样。
4. 只说 LoRA 省参数，不会算参数量。
5. 认为 attention 是黑盒，不知道它就是相似度矩阵和加权求和。
6. 不知道范数和训练稳定性、检索相似度都有关系。

1.16 小练习

1. 给定 X : $[32, 128, 768]$, W : $[768, 3072]$, 写出 $X @ W$ 的 shape。
2. 给定 Q, K, V : $[B, H, T, d_h]$, 写出 attention score 和 output 的 shape。
3. 计算一个 4096×4096 矩阵的参数量。
4. 如果 LoRA rank 为 16, $d_{in} = d_{out} = 4096$, 计算 LoRA 参数量。
5. 解释为什么 embedding 检索常做归一化。
6. 用 PyTorch 写一个函数, 输入 Q, K, V , 输出 scaled dot-product attention。

1.17 本章总结

线性代数不是大模型之外的数学基础, 而是大模型内部计算的基本语言。

本章最重要的结论:

1. Token hidden state 是向量。
2. Linear layer 是矩阵乘法。
3. Attention 是相似度矩阵加加权求和。
4. Shape 推导是理解 Transformer 的关键能力。
5. 范数连接训练稳定性和向量检索。
6. SVD 和低秩近似帮助理解 LoRA。
7. RAG embedding 检索本质是向量空间最近邻搜索。

面试中讲线性代数, 不要停留在公式定义, 要主动连接到 attention、embedding、LoRA、训练稳定性和系统评估。

第二章：概率论

概率论是理解语言模型训练目标的核心。大模型不是直接输出“正确答案”, 而是在每个位置输出一个 token 概率分布; 预训练是在最大化真实文本序列的概率; 解码是在概率分布上做 greedy、sampling、top-k、top-p; 评估里的 perplexity、本质上也来自概率。

如果概率论不清楚, 很多大模型概念会变成口号: next-token prediction、最大似然、交叉熵、KL、reward model、DPO、calibration、uncertainty、hallucination 都会只停留在名词层面。

本章目标: 用面向 LLM 的方式讲清随机变量、概率分布、条件概率、链式法则、贝叶斯公式、期望、方差、最大似然和语言模型概率分解。

0. 本讲资料边界与第二轮精修口径

本章第二轮精修时, 重点核对了 MIT OCW 6.041 概率系统课程、Stanford CS229 概率与统计复习资料、PyTorch `torch.distributions` 中 categorical distribution 的 logits / probs / sample / log_prob 口径、NumPy `random.choice` 的概率采样参数, 以及前序语言模型、交叉熵、解码策略、评估统计和校准章节。

本章不扩展成完整概率论教材, 而是服务 LLM 面试和工程理解:

1. 重点讲随机变量、离散概率分布、条件概率、链式法则、Bayes、期望、方差、MLE、NLL、PPL、采样和校准。
2. 不展开测度论、连续分布积分推导、马尔可夫链完整理论、贝叶斯网络、MCMC 或统计推断全套证明。
3. 公式必须能落到语言模型: 下一个 token 是随机变量, logits 经过 softmax 变成条件分布, 训练最大化真实 token 条件概率, 解码从分布里选 token。
4. demo 用标准库实现概率计算, 不依赖外部模型或随机 API, 让读者能直接看到每一步概率如何被构造和消费。

2.1 为什么 LLM 离不开概率论

语言模型的输出不是一个 token, 而是整个词表上的概率分布。

例如上下文是:

The capital of France is

模型可能输出：

Paris	0.72
Lyon	0.04
the	0.03
located	0.02

...

训练时，真实下一个 token 是 Paris，模型就应该给它更高概率。

推理时，模型可以选择概率最高的 token，也可以按概率采样，或者只在 top-k / top-p 候选集中采样。

所以大模型的很多问题都是概率问题：

1. 训练：如何提高真实 token 的概率。
2. 推理：如何从概率分布中选择 token。
3. 评估：如何衡量模型给真实文本的概率。
4. 幻觉：为什么高概率文本不等于真实事实。
5. 对齐：如何让概率分布更符合人类偏好。
6. 不确定性：模型什么时候应该表达“不知道”。

2.1.1 关键公式与 LLM 概率速查

下一个 token 可以看成离散随机变量：

$$X_t \in \{1, 2, \dots, V\}$$

其中 V 是词表大小。给定上下文 $x_{1:t-1}$ ，语言模型输出条件概率分布：

$$p_{\theta}(v \mid x_{1:t-1})$$

logits 到概率分布的 softmax：

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^V \exp(z_j)}$$

实际实现中常先减去最大 logit，避免指数溢出：

$$p_i = \frac{\exp(z_i - z_{\max})}{\sum_{j=1}^V \exp(z_j - z_{\max})}$$

条件概率：

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

边缘概率：

$$P(A) = \sum_b P(A, b)$$

自回归链式分解：

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{1:t-1})$$

语言模型最大似然目标：

$$\theta^* = \arg \max_{\theta} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{1:t-1})$$

负对数似然：

$$L_{NLL} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{1:t-1})$$

平均 token loss 和困惑度：

$$\bar{L} = \frac{1}{T} L_{NLL}$$

$$PPL = \exp(\bar{L})$$

离散随机变量的期望：

$$E[X] = \sum_x x P(X = x)$$

方差：

$$\text{Var}(X) = E[X^2] - E[X]^2$$

Bayes 公式：

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}$$

其中：

$$P(E) = P(E \mid H)P(H) + P(E \mid \neg H)P(\neg H)$$

temperature softmax：

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

tau 越小分布越尖锐，**tau** 越大分布越平。概率论面试题的关键不是背这些公式，而是能解释它们分别对应训练、解码、评估、不确定性和幻觉治理中的哪一步。

2.2 随机变量：把不确定结果形式化

随机变量是对不确定结果的数学表示。

在语言模型中，下一个 token 就可以看成一个离散随机变量。

如果词表大小是 V ，那么下一个 token 可能取 V 个值。模型输出一个长度为 V 的概率分布：

$$P(\text{token} = v \mid \text{context})$$

这里的 context 是前面的 token 序列。

比如：

$$P(\text{next_token} = \text{Paris} \mid \text{The capital of France is})$$

面试回答：

在语言模型里，可以把下一个 token 看成随机变量。给定上下文，模型输出这个随机变量在整个词表上的条件概率分布。训练目标就

2.3 概率分布：模型输出的真正对象

概率分布描述每个可能结果发生的概率。

离散分布需要满足两个条件：

1. 每个概率不小于 0。
2. 所有概率加起来等于 1。

神经网络最后通常输出 logits。Logits 不是概率，可以是任意实数。需要经过 softmax 才变成概率分布：

$$\text{prob}_i = \exp(\text{logit}_i) / \sum_j \exp(\text{logit}_j)$$

Softmax 的作用是：

1. 把任意实数变成正数。
2. 让所有输出加起来等于 1。
3. 保留相对大小，logit 越大，对应概率越大。

面试回答：

语言模型最后一层输出 logits，logits 本身不是概率。Softmax 把 logits 转成词表上的概率分布，使每个 token 的概率非负且总和

2.4 联合概率、条件概率和边缘概率

联合概率表示多个事件同时发生的概率。

例如：

$$P(X = x, Y = y)$$

条件概率表示在一个事件已经发生的条件下，另一个事件发生的概率：

$$P(Y = y \mid X = x)$$

条件概率的定义是：

$$P(Y \mid X) = P(X, Y) / P(X)$$

在语言模型中，最重要的是条件概率：

$$P(\text{next token} \mid \text{previous tokens})$$

例如：

$$P(\text{Paris} \mid \text{The capital of France is})$$

边缘概率可以理解为把不关心的变量加总掉。例如：

$$P(X) = \sum_y P(X, Y = y)$$

面试中不需要把边缘概率讲得很抽象。更重要的是理解：语言模型不是只学单个 token 的无条件概率，而是学上下文条件下的 token 分布。

2.5 概率链式法则：自回归语言模型的基础

任意 token 序列的联合概率都可以用链式法则分解。

给定序列：

$$x_1, x_2, \dots, x_T$$

联合概率可以写成：

$$\begin{aligned} &P(x_1, x_2, \dots, x_T) \\ &= P(x_1) * P(x_2 | x_1) * \dots * P(x_T | x_1, \dots, x_{T-1}) \end{aligned}$$

这就是 GPT 类自回归语言模型的概率基础。

模型不需要一次直接建模整个长序列概率，而是每一步建模：

$$P(x_t | x_1, \dots, x_{t-1})$$

面试回答：

自回归语言模型的基础是概率链式法则。任意序列的联合概率都可以分解成一系列条件概率的乘积。GPT 选择从左到右建模，每一步模

2.6 为什么 next-token prediction 是概率建模

Next-token prediction 不是简单分类任务，而是在每个上下文位置建模一个条件概率分布。

训练样本可以写成：

input: $x_1, \dots, x_{t-1}$
label: x_t

模型输出：

$$P(v | x_1, \dots, x_{t-1}) \text{ for every token } v \text{ in vocabulary}$$

训练希望：

$$P(x_t | x_1, \dots, x_{t-1})$$

尽可能大。

这解释了为什么语言模型可以通过大量文本训练：每个 token 位置都提供一个监督信号。

一篇长度为 T 的文本，几乎可以提供 T-1 个 next-token prediction 训练样本。

2.7 最大似然估计：让训练数据更可能出现

最大似然估计，也就是 MLE，目标是找到一组参数，让训练数据出现的概率尽可能大。

假设模型参数是 θ ，训练集有很多 token 序列。MLE 的目标可以直观写成：

choose θ to maximize $P(\text{training data} | \theta)$

对语言模型来说，就是最大化每个真实 token 的条件概率：

$$\text{maximize } \prod_t P_{\theta}(x_t | x_1, \dots, x_{t-1})$$

很多概率相乘会非常小，所以实际优化 log probability：

$$\text{maximize } \sum_t \log P_{\theta}(x_t | x_1, \dots, x_{t-1})$$

训练 loss 通常是负 log likelihood：

$\text{loss} = - \sum_t \log P_{\theta}(x_t \mid x_1, \dots, x_{t-1})$

面试回答：

最大似然估计的目标是让训练数据在模型下的概率尽可能高。对自回归语言模型来说，就是让每个真实下一个 token 的条件概率尽可能高。

2.8 期望：平均意义下的目标

期望可以理解为随机变量的长期平均值。

离散情况下：

$$E[X] = \sum_x P(X = x) * x$$

机器学习里的很多目标都可以看成期望。

例如训练 loss 本质上是数据分布上的平均损失：

$$\text{expected loss} = E_{\text{data}}[\text{loss}(\text{model}, \text{sample})]$$

实际训练时，我们拿不到完整数据分布，只能用 mini-batch 近似这个期望：

$$\text{batch loss} \approx \text{expected loss}$$

这就是为什么 batch size 会影响梯度噪声。batch 越小，对真实期望的估计噪声越大；batch 越大，估计更稳定，但计算和显存成本更高。

面试回答：

训练目标可以看成数据分布上的期望损失。实际训练用 mini-batch loss 近似这个期望，因此 batch size 会影响估计噪声。小 batch

2.9 方差：不确定性和噪声大小

方差衡量随机变量围绕均值波动的程度。

直观上：

1. 方差小，结果集中，稳定。
2. 方差大，结果分散，不稳定。

在大模型训练和评估中，方差很常见：

1. Mini-batch 梯度有方差。
2. 不同 random seed 训练结果有方差。
3. LLM-as-a-judge 评分有方差。
4. 人工评估样本太少时，估计方差大。
5. Sampling 解码会带来输出方差。

这解释了为什么一次实验结果不能轻易下结论。

如果一个方法在 100 条样本上提升 1 个百分点，但样本方差很大，这个提升可能没有统计意义。

面试回答：

方差衡量随机结果的波动程度。训练中 mini-batch 梯度是对全量梯度的估计，会有方差；评估中样本量太小也会导致指标不稳定。所

2.10 贝叶斯公式：根据证据更新信念

贝叶斯公式描述如何根据证据更新概率：

$$P(H \mid E) = P(E \mid H) * P(H) / P(E)$$

其中：

1. H 是假设。
2. E 是证据。

3. $P(H)$ 是先验概率。
4. $P(H | E)$ 是看到证据后的后验概率。

贝叶斯公式在大模型中不一定每天直接写，但它的思想很重要：根据新证据更新判断。

例如 RAG 中，模型原本可能根据语言先验猜答案；加入检索证据后，应该把回答约束到 evidence 上。

再例如评估模型质量时，单个 benchmark 分数只是证据之一。你还要结合人评、bad case、线上反馈和污染检查，更新对模型能力的判断。

面试回答：

贝叶斯公式的核心思想是用证据更新信念。先验表示看到证据前的判断，后验表示结合证据后的判断。在 LLM 场景中，RAG、工具验证

2.11 独立性和条件独立

如果两个事件互不影响，可以说它们独立：

$$P(A, B) = P(A) * P(B)$$

但真实语言中，token 几乎不是独立的。

例如在句子：

The capital of France is Paris

Paris 的概率强烈依赖前面的上下文。语言模型的核心就是建模这种依赖关系。

早期一些简单模型会做独立性或 Markov 假设，例如只看前一个词或前几个词。Transformer 的优势是可以在上下文窗口内建模更长距离依赖，而不是只依赖固定短窗口。

面试回答：

语言中的 token 通常不是独立的，下一个 token 强依赖前面的上下文。自回归语言模型显式建模 $P(\text{next token} | \text{previous tokens})$ 。gram 模型，Transformer 可以通过 self-attention 使用更长范围的上下文信息。

2.12 采样：从概率分布生成文本

推理时，模型输出概率分布后，需要选择下一个 token。

常见策略：

1. Greedy decoding：选概率最高的 token。
2. Sampling：按概率随机采样。
3. Temperature：调整分布尖锐程度。
4. Top-k：只从概率最高的 k 个 token 中采样。
5. Top-p：从累计概率达到 p 的候选集合中采样。

Temperature 的直觉：

1. 温度低，分布更尖锐，输出更确定。
2. 温度高，分布更平，输出更多样但更不稳定。

面试回答：

语言模型每一步输出概率分布，解码策略决定如何从分布中选 token。Greedy 每次选最高概率，稳定但可能重复；sampling 增加多样性，k 和 top-p 限制候选集合，减少低概率 token 带来的不稳定。

2.13 概率高不等于事实真

这是理解 hallucination 的关键。

语言模型学的是：在当前上下文和训练分布下，什么 token 更可能出现。它不是直接优化事实真伪。

一个回答可能语言上很高概率，但事实是错的。原因包括：

1. 训练数据里有错误信息。
2. 模型缺少某个事实。
3. 上下文证据不足。
4. SFT/RLHF 奖励了流畅和完整回答。
5. 解码策略让模型继续生成看似合理的文本。

面试回答：

语言模型输出的是高概率文本，不等于经过事实验证的真命题。Next-token prediction 优化的是训练分布上的条件概率，而不是 true

2.14 校准：概率能否反映真实可信度

Calibration 关注模型的置信度是否可靠。

如果模型对一批问题都给出 80% 置信度，那么理想情况下，这些回答中大约 80% 应该正确。

大模型常常存在校准问题：

1. 错了也很自信。
2. 对简单问题过度保守。
3. 对分布外问题不知道自己不知道。
4. 经过 SFT/RLHF 后语言更自信，但概率未必更可信。

在高风险场景中，校准非常重要。模型不确定时应该检索、调用工具、请求澄清或拒答，而不是自信编造。

面试回答：

校准衡量模型概率或置信度是否和真实正确率匹配。LLM 可能语言上很自信但事实错误，因此高风险场景不能只相信模型表达的置信。

2.15 最小可运行概率审计 demo

下面用纯 Python 写一个概率审计 demo，覆盖 bigram 条件概率、softmax、序列 log probability、NLL / PPL、Bayes 更新、期望方差、top-k / top-p 和可复现采样。

```
from collections import Counter, defaultdict
from math import exp, log
from random import Random

texts = [
    "I love machine learning",
    "I love deep learning",
    "machine learning is useful",
    "deep learning is powerful",
]

counts = defaultdict(Counter)
start = "<s>"
end = "</s>"

for text in texts:
    tokens = [start] + text.split() + [end]
    for current_token, next_token in zip(tokens, tokens[1:]):
        counts[current_token][next_token] += 1

def next_token_distribution(current_token):
    next_counts = counts[current_token]
    total = sum(next_counts.values())
    if total == 0:
        return {}
```

```

    return {
        token: count / total
        for token, count in next_counts.items()
    }

def sequence_logprob(tokens):
    seq = [start] + tokens + [end]
    logprob = 0.0
    step_probs = []
    for current_token, next_token in zip(seq, seq[1:]):
        dist = next_token_distribution(current_token)
        prob = dist.get(next_token, 0.0)
        step_probs.append((current_token, next_token, round(prob, 3)))
        logprob += log(prob)
    return logprob, step_probs

def softmax(logits, temperature=1.0):
    scaled = [value / temperature for value in logits]
    max_logit = max(scaled)
    exps = [exp(value - max_logit) for value in scaled]
    total = sum(exps)
    return [value / total for value in exps]

def entropy(probs):
    return -sum(prob * log(prob) for prob in probs if prob > 0)

def top_k(tokens, probs, k):
    ranked = sorted(zip(tokens, probs), key=lambda item: item[1], reverse=True)
    return [(token, round(prob, 3)) for token, prob in ranked[:k]]

def top_p(tokens, probs, p):
    ranked = sorted(zip(tokens, probs), key=lambda item: item[1], reverse=True)
    kept = []
    total = 0.0
    for token, prob in ranked:
        kept.append((token, prob))
        total += prob
        if total >= p:
            break
    return [(token, round(prob, 3)) for token, prob in kept], round(total, 3)

def sample_from(tokens, probs, seed=7):
    rng = Random(seed)
    threshold = rng.random()
    cumulative = 0.0
    for token, prob in zip(tokens, probs):
        cumulative += prob
        if threshold <= cumulative:
            return token, round(threshold, 3)

```

```

    return tokens[-1], round(threshold, 3)

def expectation(values, probs):
    return sum(value * prob for value, prob in zip(values, probs))

def variance(values, probs):
    mean = expectation(values, probs)
    second_moment = sum(value * value * prob for value, prob in zip(values, probs))
    return second_moment - mean * mean

vocab = ["Paris", "Lyon", "the", "located"]
logits = [2.0, 1.1, 0.8, 0.1]
probs = softmax(logits)
true_token = "Paris"
true_prob = probs[vocab.index(true_token)]
nll = -log(true_prob)

seq_logprob, step_probs = sequence_logprob(["I", "love", "machine", "learning"])
avg_nll = -seq_logprob / len(step_probs)
ppl = exp(avg_nll)

prior = 0.6
evidence_given_true = 0.85
evidence_given_false = 0.25
evidence_prob = evidence_given_true * prior + evidence_given_false * (1 - prior)
posterior = evidence_given_true * prior / evidence_prob

length_values = [10, 20, 30]
length_probs = [0.2, 0.5, 0.3]
length_mean = expectation(length_values, length_probs)
length_var = variance(length_values, length_probs)

top_p_tokens, top_p_mass = top_p(vocab, probs, 0.8)
top_p_raw = [(token, prob) for token, prob in zip(vocab, probs) if token in {name for name, _ in top_p_tokens}]
top_p_total = sum(prob for _, prob in top_p_raw)
top_p_probs = [prob / top_p_total for _, prob in top_p_raw]
sampled_token, sample_threshold = sample_from([token for token, _ in top_p_raw], top_p_probs)

checks = {
    "softmax_sums_to_one": abs(sum(probs) - 1.0) < 1e-9,
    "posterior_uses_evidence": posterior > prior,
    "ppl_from_avg_nll": round(exp(avg_nll), 3) == round(ppl, 3),
    "top_p_mass_enough": top_p_mass >= 0.8,
    "variance_non_negative": length_var >= 0,
}

print("dist_I=", next_token_distribution("I"))
print("dist_learning=", next_token_distribution("learning"))
print("step_probs=", step_probs)
print("sequence_logprob=", round(seq_logprob, 3))
print("avg_nll=", round(avg_nll, 3), "ppl=", round(ppl, 3))
print("softmax_probs=", {token: round(prob, 3) for token, prob in zip(vocab, probs)})

```

```

print("true_token_nll=", round(nll, 3), "entropy=", round(entropy(probs), 3))
print("top_k=", top_k(vocab, probs, 2))
print("top_p=", top_p_tokens, "mass=", top_p_mass)
print("top_p_sample=", sampled_token, "threshold=", sample_threshold)
print("bayes_posterior=", round(posterior, 3))
print("length_mean_var=", (round(length_mean, 3), round(length_var, 3)))
print("checks=", checks)

```

运行后可以看到:

```

dist_I= {'love': 1.0}
dist_learning= {'</s>': 0.5, 'is': 0.5}
step_probs= [('<s>', 'I', 0.5), ('I', 'love', 1.0), ('love', 'machine', 0.5), ('machine', 'learning', 1.0), ('learning', 'the', 0.5)]
sequence_logprob= -2.079
avg_nll= 0.416 ppl= 1.516
softmax_probs= {'Paris': 0.538, 'Lyon': 0.219, 'the': 0.162, 'located': 0.081}
true_token_nll= 0.619 entropy= 1.164
top_k= [('Paris', 0.538), ('Lyon', 0.219)]
top_p= [('Paris', 0.538), ('Lyon', 0.219), ('the', 0.162)] mass= 0.919
top_p_sample= Paris threshold= 0.324
bayes_posterior= 0.836
length_mean_var= (21.0, 49.0)
checks= {'softmax_sums_to_one': True, 'posterior_uses_evidence': True, 'ppl_from_avg_nll': True, 'top_p_mass_enough': True}

```

这个 demo 要看懂五点:

1. bigram 只是最小条件概率模型, GPT 的区别是用完整上下文和神经网络参数化更复杂的条件分布。
2. 序列概率来自每一步条件概率的乘积, 工程上用 log probability 求和, 避免概率连乘下溢。
3. NLL 是真实 token 概率的负 log, PPL 是平均 NLL 的指数形式。
4. top-k / top-p 不是改变模型本身, 而是在解码时限制候选集合再采样。
5. Bayes、期望和方差不是抽象公式, 它们对应证据更新、平均成本 / 长度估计和评估不稳定性。

2.16 面试高频问题

2.16.1 为什么语言模型可以只预测下一个 token

回答:

根据概率链式法则, 任意 token 序列的联合概率都可以分解成一系列条件概率的乘积。自回归语言模型选择从左到右建模这些条件概率。

2.16.2 最大似然和交叉熵是什么关系

回答:

最大似然是让训练数据在模型下的概率尽可能高。对分类或语言模型来说, 最大化真实标签的 log probability, 等价于最小化 negative log likelihood。在交叉熵标签下, 这和最小化交叉熵是等价的。

2.16.3 为什么 batch size 会影响训练稳定性

回答:

训练目标是数据分布上的期望损失, 但实际用 mini-batch 估计。batch size 小时, 估计噪声和梯度方差更大; batch size 大时, 梯度估计更准确。

2.16.4 为什么概率高不代表事实正确

回答:

语言模型优化的是训练分布上的 token 条件概率, 而不是事实验证。一个错误说法如果在训练分布或当前上下文中看起来语言上合理, 模型就会赋予高概率。

2.17 常见误区

1. 把概率最高理解成事实正确。
2. 只会说 next-token prediction, 不会讲链式法则。
3. 知道最大似然, 但讲不清为什么取 log。
4. 把 batch loss 当成真实总体 loss。
5. 忽略评估结果的方差和样本量。
6. 认为 temperature 可以解决幻觉。
7. 把模型输出的自信语气当成可靠置信度。

2.18 小练习

1. 用自己的话解释随机变量和概率分布。
2. 写出长度为 4 的 token 序列的自回归概率分解。
3. 解释为什么最大化概率乘积通常要改成最大化 log probability 之和。
4. 用 bigram 例子计算 $P(\text{learning} \mid \text{machine})$ 。
5. 解释 batch size 为什么会影响梯度噪声。
6. 说明 greedy decoding、sampling、top-k、top-p 的区别。
7. 举一个“高概率但不真实”的语言模型输出例子。

2.19 本章总结

概率论是语言模型训练和生成的底层语言。

本章最重要的结论：

1. 下一个 token 可以看成随机变量。
2. 模型输出的是词表上的条件概率分布。
3. 自回归建模来自概率链式法则。
4. 最大似然让训练数据在模型下更可能出现。
5. 训练 loss 是对数据分布期望损失的 mini-batch 近似。
6. 方差解释了训练和评估为什么不稳定。
7. 解码是在概率分布上选择 token。
8. 概率高不等于事实真, 幻觉需要额外治理。

面试中讲概率论, 不要只背公式。要主动把随机变量、条件概率、链式法则、MLE、期望方差和 LLM 的训练、解码、评估、幻觉联系起来。

第三章：信息论

信息论是理解大模型 loss、perplexity、KL 约束、偏好优化和评估指标的关键。概率论告诉我们模型输出的是概率分布, 信息论进一步告诉我们: 这个分布有多不确定, 模型预测真实 token 需要付出多少代价, 两个分布差异有多大, 以及为什么低 loss 不一定等于好模型。

很多面试回答会卡在这里: 知道 cross entropy 是分类损失, 但讲不清它和最大似然的关系; 知道 KL 散度, 但讲不清为什么 RLHF 需要 KL penalty; 知道 perplexity, 但不知道它为什么受 tokenizer 影响; 知道 DPO 有 reference model, 但讲不清它为什么像一种隐式 KL 约束。

本章目标: 讲清熵、交叉熵、KL 散度、互信息、perplexity, 以及它们如何服务于 LLM 训练、对齐、解码、评估和安全。

0. 本讲资料边界与第二轮精修口径

第二轮精修时, 本章按 WRITING_PLAN.md 先核对信息论基础定义、主流深度学习框架中的 cross entropy 口径、语言模型 perplexity 评估说明、RLHF 中 reference / KL 约束, 以及 DPO 与带 KL 约束偏好优化的关系。

本章聚焦大模型算法岗最常用的信息论工具:

1. 单个 token 的信息量、NLL 和平均 token loss。
2. 熵、交叉熵、KL 散度和三者关系。
3. Perplexity 的定义、直觉和 tokenizer / 数据集边界。
4. 互信息在表示学习、多模态对齐、RAG 和数据选择中的直觉。
5. KL penalty、reference model 和偏好优化中的分布约束。

本章不展开完整编码定理、信道容量、率失真理论、信息瓶颈完整推导、变分推断全套数学或 PPO / DPO 的完整证明。面试中更重要的是：能把公式说回 LLM loss、PPL、RLHF、DPO、RAG 证据质量和安全信息粒度，而不是孤立背公式。

3.1 为什么大模型需要信息论

语言模型每一步输出一个概率分布。信息论关心几个问题：

1. 这个分布有多不确定？这是熵。
2. 用模型分布解释真实数据要付出多少代价？这是交叉熵。
3. 模型分布和目标分布差多远？这是 KL 散度。
4. 模型平均每一步有多少有效候选？这是 perplexity 的直觉。
5. 一个变量知道后能减少另一个变量多少不确定性？这是互信息。

这些问题直接出现在大模型中：

1. Pretraining 用 cross entropy loss。
2. Perplexity 衡量语言建模困惑程度。
3. RLHF 用 KL penalty 限制 policy 偏离 reference。
4. DPO 中 reference model 起到类似锚点作用。
5. Mutual information 直觉可以帮助理解 representation 和 modality alignment。

3.1.1 关键公式与 LLM 信息论速查

下面默认使用自然对数。此时信息量、loss 和熵的单位可以理解为 nat；如果使用 $\log_2$ ，单位是 bit。大多数深度学习框架里的 cross entropy / NLL 默认按自然对数计算，所以语言模型里常见的 perplexity 写成 $\exp(\text{平均 token loss})$ 。

单个事件的信息量：

$$I(x) = -\log p(x)$$

如果事件概率越小， $I(x)$ 越大。语言模型里真实 token 概率越低，单 token NLL 越大。

离散分布的熵：

$$H(p) = -\sum_x p(x) \log p(x)$$

$H(p)$ 衡量分布本身的不确定性。分布越均匀，熵越高；分布越集中，熵越低。

真实分布 p 和模型分布 q 的交叉熵：

$$H(p, q) = -\sum_x p(x) \log q(x)$$

如果标签是 one-hot，真实 token 为 y ，则交叉熵退化成：

$$L_{\text{token}} = -\log q(y)$$

训练集上的平均 token loss 常写成：

$$L_{\text{avg}} = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i | c_i)$$

其中 c_i 是第 i 个预测位置可见的上下文, x_i 是真实下一个 token, N 是参与 loss 的 token 数。

Perplexity:

$$\text{PPL} = \exp(L_{\text{avg}})$$

它可以粗略理解为平均每个位置的有效候选数, 但只有在 tokenizer、语料、截断方式和 loss 口径一致时才适合比较。

KL 散度:

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

KL 非负、非对称, 不是普通距离。 $D_{\text{KL}}(p \parallel q)$ 强调 q 不要漏掉 p 的高概率区域; 换方向后优化偏好会变。

交叉熵分解:

$$H(p, q) = H(p) + D_{\text{KL}}(p \parallel q)$$

当真实数据分布 p 固定时, $H(p)$ 对模型参数是常数, 所以最小化交叉熵等价于把模型分布 q 推近 p 。

互信息:

$$I(X; Y) = H(X) - H(X | Y)$$

等价地:

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

互信息衡量知道 Y 后能减少多少关于 X 的不确定性。RAG 里, 一个 chunk 和问题词面相似不够; 它还要真正减少答案的不确定性。

RLHF 中常见的 KL 约束直觉:

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}}[r(y)] - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

beta 越大, policy 越不容易远离 reference; beta 越小, reward hacking 和语言退化风险更高。

DPO 的核心比较量可以写成:

$$z = \beta[(\log \pi_{\theta}(y_w | x) - \log \pi_{\text{ref}}(y_w | x)) - (\log \pi_{\theta}(y_l | x) - \log \pi_{\text{ref}}(y_l | x))]$$

$$L_{\text{DPO}} = -\log \sigma(z)$$

这里 y_w 是 chosen response, y_l 是 rejected response。这个式子展示了 reference model 为什么是偏好变化的锚点, 而不只是一个可有可无的 baseline。

3.2 信息量：越意外，信息越多

一个事件的信息量和它发生的概率有关。

直觉是：越罕见的事件，一旦发生，信息量越大。

常见定义是：

$$I(x) = -\log p(x)$$

其中 $p(x)$ 是事件 x 发生的概率， $I(x)$ 是观察到 x 后获得的信息量。

如果一个 token 的概率很高，那么看到它并不意外，信息量小。如果一个 token 的概率很低，那么看到它很意外，信息量大。

这和语言模型 loss 直接相关。真实 token 的 loss 通常就是：

$$L_{\text{token}} = -\log p_{\theta}(y | c)$$

其中 c 是上下文， y 是真实下一个 token， $p_{\theta}(y | c)$ 是模型给真实 token 的条件概率。

如果模型给真实 token 的概率高，loss 小；如果给真实 token 的概率低，loss 大。

面试回答：

信息量可以理解为事件的意外程度，通常写成负 log 概率。语言模型训练中，真实 token 的 negative log likelihood 就是模型看到

3.3 熵：分布的不确定性

熵衡量一个概率分布的不确定性。

离散分布的熵可以写成：

$$H(p) = -\sum_x p(x) \log p(x)$$

其中求和遍历所有可能事件或词表 token。概率为 0 的项通常按极限理解为 0。

直觉上：

1. 分布越集中，熵越低。
2. 分布越均匀，熵越高。

例如一个模型在某个位置非常确定：

Paris 0.99
other tokens 0.01

这个分布熵很低。

如果模型很不确定：

token A 0.20
token B 0.18
token C 0.17
...

熵就更高。

注意：高熵不一定坏。开放式创作任务本来就可能多个合理答案；事实问答中，如果模型应该确定却高熵，可能说明知识不足或上下文不够。

面试回答：

熵衡量概率分布的不确定性。分布越集中，熵越低；分布越平，熵越高。在语言模型里，某些位置应该低熵，比如事实补全；某些开

3.4 交叉熵：用模型分布解释真实分布的代价

交叉熵衡量：如果真实分布是 P ，但我们用模型分布 Q 来编码或预测，平均代价是多少。

可以写成：

$$H(p, q) = - \sum_x p(x) \log q(x)$$

其中 p 是真实分布， q 是模型分布。交叉熵越低，表示模型在真实分布高概率的位置也给了更高概率。

在监督学习中，真实标签通常是 one-hot。也就是说，真实 token 的概率是 1，其他 token 是 0。

此时交叉熵会简化为：

$$L_{\text{token}} = - \log q(y)$$

其中 y 是真实类别或真实 token。

这就是语言模型常见的 token-level loss。

面试回答：

交叉熵可以理解为用模型分布 Q 去解释真实分布 P 的平均代价。在 one-hot 标签下，真实类别概率为 1，所以交叉熵等于真实 token

3.5 交叉熵和最大似然的关系

最大似然是让训练数据在模型下的概率尽可能高。

对语言模型来说，目标是最大化：

$$\prod_{t=1}^T p_{\theta}(x_t \mid x_{1:t-1})$$

取 log 后变成：

$$\sum_{t=1}^T \log p_{\theta}(x_t \mid x_{1:t-1})$$

训练通常最小化负 log probability：

$$- \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{1:t-1})$$

如果只对部分 token 计算 loss，例如 chat SFT 中只对 assistant token 计算 loss，就把求和限制在参与训练的 token 集合上。

这和 one-hot 标签下的交叉熵一致。

面试回答：

最大似是最大化训练数据中真实 token 的概率。取 log 后，最大化 log probability 等价于最小化 negative log likelihood。在 one-hot 标签下，negative log likelihood 和 cross entropy 是同一个形式，所以语言模型训练常说最小化交叉熵。

3.6 KL 散度：两个分布的差异

KL 散度衡量用一个分布 Q 近似另一个分布 P 时的额外代价。

常见形式：

$$D_{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

当 $p(x) > 0$ 但 $q(x) = 0$ 时，KL 会发散；工程中通常要依赖 softmax、label smoothing、clip 或数值稳定处理避免不可计算。

直觉上，如果 Q 和 P 很接近，KL 小；如果 Q 在 P 认为重要的地方给了很低概率，KL 会变大。

KL 有几个重要性质：

1. KL 不小于 0。
2. KL 等于 0 表示两个分布相同。
3. KL 不是对称的， $KL(P \parallel Q)$ 和 $KL(Q \parallel P)$ 通常不同。
4. KL 不是严格意义上的距离。

为什么 KL 不对称重要？

因为 $KL(P \parallel Q)$ 更强调 Q 不要漏掉 P 的高概率区域；而 $KL(Q \parallel P)$ 更强调 Q 不要把概率放到 P 不支持的区域。不同方向会导致不同优化行为。

面试回答：

KL 散度衡量两个概率分布的差异，可以理解为用 Q 近似 P 时多付出的信息代价。KL 非负，但不对称，所以不是普通距离。在大模型中

3.7 交叉熵、熵和 KL 的关系

交叉熵可以分解为真实分布自身的不确定性，加上模型分布和真实分布的差异：

$$H(p, q) = H(p) + D_{KL}(p \parallel q)$$

这个关系非常重要。

如果真实数据分布 P 固定，那么 $H(P)$ 是常数。最小化交叉熵，就等价于最小化 $KL(P \parallel Q)$ ，也就是让模型分布 Q 接近真实分布 P 。

面试回答：

交叉熵可以分成两部分：真实分布自身的熵，以及真实分布和模型分布之间的 KL。数据分布固定时，真实熵是常数，所以最小化交叉

3.8 Perplexity：平均每步有多少有效选择

Perplexity 通常是 average negative log likelihood 的指数形式。

如果平均 token loss 是：

$$L_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N L_i$$

那么 perplexity 是：

$$\text{PPL} = \exp(L_{\text{avg}})$$

直觉上，perplexity 可以理解为模型平均每一步有多少个有效候选。

例如：

1. Perplexity 越低，模型越不困惑。
2. Perplexity 越高，模型越难预测真实文本。

但 perplexity 有很多边界：

1. 不同 tokenizer 下不一定可比。
2. 低 perplexity 不一定代表聊天质量好。
3. 对齐后的模型可能牺牲部分 perplexity，但用户体验更好。
4. 如果验证集被污染，perplexity 会失真。
5. Perplexity 不能直接衡量事实性、安全性和工具调用能力。

面试回答：

Perplexity 是平均 token loss 的指数，可以理解为模型平均每一步的有效候选数。它适合衡量语言建模能力，但不同 tokenizer、数

3.9 为什么低 loss 不一定代表好模型

低 cross entropy loss 表示模型更擅长预测验证集 token，但这不等于所有能力更强。

原因包括：

1. 验证集不代表真实用户任务。
2. 数据污染让 loss 虚低。
3. 模型可能学到模板和风格，而不是推理能力。
4. SFT/RLHF 后的交互质量不完全由 pretraining loss 决定。
5. 安全、诚实、引用和工具调用不是单纯 next-token loss 能完全衡量。

面试回答：

Loss 低说明模型更好地拟合了评估数据的 token 分布，但不保证真实任务能力更强。如果评估集不代表目标任务、存在污染，或者我

3.10 RLHF 中的 KL penalty

RLHF 中常见目标会加入 KL penalty，约束 policy model 不要偏离 reference model 太远。

直觉上，reward model 会给回答打分，但如果只最大化 reward，policy 可能利用 reward model 漏洞，出现 reward hacking、语言退化或过度拒答。

KL penalty 的作用是：

1. 限制 policy 分布偏离 reference。
2. 保持语言质量和基础能力。
3. 降低 reward hacking 风险。
4. 让优化过程更稳定。

可以直观写成：

$$J = R - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

其中 beta 控制约束强度。

1. beta 太小，约束弱，模型可能乱跑。
2. beta 太大，约束强，模型学不动。

面试回答：

RLHF 中 KL penalty 用来限制当前 policy 不要偏离 reference model 太远。因为 reward model 只是人类偏好的 proxy，如果无约束地

3.11 DPO 和 KL 约束的关系

DPO 不显式训练 reward model，也不跑 PPO，但它和带 KL 约束的偏好优化有理论联系。

DPO 使用 preference pair:

prompt, chosen response, rejected response

它比较 policy model 和 reference model 对 chosen/rejected 的相对 log probability 差异。

直觉上，DPO 希望 policy 相比 reference 更偏好 chosen，而不是 rejected。

Reference model 的作用类似锚点：

1. 防止 policy 任意漂移。
2. 提供“偏离原模型多少”的参照。
3. 让偏好优化不只是简单提高 chosen 概率。

面试回答：

DPO 可以看作把带 KL 约束的偏好优化改写成直接基于 chosen/rejected log probability 的目标。它不显式加入 PPO 式 KL penalty,

3.12 互信息：知道一个变量能减少多少不确定性

互信息衡量两个变量之间共享的信息量。

直觉上，如果知道 X 能大幅减少对 Y 的不确定性，那么 X 和 Y 的互信息高。

在大模型和多模态中，互信息直觉很有用：

1. 图文对齐希望图像表示和文本表示共享语义信息。
2. RAG 希望检索文档对答案有高信息量。
3. Prompt 中无关上下文和答案互信息低，可能只是噪声。
4. 表示学习希望 hidden state 保留对任务有用的信息。

面试中不一定要求推导互信息公式，更重要的是直觉。

常见公式是：

$$I(X; Y) = H(X) - H(X | Y)$$

也可以写成联合分布和边缘分布的 KL：

$$I(X; Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

所以互信息高，不等于表面相似度高，而是一个变量确实能减少另一个变量的不确定性。

面试回答：

互信息衡量一个变量能减少另一个变量多少不确定性。多模态对齐可以理解为让图像和文本表示共享更多语义信息；RAG 中好的检索结果

3.13 信息论视角理解 RAG

RAG 可以看成给模型提供额外证据，降低回答不确定性。

一个好检索结果应该：

1. 和问题相关。
2. 包含回答所需证据。
3. 减少模型对答案的不确定性。
4. 不引入冲突或噪声。

如果检索到的 chunk 和问题词面相似，但不包含答案证据，那么它对答案的信息量低，甚至会干扰模型。

这解释了为什么 RAG 不能只看 embedding similarity，还要看：

1. Recall@k。
2. Rerank 后的 evidence quality。
3. Context precision。
4. Citation accuracy。
5. Faithfulness。

3.14 信息论视角理解安全和拒答

安全场景中，信息论也有直觉价值。

例如模型面对高风险请求时，不应该提供会显著降低危险操作门槛的信息。安全拒答不是拒绝所有内容，而是控制输出中高风险信息的粒度。

对于资料不足的问题，模型应该表达不确定性，而不是输出看似高信息量但无依据的细节。

面试回答：

安全回答不是简单减少信息量，而是控制危险信息和无依据信息。对正常教育性请求，模型可以提供高层解释；对高风险可操作请求，

3.15 最小可运行信息论审计 demo

下面用纯 Python 展示本章几个关键关系：熵比较、交叉熵、KL 分解、one-hot NLL、平均 NLL / PPL、互信息、KL penalty 以及 RAG 证据降低不确定性的直觉。

```
import math
```

```
EPS = 1e-12
```

```
def entropy(dist):  
    return -sum(p * math.log(p) for p in dist.values() if p > 0)
```

```
def cross_entropy(p, q):  
    return -sum(px * math.log(max(q.get(x, 0.0), EPS)) for x, px in p.items() if px > 0)
```

```
def kl(p, q):  
    return sum(px * math.log(px / max(q.get(x, 0.0), EPS)) for x, px in p.items() if px > 0)
```

```
def mutual_information(joint):  
    px = {}  
    py = {}  
    for (x, y), p in joint.items():  
        px[x] = px.get(x, 0.0) + p  
        py[y] = py.get(y, 0.0) + p  
  
    total = 0.0  
    for (x, y), p in joint.items():  
        if p > 0:  
            total += p * math.log(p / (px[x] * py[y]))  
    return total
```

```

peaked = {"A": 0.92, "B": 0.04, "C": 0.03, "D": 0.01}
uniform = {"A": 0.25, "B": 0.25, "C": 0.25, "D": 0.25}

true_dist = {"Paris": 0.7, "Lyon": 0.2, "Marseille": 0.1}
model_good = {"Paris": 0.6, "Lyon": 0.25, "Marseille": 0.15}
model_bad = {"Paris": 0.2, "Lyon": 0.3, "Marseille": 0.5}

h_true = entropy(true_dist)
ce_good = cross_entropy(true_dist, model_good)
ce_bad = cross_entropy(true_dist, model_bad)
kl_good = kl(true_dist, model_good)

one_hot = {"Paris": 1.0, "Lyon": 0.0, "Marseille": 0.0}
one_hot_nll = cross_entropy(one_hot, model_good)

token_probs = [0.6, 0.75, 0.5, 0.8]
nlls = [-math.log(p) for p in token_probs]
avg_nll = sum(nlls) / len(nlls)
ppl = math.exp(avg_nll)

joint = {
    ("image_cat", "text_cat"): 0.45,
    ("image_cat", "text_dog"): 0.05,
    ("image_dog", "text_cat"): 0.10,
    ("image_dog", "text_dog"): 0.40,
}
mi = mutual_information(joint)

ref = {"helpful": 0.55, "refuse": 0.25, "unsafe_detail": 0.20}
policy = {"helpful": 0.70, "refuse": 0.25, "unsafe_detail": 0.05}
kl_policy_ref = kl(policy, ref)
reward = 1.35
beta = 0.2
objective = reward - beta * kl_policy_ref

without_doc = {"correct": 0.45, "wrong_a": 0.35, "wrong_b": 0.20}
with_doc = {"correct": 0.85, "wrong_a": 0.10, "wrong_b": 0.05}

checks = {
    "entropy_order": entropy(uniform) > entropy(peaked),
    "ce_decomposition": abs(ce_good - (h_true + kl_good)) < 1e-9,
    "better_model_lower_ce": ce_good < ce_bad,
    "one_hot_nll": abs(one_hot_nll + math.log(model_good["Paris"])) < 1e-9,
    "ppl_positive": ppl > 1.0,
    "mutual_information_positive": mi > 0,
    "kl_non_negative": kl_policy_ref >= 0,
    "evidence_reduces_entropy": entropy(with_doc) < entropy(without_doc),
}

print("entropy_peaked=", round(entropy(peaked), 3))
print("entropy_uniform=", round(entropy(uniform), 3))
print("h_true=", round(h_true, 3), "ce_good=", round(ce_good, 3), "kl_good=", round(kl_good, 3))
print("ce_bad=", round(ce_bad, 3), "one_hot_nll=", round(one_hot_nll, 3))
print("avg_nll=", round(avg_nll, 3), "ppl=", round(ppl, 3))
print("mutual_information=", round(mi, 3))

```

```
print("kl_policy_ref=", round(kl_policy_ref, 3), "kl_penalized_objective=", round(objective, 3))
print("entropy_without_doc=", round(entropy(without_doc), 3), "entropy_with_doc=", round(entropy(with_doc), 3))
print("checks=", checks)
```

预期输出：

```
entropy_peaked= 0.357
entropy_uniform= 1.386
h_true= 0.802 ce_good= 0.825 kl_good= 0.023
ce_bad= 1.437 one_hot_nll= 0.511
avg_nll= 0.429 ppl= 1.535
mutual_information= 0.275
kl_policy_ref= 0.099 kl_penalized_objective= 1.33
entropy_without_doc= 1.049 entropy_with_doc= 0.518
checks= {'entropy_order': True, 'ce_decomposition': True, 'better_model_lower_ce': True, 'one_hot_nll': True, 'ppl_posit
```

这里要看懂：

1. $\text{entropy_uniform} > \text{entropy_peaked}$ ，说明越均匀越不确定。
2. $\text{ce_good} = h_true + \text{kl_good}$ ，验证交叉熵分解。
3. $\text{ce_good} < \text{ce_bad}$ ，说明更接近真实分布的模型交叉熵更低。
4. one_hot_nll 等于真实类别概率的负 $\log$ 。
5. $\text{ppl} = \exp(\text{avg_nll})$ ，说明 PPL 来自平均 token NLL。
6. $\text{mutual_information} > 0$ ，说明图文变量存在共享信息。
7. kl_policy_ref 给 KL penalty 提供约束量。
8. $\text{entropy_with_doc} < \text{entropy_without_doc}$ ，说明高质量证据能降低答案分布不确定性。

语言模型中，这些计算会扩展到词表维度、batch 维度和序列维度；工程实现会由 `CrossEntropyLoss`、`mask`、`reduction` 和 `tokenizer` 共同决定最终 loss 口径。

3.16 面试高频问题

3.16.1 交叉熵和 KL 有什么关系

回答：

交叉熵可以分解为真实分布的熵加上真实分布到模型分布的 KL。数据分布固定时，真实熵是常数，所以最小化交叉熵等价于让模型分

3.16.2 Perplexity 是什么

回答：

Perplexity 是平均 token loss 的指数形式，可以直观理解为模型平均每一步有多少有效候选。它越低说明模型越不困惑，但不同 tok

3.16.3 为什么 RLHF 需要 KL penalty

回答：

Reward model 只是人类偏好的 proxy。如果 policy 无约束地最大化 reward，可能出现 reward hacking、语言质量退化或过度拒答。

3.16.4 DPO 为什么需要 reference model

回答：

Reference model 给 DPO 提供偏好变化的锚点。DPO 不只是让 policy 提高 chosen 的概率，而是让 policy 相对 reference 更偏好 ch

3.16.5 为什么 perplexity 不能跨 tokenizer 随便比较

回答：

Perplexity 是平均 token NLL 的指数。Tokenizer 会改变文本被切成多少个 token，也会改变每个 token 的预测难度，所以不同 tok

3.16.6 KL 散度为什么不是普通距离

回答：

KL 非负，但不对称， $KL(p||q)$ 和 $KL(q||p)$ 的优化含义不同。如果 p 在某个区域有概率而 q 给 0，KL 还会发散。因此 KL 更适合解释

3.16.7 互信息在 RAG 里怎么理解

回答：

互信息衡量一个变量能减少另一个变量多少不确定性。RAG 中，一个 chunk 和 query 词面相似不够，关键是它能否减少答案的不确定

3.17 常见误区

1. 把交叉熵只理解成分类 API，不理解 negative log probability。
2. 以为 KL 是对称距离。
3. 以为 perplexity 可以直接比较所有模型。
4. 认为低 loss 就一定更 helpful。
5. 认为 KL penalty 越大越安全。
6. 认为 DPO 没有 reward model 就没有约束。
7. 把互信息当成必须背公式，而不是理解“减少不确定性”的直觉。
8. 脱离 tokenizer、数据集和 loss mask 比较 perplexity。

3.18 小练习

1. 如果真实 token 概率是 0.8，计算它的 negative log likelihood。
2. 如果平均 loss 是 2.0，计算 perplexity。
3. 解释为什么交叉熵等于真实分布熵加 KL。
4. 举例说明 KL 为什么不对称。
5. 说明为什么低 perplexity 不一定代表聊天体验好。
6. 用自己的话解释 RLHF 中 KL penalty 的作用。
7. 用自己的话解释 DPO 中 reference model 的作用。
8. 用纯 Python 验证 $H(p,q)=H(p)+KL(p||q)$ 。
9. 构造一个 toy 联合分布，计算互信息并解释它代表什么。

3.19 本章总结

信息论把概率分布和学习目标联系起来。

本章最重要的结论：

1. 信息量是负 log 概率，真实 token 概率越低，loss 越高。
2. 熵衡量分布不确定性。
3. 交叉熵衡量用模型分布解释真实分布的平均代价。
4. One-hot 标签下，交叉熵就是真实 token 的 negative log probability。
5. KL 衡量分布差异，但不是对称距离。
6. Perplexity 是平均 token loss 的指数形式。
7. RLHF 中 KL penalty 用来限制 policy 偏离 reference。
8. DPO 的 reference model 也起到类似锚点作用。
9. 互信息直觉可以帮助理解多模态对齐、RAG 和表示学习。

面试中讲信息论，要主动连接 cross entropy、MLE、perplexity、KL penalty、DPO、RAG 和安全，而不是只背公式。

第四章：优化基础

优化是大模型训练的发动机。概率论和信息论告诉我们 loss 是什么，优化告诉我们如何通过梯度一步步降低 loss。大模型训练中的很多高频问题，本质上都是优化问题：learning rate 为什么重要，warmup 为什么必要，AdamW 为什么常用，gradient clipping 解决什么，loss spike 怎么排查，batch size 为什么影响稳定性。

很多候选人会背“用 AdamW、warmup、cosine decay、gradient clipping”，但讲不清它们为什么需要、解决什么问题、有什么副作用。面试官一旦追问“为什么 warmup 能缓解训练早期不稳定”“AdamW 和 Adam 的 weight decay 有什么区别”“batch size 变大 learning rate 怎么调”，回答就容易停在经验层。

本章目标：讲清梯度、Hessian、SGD、Momentum、Adam、AdamW、学习率、warmup、gradient clipping、凸优化和非凸优化直觉，并把它们连接到 LLM 训练稳定性。

0. 本讲资料边界与第二轮精修口径

第二轮精修时，本章按 WRITING_PLAN.md 先核对 Adam 原论文、AdamW / decoupled weight decay 论文、PyTorch AdamW 和 gradient clipping 文档、常见 learning rate scheduler 文档，以及 Transformer 训练中 warmup schedule 的公开资料边界。

本章聚焦大模型算法岗最常用的优化基础：

1. 目标函数、梯度、学习率和梯度下降。
2. Mini-batch SGD、global batch、gradient accumulation 和优化噪声。
3. Momentum、Adam、AdamW 的核心更新机制。
4. Warmup、cosine decay、gradient clipping 和 loss spike 排查。
5. Hessian / 曲率、凸优化 / 非凸优化和训练稳定性直觉。

本章不展开完整凸优化教材、二阶优化算法族、收敛性证明、复杂自适应优化器谱系、Muon / Sophia / Lion 等较新优化器专题或分布式优化器实现细节。后续训练工程章会再讨论 ZeRO、optimizer state sharding、混合精度和 checkpoint resume。这里的目标是让读者能在面试中把“为什么这么训练”讲清楚，并能用最小代码验证关键机制。

4.1 优化在大模型训练中解决什么问题

训练大模型时，我们有一个参数量巨大的模型和一个 loss function。

目标是找到一组参数，使训练数据上的 loss 尽可能低，同时希望模型能泛化到未见数据。

可以直观写成：

$$\theta^* = \arg \min_{\theta} L(\theta)$$

其中 θ 是模型参数， $L(\theta)$ 是训练目标，例如语言模型的平均 token cross entropy。

这个问题很难，因为：

1. 参数量巨大，可能是几十亿到万亿级。
2. Loss surface 是非凸的。
3. 数据是 mini-batch 随机采样，有噪声。
4. 混合精度会带来数值问题。
5. 分布式训练会带来同步和恢复问题。
6. 训练成本极高，不能随便试错。

所以优化不是“调用 optimizer.step()”这么简单，而是要理解训练动态、稳定性和工程约束。

4.1.1 关键公式与 LLM 优化速查

经验风险最小化：

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i)$$

其中 ℓ 是单样本损失， N 是训练样本或参与 loss 的 token 数。

梯度：

$$g_t = \nabla_{\theta} L(\theta_t)$$

梯度下降：

$$\theta_{t+1} = \theta_t - \eta_t g_t$$

η_t 是第 t 步学习率。学习率太大容易震荡或发散，太小会收敛慢。

Mini-batch 梯度估计：

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\theta} \ell_i(\theta_t)$$

其中 B_t 是当前 mini-batch。Batch 越小，梯度噪声通常越大。

Global batch：

$$B_{\text{global}} = B_{\text{micro}} \cdot A_{\text{grad}} \cdot N_{\text{dp}}$$

其中 B_{micro} 是每卡 micro batch， A_{grad} 是 gradient accumulation steps， N_{dp} 是 data parallel worker 数。

Momentum：

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta_t v_t$$

Adam 的一阶矩、二阶矩和 bias correction：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

AdamW 的解耦 weight decay:

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta_t \lambda \theta_t$$

其中 `lambda` 是 weight decay 系数。关键点是：衰减项不混进梯度再被 Adam 的自适应缩放处理，而是单独作用在参数上。

Linear warmup:

$$\eta_t = \eta_{\max} \frac{t+1}{T_{\text{warm}}}, \quad 0 \leq t < T_{\text{warm}}$$

Cosine decay 的常见写法:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi r_t))$$

其中 `r_t` 是 warmup 之后归一化到 `[0,1]` 的训练进度。

Global norm clipping:

$$s = \min(1, C/(\|g\|_2 + \epsilon))$$

$$g_{\text{clip}} = sg$$

其中 `C` 是最大梯度范数。Clipping 只能限制异常更新，不能替代对数据、mask、学习率和 optimizer state 的根因排查。

二阶曲率直觉:

$$L(\theta + \Delta) \approx L(\theta) + g^T \Delta + \frac{1}{2} \Delta^T H \Delta$$

`H` 是 Hessian。曲率越大的方向，对学习率越敏感，也更容易震荡。

优化上线门禁可以抽象成:

$$G_{\text{opt}} = G_{\text{loss}} \wedge G_{\text{grad}} \wedge G_{\text{lr}} \wedge G_{\text{state}}$$

也就是 loss 正常下降、grad norm 无异常爆炸、学习率 schedule 符合预期、optimizer / scheduler / scaler 状态能正确保存和恢复。

4.2 梯度：loss 下降最快的局部方向

梯度表示 loss 对参数的变化敏感度。

如果参数是 `w`，loss 是 `L(w)`，梯度可以理解为:

$$g = \frac{dL}{dw}$$

梯度告诉我们：如果参数往某个方向移动，loss 会如何变化。

梯度下降的基本更新是：

$$w_{\text{new}} = w - \eta g$$

其中 η 是 learning rate, g 是当前梯度。

为什么是减号？因为梯度指向 loss 增长最快的方向，想降低 loss，就往反方向走。

面试回答：

梯度表示 loss 对参数的局部变化方向。梯度下降沿负梯度方向更新参数，因为梯度方向是 loss 增长最快的方向。Learning rate 控制

4.3 一个最小梯度下降例子

考虑一个简单函数：

$$L(w) = (w - 5)^2$$

它的最小值在 $w = 5$ 。

梯度是：

$$\frac{dL}{dw} = 2(w - 5)$$

如果 $w = 0$ ，梯度是 -10。按照梯度下降：

$$w_{\text{new}} = w - \eta \frac{dL}{dw}$$

如果 $\eta = 0.1$ ，则：

$$w_{\text{new}} = 0 - 0.1(-10) = 1$$

参数从 0 朝 5 移动。

这个例子虽然简单，但说明了训练的核心：反复计算梯度，沿降低 loss 的方向更新参数。

4.4 Learning rate：优化中最敏感的旋钮

Learning rate 决定每次参数更新幅度。

如果 learning rate 太小：

1. 收敛慢。
2. 训练成本高。
3. 可能卡在不好的区域很久。

如果 learning rate 太大：

1. Loss 震荡。
2. Loss spike。
3. NaN/Inf。
4. 模型发散。

大模型训练中，learning rate 不能孤立选择，它和以下因素有关：

1. 模型规模。

2. Batch size。
3. Optimizer。
4. 数据质量。
5. Warmup steps。
6. Precision。
7. Gradient clipping。

面试回答：

Learning rate 控制每一步参数更新幅度。它太小会收敛慢，太大可能导致 loss spike 或发散。大模型训练中 learning rate 要和 ba

4.5 SGD：随机估计全量梯度

全量梯度需要遍历整个训练集，成本太高。SGD 使用一个 mini-batch 估计梯度。

直觉：

$$\frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell_i(\theta)$$

左边是 mini-batch 梯度，右边是全量数据梯度。

这种估计有噪声。噪声既有坏处，也有好处。

坏处：

1. Loss 曲线抖动。
2. 小 batch 不稳定。
3. 梯度方向可能偏离真实下降方向。

好处：

1. 每步成本低。
2. 噪声可能帮助逃离某些不好的局部区域。
3. 有时泛化更好。

Batch size 越大，梯度估计越稳定，但计算和显存成本越高。

面试回答：

SGD 用 mini-batch 梯度近似全量数据梯度。Mini-batch 带来噪声，小 batch 噪声大但便宜，大 batch 更稳定但成本高，也可能影响

4.6 Momentum：给梯度下降加惯性

普通 SGD 每一步只看当前梯度。Momentum 会维护一个历史梯度的移动平均，让更新方向更平滑。

直觉上，Momentum 像给优化过程加惯性：

1. 如果多个 step 梯度方向一致，就加速前进。
2. 如果梯度来回震荡，就相互抵消一部分。

可以直观写成：

$$v_t = \beta v_{t-1} + g_t$$

$$w_{t+1} = w_t - \eta v_t$$

Momentum 对峡谷形 loss surface 很有用：一个方向很陡，一个方向很平。没有 momentum 时，参数会在陡峭方向来回震荡；有 momentum 后，更新更稳定。

4.7 Adam：自适应学习率

Adam 可以理解为 Momentum + 自适应缩放。

它维护两个统计量：

1. 一阶矩：梯度的移动平均，类似 momentum。
2. 二阶矩：梯度平方的移动平均，表示梯度尺度。

直觉上：

1. 一阶矩告诉我们往哪个方向走。
2. 二阶矩告诉我们这个方向梯度通常有多大。
3. 梯度大的参数方向，步子会被相对缩小。
4. 梯度小的参数方向，步子会被相对放大。

这让 Adam 在高维、稀疏、非平稳优化中很实用。

Adam 的核心更新可以写成：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

其中 m_t 是一阶矩， v_t 是二阶矩， $\hat{m}_t$ 和 $\hat{v}_t$ 是 bias correction 之后的估计。

面试回答：

Adam 维护梯度的一阶矩和二阶矩。一阶矩类似 momentum，表示平均更新方向；二阶矩表示梯度平方的平均大小，用来做自适应缩放。

4.8 AdamW：解耦 weight decay

AdamW 是大模型训练中非常常见的优化器。

要理解 AdamW，先理解 weight decay。

Weight decay 的目标是限制权重过大，起到正则化作用。普通写法类似：

$$w_{t+1} = w_t - \eta g_t - \eta \lambda w_t$$

其中 λ 是 weight decay 系数。

在 Adam 中，如果把 weight decay 直接加到 gradient 里，会受到 Adam 自适应缩放影响，导致正则化效果和预期不一致。

AdamW 的核心是 decoupled weight decay：把 weight decay 从梯度更新中解耦出来，单独对参数做衰减。

可以写成：

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda w_t$$

这和“把 λw 加到梯度里再交给 Adam 缩放”不是同一个行为。

面试回答：

AdamW 和 Adam 的关键区别是 weight decay 的处理。Adam 如果把 L2 正则项直接加进 gradient，会被 Adam 的自适应学习率缩放影响。

注意：并不是所有参数都适合 weight decay。大模型训练中常常不对 bias、LayerNorm 或 RMSNorm 参数做 weight decay，因为这些参数主要控制平移或尺度，衰减它们可能损害训练稳定性。

4.9 Warmup：训练早期不要走太猛

大模型训练常用 warmup：学习率从很小逐渐升到峰值。

为什么需要 warmup？

训练刚开始时：

1. 参数还没有形成稳定表示。
2. 激活和梯度分布不稳定。
3. Adam 的一阶矩和二阶矩估计还不可靠。
4. 大学习率容易导致 loss spike 或 NaN。

Warmup 让模型先用小步子进入稳定区域，再逐渐加大学习率。

面试回答：

Warmup 的作用是缓解训练早期不稳定。刚开始模型参数、激活和 Adam 的动量统计都不稳定，如果直接用 peak learning rate，容易

4.10 Learning rate decay：后期小步收敛

Warmup 之后，learning rate 通常会逐渐下降。常见 schedule 包括 cosine decay、linear decay、constant with decay 等。

为什么要 decay？

1. 训练前期需要较大步子快速下降。
2. 训练后期需要较小步子细致收敛。
3. 大学习率持续太久可能导致 loss 震荡。

Cosine decay 的直觉是：平滑地把学习率从峰值降到较小值。

常见形式是：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi r_t))$$

其中 r_t 是 warmup 后从 0 增加到 1 的训练进度。

面试中不必背所有 schedule 公式，但要能讲 trade-off：schedule 不是独立超参，它和总训练 token、batch size、模型规模、数据质量一起决定训练动态。

4.11 Gradient clipping：限制异常更新

Gradient clipping 用于限制梯度范数过大。

常见做法是：如果 gradient norm 超过阈值，就按比例缩小所有梯度，使总 norm 不超过阈值。

公式写成：

$$s = \min(1, C/(\|g\|_2 + \epsilon))$$

$$g_{\text{clip}} = sg$$

其中 C 是最大允许的 global grad norm。

它解决的问题：

1. 梯度爆炸。
2. 异常 batch 导致超大更新。
3. 混合精度下的数值不稳定。
4. Loss spike 风险。

但 clipping 不是万能的。如果 loss spike 来自数据错、mask 错、学习率过大或 checkpoint 恢复错误，clipping 只能缓解，不能代替根因排查。

面试回答：

Gradient clipping 用来限制梯度范数，防止某一步更新过大导致训练不稳定。它对梯度爆炸和异常 batch 有缓解作用，但不能替代根因排查。

4.12 Hessian 和曲率直觉

梯度是一阶信息，告诉我们往哪里走。Hessian 是二阶信息，描述 loss surface 的曲率。

直觉上：

1. 曲率大，loss 对参数变化很敏感。
2. 曲率小，loss 变化平缓。
3. 不同方向曲率不同，会导致优化困难。

深度学习通常不会直接计算完整 Hessian，因为参数太多，成本不可接受。但 Hessian 直觉很重要。

二阶近似可以写成：

$$L(\theta + \Delta) \approx L(\theta) + g^T \Delta + \frac{1}{2} \Delta^T H \Delta$$

其中 H 是 Hessian。它描述不同方向上的曲率。

例如：

1. Learning rate 太大，在高曲率方向容易震荡。
2. Adaptive optimizer 可以缓解不同参数方向梯度尺度差异。
3. Sharp minima 和 flat minima 常用来讨论泛化直觉。

面试回答：

Hessian 描述 loss surface 的二阶曲率。大模型不会显式计算完整 Hessian，但曲率直觉有用：高曲率方向对学习率更敏感，容易震荡。

4.13 凸优化和非凸优化直觉

凸优化中，局部最优就是全局最优，理论分析更清楚。

深度网络训练是非凸优化：

1. 有大量鞍点。
2. 有许多局部区域。
3. 参数对称性很多。
4. Loss surface 高维复杂。

但实际中，大模型仍然可以训练成功，原因包括：

1. 过参数化带来很多可行解。
2. SGD 噪声帮助探索。
3. 残差连接、归一化、初始化改善优化。
4. 大规模数据和计算让统计规律更稳定。
5. AdamW、warmup、clipping 等 recipe 已经高度工程化。

面试回答：

大模型训练是非凸优化，不能用简单凸优化理论解释全部现象。但过参数化、残差结构、归一化、合适初始化、AdamW、warmup 和大模型训练技巧是重要因素。

4.14 Batch size、gradient accumulation 和优化噪声

大模型训练中要区分 micro batch 和 global batch。

$$B_{\text{global}} = B_{\text{micro}} \cdot A_{\text{grad}} \cdot N_{\text{dp}}$$

其中 B_{micro} 是单卡 micro batch， A_{grad} 是梯度累积步数， N_{dp} 是 data parallel worker 数。

Micro batch 受单卡显存限制。Gradient accumulation 用多个 micro-batch 累积梯度，模拟更大的 effective batch。

Batch size 影响：

1. 梯度噪声。
2. 硬件吞吐。
3. 学习率选择。
4. 更新步数。
5. 泛化和收敛动态。

面试回答：

LLM 训练中要看 global batch size，而不是单卡 micro batch。Global batch 由 micro batch、gradient accumulation 和 data parallel worker 数决定。

4.15 Loss spike 的优化视角

Loss spike 可能来自很多原因，优化相关原因包括：

1. Learning rate 太大。
2. Warmup 太短。
3. Gradient norm 突然变大。
4. Gradient clipping 缺失或阈值不合适。
5. AdamW 状态异常。
6. Checkpoint resume 后 optimizer state 不连续。
7. FP16 overflow 或 loss scaling 问题。

排查时不要只说 “调小学习率”。

更好的回答：

我会先定位 spike 的 step，检查 learning rate、grad norm、optimizer state、mixed precision 和 batch 数据。如果发生在 warmup 阶段，可能是学习率或 warmup 策略问题。

4.16 最小可运行优化审计 demo

下面用纯 Python 把本章关键机制串起来：稳定学习率和过大学习率的差异、Momentum、global norm clipping、warmup + cosine decay、AdamW 解耦 weight decay，以及 global batch 计算。

```
import math
```

```
def loss(w):  
    return (w - 5.0) ** 2
```

```
def grad(w):  
    return 2.0 * (w - 5.0)
```

```

def gd_trace(lr, steps=6, w=0.0):
    trace = []
    for step in range(steps):
        g = grad(w)
        w = w - lr * g
        trace.append((step + 1, round(w, 3), round(loss(w), 3)))
    return trace

def momentum_trace(lr=0.05, beta=0.8, steps=6, w=0.0):
    v = 0.0
    trace = []
    for step in range(steps):
        g = grad(w)
        v = beta * v + g
        w = w - lr * v
        trace.append((step + 1, round(w, 3), round(v, 3), round(loss(w), 3)))
    return trace

def clip_by_global_norm(grads, max_norm):
    norm = math.sqrt(sum(g * g for g in grads))
    scale = min(1.0, max_norm / (norm + 1e-12))
    clipped = [g * scale for g in grads]
    clipped_norm = math.sqrt(sum(g * g for g in clipped))
    return norm, scale, clipped, clipped_norm

def lr_schedule(step, total_steps, warmup_steps, peak_lr, min_lr):
    if step < warmup_steps:
        return peak_lr * (step + 1) / warmup_steps
    denom = max(1, total_steps - warmup_steps - 1)
    progress = min(1.0, (step - warmup_steps) / denom)
    cosine = 0.5 * (1.0 + math.cos(math.pi * progress))
    return min_lr + (peak_lr - min_lr) * cosine

def adam_first_step(params, grads, lr=1e-3, weight_decay=0.1, decoupled=True):
    out = []
    for w, g in zip(params, grads):
        if decoupled:
            m_hat = g
            v_hat = g * g
            adaptive_step = lr * m_hat / (math.sqrt(v_hat) + 1e-8)
            new_w = w - adaptive_step - lr * weight_decay * w
        else:
            adjusted_grad = g + weight_decay * w
            m_hat = adjusted_grad
            v_hat = adjusted_grad * adjusted_grad
            adaptive_step = lr * m_hat / (math.sqrt(v_hat) + 1e-8)
            new_w = w - adaptive_step
        out.append(round(new_w, 6))
    return out

```

```

stable = gd_trace(lr=0.1)
unstable = gd_trace(lr=1.1)
momentum = momentum_trace()

norm, scale, clipped, clipped_norm = clip_by_global_norm([3.0, 4.0, 12.0], 5.0)
schedule = [
    round(lr_schedule(s, total_steps=10, warmup_steps=3, peak_lr=1e-3, min_lr=1e-4), 6)
    for s in range(10)
]

params = [10.0, 0.1]
grads = [0.1, 0.1]
adam_l2 = adam_first_step(params, grads, decoupled=False)
adamw = adam_first_step(params, grads, decoupled=True)
global_batch = 2 * 4 * 8

checks = {
    "stable_lr_decreases_loss": stable[-1][2] < stable[0][2],
    "too_large_lr_diverges": unstable[-1][2] > unstable[0][2],
    "clip_norm_limited": round(clipped_norm, 6) <= 5.0,
    "warmup_reaches_peak": schedule[2] == 0.001,
    "cosine_decays": schedule[-1] == 0.0001,
    "adamw_decays_large_weight_more": (params[0] - adamw[0]) > (params[1] - adamw[1]),
    "global_batch": global_batch == 64,
}

print("gd_lr_0.1=", stable)
print("gd_lr_1.1=", unstable)
print("momentum=", momentum)
print(
    "clip=",
    {
        "norm": round(norm, 3),
        "scale": round(scale, 3),
        "clipped": [round(x, 3) for x in clipped],
        "clipped_norm": round(clipped_norm, 3),
    },
)
print("lr_schedule=", schedule)
print("adam_l2_first_step=", adam_l2)
print("adamw_first_step=", adamw)
print("global_batch=", global_batch)
print("checks=", checks)

```

预期输出:

```

gd_lr_0.1= [(1, 1.0, 16.0), (2, 1.8, 10.24), (3, 2.44, 6.554), (4, 2.952, 4.194), (5, 3.362, 2.684), (6, 3.689, 1.718)]
gd_lr_1.1= [(1, 11.0, 36.0), (2, -2.2, 51.84), (3, 13.64, 74.65), (4, -5.368, 107.495), (5, 17.442, 154.793), (6, -
9.93, 222.903)]
momentum= [(1, 0.5, -10.0, 20.25), (2, 1.35, -17.0, 13.322), (3, 2.395, -20.9, 6.786), (4, 3.492, -
21.93, 2.276), (5, 4.52, -20.561, 0.231), (6, 5.39, -17.41, 0.152)]
clip= {'norm': 13.0, 'scale': 0.385, 'clipped': [1.154, 1.538, 4.615], 'clipped_norm': 5.0}
lr_schedule= [0.000333, 0.000667, 0.001, 0.001, 0.00094, 0.000775, 0.00055, 0.000325, 0.00016, 0.0001]
adam_l2_first_step= [9.999, 0.099]
adamw_first_step= [9.998, 0.09899]
global_batch= 64

```

checks= {'stable_lr_decreases_loss': True, 'too_large_lr_diverges': True, 'clip_norm_limited': True, 'warmup_reaches_peak': True}

这里要看懂：

1. lr=0.1 的 loss 稳定下降，lr=1.1 在同一个凸二次函数上都会震荡发散，说明 learning rate 不是越大越好。
2. Momentum 用历史梯度累积更新方向，能更快接近目标，但也可能越过最优点，所以仍需要学习率和动量系数配合。
3. Gradient clipping 把全局范数从 13.0 缩到 5.0，但只是在限制更新幅度。
4. Warmup 让学习率逐步达到 peak，cosine decay 后期平滑降低学习率。
5. AdamW 对大权重产生更明显的解耦衰减，而不是把 L2 项混入 Adam 的自适应梯度缩放。
6. global_batch=64 来自 micro batch、gradient accumulation 和 data parallel size 的乘积。

真实 LLM 训练会更复杂，还会加入 mixed precision、distributed training、scheduler state、optimizer state sharding、checkpoint、evaluation 和数据质量审计。但这些工程机制都建立在本 demo 展示的更新、裁剪、调度和状态管理之上。

4.17 面试高频问题

4.17.1 为什么大模型训练常用 warmup

回答：

训练初期参数、激活和 Adam 的动量统计都不稳定。如果一开始就用 peak learning rate，容易造成 loss spike 或发散。Warmup 让学

4.17.2 Adam 和 AdamW 有什么区别

回答：

AdamW 的关键是 decoupled weight decay。Adam 如果把 L2 正则直接加到 gradient 中，会被自适应学习率缩放影响。AdamW 把 weight

4.17.3 为什么要做 gradient clipping

回答：

Gradient clipping 限制梯度范数，防止某一步更新过大。它能缓解梯度爆炸、异常 batch 和训练不稳定，降低 loss spike 风险。但

4.17.4 Batch size 和 learning rate 怎么关联

回答：

Batch size 影响梯度噪声和更新步数。更大的 global batch 通常梯度更稳定，可以支持更大的 learning rate，但不是简单越大越好

4.17.5 Adam 的一阶矩和二阶矩分别是什么

回答：

一阶矩是梯度的指数移动平均，类似 momentum，表示比较稳定的更新方向；二阶矩是梯度平方的指数移动平均，表示每个参数方向的

4.17.6 训练出现 loss spike 你怎么从优化角度排查

回答：

先定位 spike 发生在哪个 step，再对齐学习率、warmup、grad norm、clipping、optimizer state、mixed precision scaler、异常

4.18 常见误区

1. 把 optimizer 当成黑盒 API。
2. 只会说 AdamW 好，不会解释 decoupled weight decay。
3. Loss spike 只会说调小 learning rate。
4. 不区分 micro batch 和 global batch。

5. 忽略 optimizer state 对 checkpoint resume 的影响。
6. 认为 gradient clipping 可以解决所有训练不稳定。
7. 不监控 grad norm, 只看 loss。

4.19 小练习

1. 对 $L(w) = (w - 5)^2$ 求梯度, 并手算 5 步梯度下降。
2. 比较 learning rate 为 0.01、0.1、1.0 时的行为。
3. 用自己的话解释 Momentum。
4. 用自己的话解释 Adam 的一阶矩和二阶矩。
5. 解释 AdamW 为什么叫 decoupled weight decay。
6. 写一个 warmup + cosine decay 的学习率函数。
7. 列出 loss spike 时与优化相关的 5 个排查项。
8. 写一个 0 依赖 Python demo, 验证 stable learning rate、too large learning rate、gradient clipping、warmup + cosine 和 AdamW decoupled weight decay。

4.20 本章总结

优化把 loss 变成参数更新, 是大模型训练能够发生的核心机制。

本章最重要的结论:

1. 梯度告诉参数如何局部降低 loss。
2. Learning rate 控制每步更新幅度, 是最敏感超参之一。
3. SGD 用 mini-batch 估计全量梯度, 因此有噪声。
4. Momentum 平滑更新方向。
5. Adam 用一阶矩和二阶矩做自适应更新。
6. AdamW 解耦 weight decay, 更适合大模型训练。
7. Warmup 缓解训练早期不稳定。
8. Gradient clipping 限制异常梯度更新。
9. Hessian 和曲率直觉帮助理解学习率敏感性。
10. Loss spike 要结合数据、梯度、优化器、精度和 checkpoint 状态排查。

面试中讲优化, 不要只背优化器名字。要能把梯度、学习率、batch size、AdamW、warmup、clipping 和训练稳定性连成一条完整链路。

第五章：矩阵分解与低秩

矩阵分解和低秩近似, 是理解 LoRA、模型压缩、embedding 几何、PCA 和表示分析的重要数学基础。大模型里很多巨大的权重矩阵和表示矩阵, 看起来维度极高, 但它们的有效变化可能集中在少数方向上。低秩方法的核心直觉就是: 不一定需要完整高维空间, 很多任务相关的信息可以用更低维的子空间表示。

这一章不追求严格数学证明, 而是围绕面试和大模型实践讲清几个问题: SVD 是什么, PCA 为什么能降维, 低秩近似为什么有用, LoRA 为什么可以用两个小矩阵表示权重更新, rank 如何影响能力和成本, 低秩方法有什么边界。

本章重点: SVD、PCA、低秩近似、rank、LoRA、模型压缩、embedding 几何和面试表达。

0. 本讲资料边界与第二轮精修口径

第二轮精修时, 本章按 WRITING_PLAN.md 先核对 NumPy / PyTorch 的 SVD 文档、PCA 中 explained variance 的常见定义、LoRA 论文、QLoRA 论文和 PEFT / LoRA 工程口径。

本章聚焦大模型算法岗常用的低秩数学:

1. Rank、SVD、奇异值能量和 rank-r 近似。
2. PCA、中心化数据矩阵、协方差矩阵和 explained variance。

3. LoRA 的低秩增量参数化、参数量估算和 rank 选择。
4. QLoRA 中量化 base model + 训练 LoRA adapter 的显存直觉。
5. 低秩压缩、LoRA、embedding 可视化之间的区别。

本章不展开完整数值线性代数算法、Lanczos / randomized SVD、GPU kernel、PEFT 全量配置、LoRA 初始化细节、adapter merge 冲突算法或 QLoRA 的 paged optimizer 实现。面试中更重要的是：能说清 “低秩近似原矩阵” 和 “低秩建模微调增量” 不是一回事，并能手算参数量、rank trade-off 和质量回归边界。

5.1 为什么大模型需要低秩直觉

大模型有大量矩阵。

例如一个 hidden size 为 4096 的线性层，权重矩阵可能是：

$$W \in \mathbb{R}^{4096 \times 4096}$$

参数量是：

$$4096 \times 4096 = 16,777,216$$

如果全参微调，每个这样的大矩阵都要更新，显存、优化器状态和通信成本都很高。

低秩方法的核心问题是：是否真的需要更新整个大矩阵？

LoRA 的答案是：很多任务适配可能只需要在一个低维子空间中调整权重，因此可以学习低秩更新。

这背后不是魔法，而是矩阵分解和低秩近似的直觉。

5.1.1 关键公式与低秩速查

矩阵 rank：

$$\text{rank}(W) = \dim(\text{span}(\text{columns}(W)))$$

也就是列空间中线性独立方向的数量。行 rank 和列 rank 相等。

SVD：

$$W = U \Sigma V^T$$

其中 U 和 V 给出正交方向， Σ 的对角线是奇异值 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ 。

Rank-r 近似：

$$W_r = U_r \Sigma_r V_r^T$$

保留前 r 个最大奇异值。直觉上，它只保留最重要的 r 个方向。

Frobenius 误差：

$$\|W - W_r\|_F^2 = \sum_{i>r} \sigma_i^2$$

奇异值能量保留率：

$$E_r = \frac{\sum_{i=1}^r \sigma_i^2}{\sum_i \sigma_i^2}$$

E_r 越高, rank-r 近似保留的矩阵能量越多。但能量高不等于下游任务一定不掉点, 仍要做任务评估。

PCA 中心化数据矩阵:

$$X_c = X - \mathbf{1}\mu^T$$

协方差矩阵:

$$C = \frac{1}{n} X_c^T X_c$$

主成分方向是协方差矩阵特征值最大的方向。第 k 个主成分 explained variance ratio:

$$R_k = \frac{\lambda_k}{\sum_j \lambda_j}$$

LoRA 增量:

$$W' = W + \Delta W$$

$$\Delta W = AB, \quad A \in \mathbb{R}^{d_{\text{in}} \times r}, \quad B \in \mathbb{R}^{r \times d_{\text{out}}}$$

LoRA 参数量:

$$N_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}})$$

相对全量更新比例:

$$\rho_{\text{LoRA}} = \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}$$

LoRA 线性层输出常写成:

$$y = xW + \frac{\alpha}{r} xAB$$

其中 α / r 是常见缩放因子, 控制低秩增量的尺度。

低秩压缩参数量:

$$N_{\text{factor}} = r(d_{\text{in}} + d_{\text{out}})$$

压缩和 LoRA 的关键区别: 压缩近似原始 W , LoRA 近似微调增量 ΔW 。

5.2 Rank：矩阵真正有多少独立方向

矩阵的 rank 可以理解为它包含多少个线性独立方向。

如果一个矩阵的很多行或列可以由其他行列组合出来，那么它的有效自由度就低，rank 就低。

直觉例子：

$$\text{row}_3 = \text{row}_1 + \text{row}_2$$

这说明 row_3 没有提供新的独立方向。

在机器学习中，rank 代表表达能力和自由度。高 rank 更灵活，但参数多；低 rank 更受限，但更省参数，也可能有正则化效果。

面试回答：

Rank 可以理解为矩阵中独立方向的数量。低秩矩阵说明它的变化主要发生在少数方向上。LoRA 利用这个直觉，假设微调所需的权重更

5.3 SVD：把矩阵拆成重要方向

SVD，也就是奇异值分解，可以把一个矩阵分解成：

$$W = U \Sigma V^T$$

直觉上：

1. V 给出输入空间中的方向。
2. Σ 表示每个方向的重要程度。
3. U 给出输出空间中的方向。

Σ 中的奇异值通常按从大到小排列。奇异值越大，说明对应方向越重要。

如果只保留最大的 r 个奇异值，就得到 rank- r 近似：

$$W_r = U_r \Sigma_r V_r^T$$

保留能量可以写成：

$$E_r = \frac{\sum_{i=1}^r \sigma_i^2}{\sum_i \sigma_i^2}$$

这就是低秩近似：用少数重要方向近似原矩阵。

面试回答：

SVD 可以把矩阵分解成一组从重要到不重要的方向。奇异值越大，说明对应方向贡献越大。如果只保留最大的几个奇异值，就能得到低秩近似。

5.4 为什么低秩近似可行

低秩近似可行的前提是：矩阵信息并不是均匀分散在所有方向上，而是集中在少数重要方向上。

在很多数据和模型中，这个假设经常近似成立：

1. 自然数据有结构，不是完全随机噪声。
2. 任务相关变化可能只发生在少数方向。
3. 高维表示中存在冗余。
4. 微调常常只需要调整部分行为，而不是重写全部知识。

但低秩近似不是万能的。如果任务需要复杂、大范围的参数变化，rank 太小会限制表达能力。

所以低秩方法的核心 trade-off 是：

1. 更低 rank：更省参数，但表达能力更受限。
2. 更高 rank：表达更强，但成本更高，也可能过拟合。

5.5 PCA：寻找数据中最重要的方向

PCA，也就是主成分分析，可以理解为寻找数据中方差最大的方向，并把数据投影到这些方向上。

直觉：

1. 数据在某些方向上变化大，说明这些方向包含更多区分信息。
2. 数据在某些方向上变化小，可能是噪声或次要信息。
3. 保留前几个主方向，就能降维并保留主要结构。

PCA 和 SVD 关系紧密。实际中，PCA 可以通过对中心化数据矩阵做 SVD 得到。

中心化数据矩阵和协方差矩阵可以写成：

$$X_c = X - \mathbf{1}\mu^T$$

$$C = \frac{1}{n} X_c^T X_c$$

PCA 保留前 r 个特征值对应的方向，explained variance ratio 可以写成：

$$E_r = \frac{\sum_{i=1}^r \lambda_i}{\sum_i \lambda_i}$$

在大模型中，PCA 的直觉常用于：

1. 可视化 embedding。
2. 分析 hidden states。
3. 理解表示空间的主要变化方向。
4. 压缩或去噪。
5. 研究模型行为方向。

面试回答：

PCA 寻找数据中方差最大的方向，并把数据投影到这些主方向上。它的作用是用更低维表示保留主要变化结构。在大模型里，可以用

5.6 LoRA：低秩更新而不是低秩权重

LoRA 的关键不是把原始权重 W 变成低秩，而是把微调时的权重增量 ΔW 表示成低秩。

全参微调：

$$W' = W + \Delta W$$

LoRA：

$$\Delta W = AB$$

其中：

$$A \in \mathbb{R}^{d_{\text{in}} \times r}, \quad B \in \mathbb{R}^{r \times d_{\text{out}}}$$

r 是 LoRA rank, 通常远小于 d_{in} 和 d_{out} 。

这意味着 LoRA 假设：任务适配所需的更新位于一个低维子空间中。

带缩放的 LoRA 输出常写成：

$$y = xW + \frac{\alpha}{r} xAB$$

其中 α 控制 adapter 增量尺度, α / r 是常见缩放方式。

面试中要特别注意：LoRA 不是说原模型权重本身低秩, 而是说微调增量可以低秩建模。

面试回答：

LoRA 的核心是低秩参数化权重更新。它冻结原始权重 W , 只学习一个低秩增量 $\Delta W = AB$ 。这样不是把原权重变成低秩, 而是假设

5.7 LoRA 参数量怎么算

假设原始权重矩阵：

$$W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$$

全量更新参数量：

$$N_{\text{full}} = d_{\text{in}} d_{\text{out}}$$

LoRA 参数量：

$$N_{\text{LoRA}} = r d_{\text{in}} + r d_{\text{out}} = r(d_{\text{in}} + d_{\text{out}})$$

例如：

$$d_{\text{in}} = 4096, \quad d_{\text{out}} = 4096, \quad r = 8$$

全量参数：

$$4096 \times 4096 = 16,777,216$$

LoRA 参数：

$$8(4096 + 4096) = 65,536$$

相对比例：

$$\rho_{\text{LoRA}} = \frac{65,536}{16,777,216} \approx 0.003906$$

参数量大幅减少。

这也是 LoRA 能降低训练显存的原因之一。更少可训练参数意味着更少梯度和 optimizer state。

但注意：前向计算仍然要经过原模型, LoRA 不等于推理成本自动大幅下降。推理时如果不 merge LoRA, 可能还会有额外计算；如果 merge 到权重里, 则推理结构更简单。

5.8 LoRA rank 如何选择

Rank 是 LoRA 最重要的超参之一。

Rank 太小：

1. 表达能力不足。
2. 复杂任务适配不够。
3. 学不到足够行为变化。

Rank 太大：

1. 参数量增加。
2. 显存和 optimizer state 增加。
3. 小数据场景更容易过拟合。
4. 低秩正则化效果减弱。

实际选择 rank 要看：

1. 任务复杂度。
2. 数据量。
3. 模型大小。
4. 目标模块。
5. 训练预算。
6. 是否需要多 adapter。

面试回答：

LoRA rank 控制低秩更新的表达能力。Rank 越大，更新空间越灵活，但参数、显存和过拟合风险也增加。Rank 越小越省，但可能表达

5.9 LoRA 应该加在哪些模块

LoRA 常加在 Transformer 的线性层上，例如：

1. W_Q 。
2. W_K 。
3. W_V 。
4. Attention output projection。
5. MLP up projection。
6. MLP down projection。
7. Gate projection。

不同模块影响不同能力。

例如：

1. Attention 相关模块影响信息选择和上下文交互。
2. MLP 相关模块影响特征变换和知识表达。
3. 只调部分模块更省，但能力变化有限。
4. 调更多模块更强，但成本更高。

面试中不要只说“加 LoRA”。要能说明加在哪里、为什么、如何评估。

5.10 QLoRA 和量化的关系

QLoRA 的核心是把 base model 权重量化存储，同时用 LoRA 训练低秩 adapter。

直觉：

1. Base model 很大，量化后显存下降。
2. LoRA adapter 很小，保持可训练。
3. 反向传播主要更新 adapter。

QLoRA 适合显存受限的微调场景。

粗略权重显存可以写成：

$$M_{\text{weight}} \approx \frac{N_{\text{param}} b}{8}$$

其中 N_{param} 是 base model 参数量， b 是每个权重的 bit 数。BF16 约为 16 bit，INT4 约为 4 bit。实际显存还要加量化 scale、LoRA adapter、activation、梯度和优化器状态。

但要注意：

1. 量化可能影响质量。
2. 训练速度不一定总是更快。
3. 需要关注量化格式、计算 dtype 和内存峰值。
4. 对格式稳定性、代码、数学和安全任务要额外评估。

面试回答：

QLoRA 可以理解量化 base model 加低秩 adapter 训练。它通过量化降低基础模型显存，同时只训练小的 LoRA 参数，因此适合资源受限的场景。

5.11 低秩和模型压缩

低秩近似也可以用于模型压缩。

如果一个权重矩阵可以近似写成两个低秩矩阵乘积：

$$W \approx AB$$

原本一次大矩阵乘法：

$$XW$$

可以变成：

$$(XA)B$$

如果 rank 很小，参数量和计算量可能下降。

但模型压缩和 LoRA 不同：

1. LoRA 学的是增量 ΔW 。
2. 压缩是近似原始权重 W 。
3. 压缩可能直接损伤原模型能力。
4. 压缩后需要评估质量回归。

面试回答：

低秩压缩是用两个小矩阵近似原始大矩阵，从而减少参数和计算。它和 LoRA 不同，LoRA 是低秩表示微调增量，通常保留原权重不动。

5.12 Embedding 几何和低维可视化

Embedding 是高维向量。为了观察结构，常用 PCA、t-SNE、UMAP 等方法降维到 2D 或 3D。

这些可视化可以帮助理解：

1. 相似样本是否聚在一起。
2. 不同类别是否可分。
3. 数据是否有异常点。

4. 模型微调前后表示是否发生变化。

但要注意：低维可视化容易误导。

1. 高维距离不一定在二维图中保真。
2. 不同降维方法有不同偏差。
3. 图上聚类不等于模型真的理解。
4. 可视化适合探索，不适合单独作为严肃结论。

面试回答：

PCA 或其他降维方法可以帮助观察 embedding 空间结构，比如类别聚类、异常点和微调前后的表示变化。但二维可视化会损失高维信

5.13 低秩方法的优点和边界

优点：

1. 参数量少。
2. 训练显存低。
3. 多任务 adapter 易管理。
4. 小数据场景可能有正则化效果。
5. 适合快速实验和个性化微调。

边界：

1. Rank 太小表达不足。
2. 复杂任务可能需要更大更新空间。
3. 多个 LoRA adapter 合并可能冲突。
4. LoRA 不一定降低基础前向计算成本。
5. 数据质量差时，低秩也会学坏。
6. 对安全、数学、代码、格式稳定性要单独评估。

面试中要避免把 LoRA 说成万能微调方法。

5.14 最小可运行低秩审计 demo

下面用纯 Python 演示本章关键机制：rank、SVD 截断误差、PCA 主方向、LoRA 低秩增量、LoRA 参数量和 QLoRA 显存直觉。

```
import math
```

```
def transpose(a):  
    return [list(row) for row in zip(*a)]
```

```
def matmul(a, b):  
    bt = transpose(b)  
    return [[sum(x * y for x, y in zip(row, col)) for col in bt] for row in a]
```

```
def add(a, b):  
    return [[x + y for x, y in zip(ar, br)] for ar, br in zip(a, b)]
```

```
def scale(a, s):  
    return [[s * x for x in row] for row in a]
```

```
def subtract(a, b):
```

```

    return [[x - y for x, y in zip(ar, br)] for ar, br in zip(a, b)]

def frobenius(a):
    return math.sqrt(sum(x * x for row in a for x in row))

def rank(matrix, tol=1e-10):
    a = [row[:] for row in matrix]
    rows = len(a)
    cols = len(a[0])
    r = 0
    for c in range(cols):
        pivot = max(range(r, rows), key=lambda i: abs(a[i][c]))
        if abs(a[pivot][c]) <= tol:
            continue
        a[r], a[pivot] = a[pivot], a[r]
        pivot_val = a[r][c]
        a[r] = [x / pivot_val for x in a[r]]
        for i in range(rows):
            if i != r:
                factor = a[i][c]
                a[i] = [x - factor * y for x, y in zip(a[i], a[r])]
        r += 1
    if r == rows:
        break
    return r

def diag(values):
    n = len(values)
    return [[values[i] if i == j else 0.0 for j in range(n)] for i in range(n)]

def rank_k_diag_approx(singular_values, k):
    kept = singular_values[:k] + [0.0] * (len(singular_values) - k)
    return diag(kept)

def covariance_2d(points):
    n = len(points)
    mx = sum(p[0] for p in points) / n
    my = sum(p[1] for p in points) / n
    centered = [(x - mx, y - my) for x, y in points]
    cxx = sum(x * x for x, _ in centered) / n
    cxy = sum(x * y for x, y in centered) / n
    cyy = sum(y * y for _, y in centered) / n
    return [[cxx, cxy], [cxy, cyy]]

def eigvals_2x2_symmetric(m):
    a, b = m[0]
    _, d = m[1]
    trace = a + d
    delta = math.sqrt((a - d) ** 2 + 4 * b * b)

```

```

    return sorted([(trace + delta) / 2, (trace - delta) / 2], reverse=True)

singular_values = [5.0, 2.0, 0.5]
w = diag(singular_values)
w_rank1 = rank_k_diag_approx(singular_values, 1)
w_rank2 = rank_k_diag_approx(singular_values, 2)
error_rank1 = frobenius(subtract(w, w_rank1))
error_rank2 = frobenius(subtract(w, w_rank2))
energy_total = sum(s * s for s in singular_values)
energy_rank2 = sum(s * s for s in singular_values[:2]) / energy_total

points = [(-2.0, -1.8), (-1.0, -1.0), (0.0, 0.1), (1.0, 0.9), (2.0, 1.8)]
cov = covariance_2d(points)
eigvals = eigvals_2x2_symmetric(cov)
pca_ratio = eigvals[0] / sum(eigvals)

a = [[0.2, -0.1], [0.0, 0.3], [0.4, 0.1], [-0.2, 0.2]]
b = [[0.5, -0.4, 0.1], [0.2, 0.3, -0.6]]
delta = matmul(a, b)

x = [[1.0, 0.5, -1.0, 2.0]]
base_w = [[0.1, -0.2, 0.3], [0.0, 0.4, -0.1], [0.2, 0.1, 0.0], [-0.3, 0.2, 0.5]]
base_out = matmul(x, base_w)
lora_delta = scale(matmul(matmul(x, a), b), 8 / 2)
lora_out = add(base_out, lora_delta)

full_params = 4096 * 4096
lora_params = 8 * (4096 + 4096)
lora_ratio = lora_params / full_params
bf16_7b_gib = 7_000_000_000 * 2 / (1024 ** 3)
int4_7b_gib = 7_000_000_000 * 0.5 / (1024 ** 3)

checks = {
    "rank_delta_le_r": rank(delta) <= 2,
    "rank2_error_lower": error_rank2 < error_rank1,
    "energy_rank2_high": energy_rank2 > 0.98,
    "pca_first_component_dominates": pca_ratio > 0.99,
    "lora_params_smaller": lora_params < full_params,
    "qlora_memory_smaller": int4_7b_gib < bf16_7b_gib,
}

print("rank_w=", rank(w), "rank_delta=", rank(delta))
print("svd_errors=", {"rank1": round(error_rank1, 3), "rank2": round(error_rank2, 3), "rank2_energy": round(energy_rank2, 3)})
print("pca_cov=", [[round(x, 3) for x in row] for row in cov])
print("pca_eigvals=", [round(x, 3) for x in eigvals], "first_ratio=", round(pca_ratio, 3))
print("lora_params=", {"full": full_params, "rank8": lora_params, "ratio": round(lora_ratio, 6)})
print("lora_delta=", [[round(x, 3) for x in row] for row in lora_delta])
print("lora_out=", [[round(x, 3) for x in row] for row in lora_out])
print("qlora_memory_gib=", {"bf16_7b": round(bf16_7b_gib, 2), "int4_7b": round(int4_7b_gib, 2)})
print("checks=", checks)

```

预期输出:

```

rank_w= 3 rank_delta= 2
svd_errors= {'rank1': 2.062, 'rank2': 0.5, 'rank2_energy': 0.991}

```

```
pca_cov= [[2.0, 1.82], [1.82, 1.66]]
pca_eigvals= [3.658, 0.002] first_ratio= 0.999
lora_params= {'full': 16777216, 'rank8': 65536, 'ratio': 0.003906}
lora_delta= [[-0.92, 1.38, -1.08]]
lora_out= [[-1.62, 1.68, 0.17]]
qlora_memory_gib= {'bf16_7b': 13.04, 'int4_7b': 3.26}
checks= {'rank_delta_le_r': True, 'rank2_error_lower': True, 'energy_rank2_high': True, 'pca_first_component_dominates': True}
```

这个例子说明：

1. 对角矩阵的奇异值可以直观看成每个方向的重要程度。保留前 2 个奇异值时，误差比 rank-1 更小，能量保留率达到 0.991。
2. PCA 的第一主方向解释了几乎全部方差，说明这组 toy 点主要沿一个方向变化。
3. $\Delta W = A B$ 的 rank 不超过 r ，这里 `rank_delta=2`。
4. 4096 x 4096 矩阵 rank-8 LoRA 的参数比例约为 0.003906。
5. QLoRA 的显存直觉来自 base model 低比特存储，例子中 7B 参数从 BF16 约 13.04 GiB 降到 INT4 权重约 3.26 GiB，但实际训练还要考虑 scale、optimizer state、activation 和 kernel 开销。

5.15 面试高频问题

5.15.1 LoRA 为什么省参数

回答：

LoRA 不学习完整的 ΔW ，而是把 ΔW 表示成两个低秩矩阵 A 和 B 的乘积。如果原矩阵参数量是 $d_{in} * d_{out}$ ，LoRA rank 为 r ，则参数量为 $2 * r * d_{in}$ 。

5.15.2 LoRA 是不是让原权重低秩

回答：

不是。LoRA 通常冻结原始权重 W ，只让微调增量 ΔW 采用低秩参数化。它的假设是任务适配所需的更新可以在低秩子空间中表达。

5.15.3 Rank 越大越好吗

回答：

不一定。Rank 越大表达能力越强，但参数量、显存、训练成本和过拟合风险也更高。Rank 太小可能学不动复杂任务。实际要根据任务需求选择。

5.15.4 PCA 对大模型有什么用

回答：

PCA 可以用来分析 embedding 或 hidden states 的主要变化方向，也可以做降维可视化和初步表示分析。它帮助我们理解高维表示的内在结构。

5.16 常见误区

1. 以为 LoRA 压缩的是原模型权重。
2. 只会说低秩省参数，不会算参数量。
3. 认为 rank 越大一定越好。
4. 忽略 LoRA 加在哪些模块。
5. 以为 LoRA 一定降低推理成本。
6. 用二维 embedding 可视化直接证明模型能力。
7. 把 PCA、SVD、LoRA 当成互不相关的概念。

5.17 小练习

1. 解释 rank 的直觉含义。
2. 用自己的话解释 SVD 为什么能做低秩近似。

3. 计算 4096×4096 矩阵全参微调参数量。
4. 如果 LoRA rank 为 8, $d_{in} = d_{out} = 4096$, 计算 LoRA 参数量。
5. 解释 LoRA 和低秩压缩的区别。
6. 说明为什么 rank 太小可能效果差。
7. 设计一个 LoRA rank 消融实验。

5.18 本章总结

矩阵分解和低秩近似是理解参数高效微调的重要数学基础。

本章最重要的结论：

1. Rank 表示矩阵中的独立方向数量。
2. SVD 可以把矩阵拆成从重要到不重要的方向。
3. 低秩近似用少数方向保留主要信息。
4. PCA 寻找数据方差最大的主方向。
5. LoRA 学的是低秩权重增量，而不是把原权重变低秩。
6. LoRA 参数量是 $r * (d_{in} + d_{out})$ 。
7. Rank 控制表达能力、成本和过拟合风险之间的 trade-off。
8. 低秩压缩和 LoRA 不是一回事。
9. Embedding 可视化只能作为探索，不能单独证明模型能力。

面试中讲 LoRA，不要只说“省显存”。要能从低秩矩阵、参数量、rank 选择、模块选择、表达能力和评估边界讲完整。

第六章：统计学习与泛化

统计学习关心一个核心问题：模型在训练数据上表现好，为什么能在没见过的数据上也表现好？这就是泛化。对大模型算法岗来说，泛化不是抽象理论，而是每天都会遇到的问题：SFT loss 下降但线上效果差，benchmark 分数高但真实用户不满意，DPO 后 win rate 提升但过度拒答，RAG 在固定评测集好但换文档就失效。

大模型的泛化问题尤其有趣。传统直觉认为参数越多越容易过拟合，但大模型参数巨大，却能在很多任务上泛化。这不是说过拟合消失了，而是过拟合、记忆、数据规模、模型规模、优化、正则化和评估污染之间的关系更复杂。

本章目标：讲清 bias-variance、overfitting、underfitting、regularization、泛化误差、大模型泛化悖论、benchmark contamination，以及如何在面试中成熟地回答“模型是在背诵还是泛化”。

0. 本讲资料边界与第二轮精修口径

第二轮精修时，本章按 WRITING_PLAN.md 先核对 Stanford CS229 统计学习与 bias-variance 相关讲义、The Elements of Statistical Learning、scikit-learn 关于 cross-validation / overfitting / underfitting 的文档、深度学习泛化相关论文，以及大模型记忆、去重和评估污染相关公开论文。

本章聚焦大模型算法岗面试和工程中最常用的统计学习口径：

1. 真实风险、经验风险、ERM 和泛化 gap。
2. 训练集、验证集、测试集、hidden test、线上分布之间的职责边界。
3. 过拟合、欠拟合、bias-variance 和正则化的诊断逻辑。
4. 大模型中的记忆、benchmark contamination、分布偏移和组合泛化。
5. 如何用一个小审计脚本同时检查 train / val / test、切片、污染和 early stopping。

本章不展开 VC dimension、Rademacher complexity、PAC-Bayes、uniform convergence、double descent 的完整数学证明，也不把统计学习写成传统机器学习教材。面试中更重要的是：能把“训练 loss 下降但线上效果差”拆成数据分布、污染、正则、模型容量、优化、评估设计和系统链路几个可排查方向。

6.1 泛化到底是什么

泛化指模型在未见数据上的表现。

训练集表现好，只说明模型拟合了训练样本；验证集、测试集、线上真实样本表现好，才说明模型有一定泛化能力。

在 LLM 中，泛化可以表现为：

1. 能回答训练集中没有逐字出现的问题。
2. 能把已学模式迁移到新表述。
3. 能组合多个知识点解决新任务。
4. 能根据 prompt 中的示例进行 in-context learning。
5. 能在新领域数据上保持合理表现。

但泛化不是无限的。模型可能在训练分布附近泛化很好，一旦遇到分布外输入、对抗样本、长尾场景或新工具环境，就会失败。

面试回答：

泛化是模型在未见数据上的表现。对 LLM 来说，泛化不只是测试集 accuracy，而是能否在新表述、新组合、新任务和真实用户分布上

6.1.1 关键公式与泛化诊断速查

真实风险，也叫期望风险：

$$R(f) = \mathbb{E}_{(x,y) \sim P}[\ell(f(x), y)]$$

其中 P 是真实数据分布， ℓ 是损失函数。真实风险是我们真正关心的目标，但完整的 P 通常不可见。

经验风险：

$$\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

其中 $S = \{(x_i, y_i)\}_{i=1}^n$ 是训练样本。训练时最常见的做法是经验风险最小化：

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_S(f)$$

泛化 gap 用验证集近似：

$$G_{\text{val}} = \hat{R}_{\text{val}}(f) - \hat{R}_{\text{train}}(f)$$

如果指标是 accuracy、win rate 这类越大越好的分数，更常看：

$$G_{\text{score}} = S_{\text{train}}(f) - S_{\text{val}}(f)$$

过拟合的常见信号是训练损失低、验证损失高，或者训练分数高、验证分数低；欠拟合的常见信号是训练和验证都差。

平方损失下的 bias-variance 分解可以写成：

$$\mathbb{E}_{D, \epsilon}[(\hat{f}_D(x) - y)^2] = B^2(x) + V(x) + \sigma^2$$

其中：

$$B^2(x) = (\mathbb{E}_D[\hat{f}_D(x)] - g(x))^2$$

$$V(x) = \mathbb{E}_D[(\hat{f}_D(x) - \mathbb{E}_D[\hat{f}_D(x)])^2]$$

$B^2(x)$ 表示模型平均预测和真实函数 $g(x)$ 的偏差， $V(x)$ 表示训练数据变化带来的预测波动， σ^2 是不可约噪声。

L2 正则化：

$$L_{\text{reg}}(\theta) = \hat{R}_S(f_\theta) + \lambda \|\theta\|_2^2$$

λ 越大，参数被约束得越强；但正则过强会欠拟合。

Early stopping 的验证集选择：

$$t^* = \arg \min_{t \in \mathcal{T}} \hat{R}_{\text{val}}(f_{\theta_t}), \quad \theta^* = \theta_{t^*}$$

其中 t 是 checkpoint step。不要用最终测试集来选 t^* ，否则测试集会被开发过程污染。

分布偏移可以用验证分布和目标分布之间的性能差近似：

$$\Delta_{\text{shift}} = \hat{R}_{\text{target}}(f) - \hat{R}_{\text{val}}(f)$$

如果用越大越好的分数，也可以看：

$$\Delta_{\text{score}} = S_{\text{val}}(f) - S_{\text{target}}(f)$$

训练-评估 exact overlap 率：

$$C_{\text{exact}} = \frac{1}{m} \sum_{j=1}^m \mathbf{1}[\text{hash}(e_j) \in H_{\text{train}}]$$

其中 e_j 是评估样本， H_{train} 是训练样本规范化后 hash 集合。 C_{exact} 大于 0 就要解释来源；如果是最终测试集，通常要阻断或重建评估集。

泛化审计门禁可以写成：

$$G_{\text{gen}} = G_{\text{gap}} \wedge G_{\text{shift}} \wedge G_{\text{leak}} \wedge G_{\text{slice}}$$

它不是一个新训练目标，而是上线或面试复盘时的 checklist：训练-验证 gap、分布偏移、污染和关键切片都不能只看平均分。

6.2 训练误差、验证误差和泛化误差

训练误差是在训练数据上的误差。

验证误差是在 held-out 数据上的误差。

泛化误差关心模型在真实数据分布上的期望误差。

实际中，我们无法直接看到完整真实分布，只能用验证集、测试集、线上抽样和人工评估近似。

如果训练误差低、验证误差高，通常说明过拟合。

如果训练误差和验证误差都高，通常说明欠拟合或训练不足。

如果验证集表现好但线上差，可能说明：

1. 验证集不代表线上分布。
2. 验证集被污染。
3. 评估指标不匹配用户体验。
4. 模型过拟合 benchmark。
5. 线上系统链路有问题。

6.3 Bias-Variance 直觉

Bias 和 variance 是传统统计学习中理解误差来源的经典框架。

Bias 可以理解为模型假设太强，学不出真实规律。

Variance 可以理解为模型对训练样本太敏感，换一批数据结果变化很大。

高 bias 的表现：

1. 训练集都学不好。
2. 模型太小或训练不足。
3. 特征或架构不合适。
4. Loss 长期很高。

高 variance 的表现：

1. 训练集很好，验证集差。
2. 小数据上过拟合。
3. 不同 seed 波动大。
4. 输出风格或能力不稳定。

在大模型中，这个框架仍然有用，但要小心。模型规模、数据规模和训练 recipe 会改变传统 bias-variance 直觉。大模型可能参数很多，但如果数据巨大、正则化合适、训练目标稳定，仍然能泛化。

面试回答：

Bias-variance 是理解误差来源的框架。Bias 高表示模型或训练不足，训练集也学不好；variance 高表示模型对训练样本过度敏感，

6.4 Overfitting：模型学到了训练集的细节

Overfitting 指模型过度贴合训练数据，导致未见数据表现变差。

LLM 中过拟合可能出现在多个阶段。

预训练中过拟合：

1. 重复数据过多。
2. 模型记住训练样本。
3. Benchmark contamination。
4. 训练 loss 低但真实能力一般。

SFT 中过拟合：

1. 数据量小。
2. 指令模板单一。
3. 模型学到固定回答风格。
4. 真实用户问题泛化差。

DPO/RLHF 中过拟合：

1. 过拟合偏好数据风格。
2. Reward model 泛化差。
3. 学会迎合 judge。
4. 过度拒答或冗长回答。

RAG 中过拟合：

1. 只在固定评测文档上调 prompt。
2. 只优化固定 query set。
3. 换文档、换领域后召回下降。

面试回答：

Overfitting 在 LLM 中不只发生在预训练，也会发生在 SFT、DPO、RAG prompt 和评估集上。典型表现是训练或固定 benchmark 表现很

6.5 Underfitting：模型还没学够

Underfitting 指模型连训练数据都拟合不好。

LLM 中可能来自：

1. 模型太小。
2. 训练 token 不够。
3. Learning rate 或优化器配置不好。
4. 数据质量太差。
5. Loss mask 错误。
6. 训练时间太短。
7. LoRA rank 太小或可训练模块太少。

面试中如果 loss 高，不要只说“模型不够大”。要先判断训练是否正常：数据、label、mask、优化器、学习率、batch、精度和代码实现都要排查。

6.6 Regularization：限制模型过度贴合训练数据

Regularization 是降低过拟合风险的方法。

常见正则化包括：

1. Weight decay。
2. Dropout。
3. Early stopping。
4. Data augmentation。
5. Label smoothing。
6. 数据去重和质量过滤。
7. LoRA 低秩约束。
8. KL penalty。

在 LLM 中，正则化不只是数学项，也可以是数据和系统策略。

例如：

1. 去重降低记忆风险。
2. 混入通用数据降低 continued pretraining 的遗忘。
3. KL penalty 限制 RLHF policy 偏离 reference。
4. LoRA rank 限制微调自由度。
5. Early stopping 防止小数据 SFT 过拟合。

面试回答：

Regularization 的目标是降低模型过度贴合训练数据的风险。LLM 中正则化不只包括 weight decay 和 dropout，也包括数据去重、ea

6.7 数据规模和模型规模

大模型泛化能力很大程度来自数据规模和模型规模共同作用。

如果模型大但数据少，可能 undertrain 或记忆训练数据。

如果数据多但模型小，模型容量不足，可能学不完复杂模式。

如果数据质量差，规模再大也会引入噪声、偏见、错误事实和安全风险。

这就是为什么 scaling law 和 Chinchilla 讨论模型参数、训练 token 和 compute 的匹配。

面试回答：

大模型不是参数越大越好，也不是 token 越多越好。泛化取决于模型容量、数据规模、数据质量和计算预算的匹配。模型太大但训练 optimal 配置和数据 mixture。

6.8 大模型为什么参数多但还能泛化

这是面试高频开放题。

传统直觉认为参数多容易过拟合。但大模型能泛化，可能来自多个因素：

1. 数据规模巨大。
2. 文本分布中有大量可复用结构。
3. 训练目标要求模型压缩统计规律。
4. Transformer 架构有归纳偏置。
5. 优化过程和正则化影响最终解。
6. 过参数化可能让优化更容易找到泛化较好的解。

更稳妥的回答不是“参数多也不会过拟合”，而是：

大模型仍然会过拟合和记忆，但它在海量数据、合适训练目标、优化 recipe 和过参数化条件下，也能学习可迁移的统计结构。泛化能 out、分布外、扰动、组合和真实用户评估验证，不能只看训练 loss 或公开榜单。

6.9 记忆和泛化不是二选一

LLM 既会记忆，也会泛化。

记忆更容易发生在：

1. 数据重复率高。
2. 样本独特且容易被识别。
3. 训练时间过长。
4. 模型容量足够大。
5. 数据包含代码、密钥、PII 或固定文本。

泛化更依赖：

1. 数据多样性。
2. 任务结构可迁移。
3. 模型容量和训练 token 匹配。
4. 评估覆盖改写和组合。
5. 训练没有只背固定模板。

面试回答：

LLM 的能力来自记忆、模式学习、组合泛化和上下文学习的共同作用。不能简单说它只是背诵，也不能说它完全理解。更可操作的判断

6.10 Benchmark contamination

Benchmark contamination 指评测数据泄漏进训练数据，导致模型在 benchmark 上表现虚高。

常见来源：

1. 公开 benchmark 被爬入预训练数据。
2. 训练集中包含题目或答案的改写版本。
3. SFT 或偏好数据包含测试集风格。
4. 开发过程中反复调 prompt 过拟合固定评测集。

污染的风险是：模型看起来能力强，但真实泛化并没有提升。

缓解方法：

1. 数据去重和近重复检测。
2. 保密评测集。
3. 动态评测集。
4. 分布外测试。
5. 线上真实样本评估。
6. 只看榜单不下结论，要做 bad case 分析。

面试回答：

Benchmark contamination 会让模型分数虚高，因为模型可能见过题目、答案或相似模板。判断真实泛化不能只看公开榜单，还要做污

6.11 分布外泛化和组合泛化

分布外泛化关注模型能否处理训练分布之外的输入。

组合泛化关注模型能否把已学组件重新组合到新场景。

例如模型见过：

A 比 B 大, B 比 C 大

也见过简单比较任务，但没见过某个具体名字组合。它能否推出 A 比 C 大，就是组合泛化的一类例子。

LLM 评估中要特别关注：

1. 改写问题。
2. 新实体组合。
3. 多步推理。
4. 长上下文扰动。
5. 对抗提示。
6. 工具环境变化。

6.12 泛化和评估设计

如果评估集太窄，模型很容易“看起来泛化”。

好的泛化评估应包括：

1. In-distribution 测试。
2. Out-of-distribution 测试。
3. 改写和扰动测试。
4. 组合泛化测试。
5. 长尾样本。
6. 人工 bad case 分析。
7. 线上真实抽样。

对于 RAG，还要测试新文档、新 query、新权限和新领域。

对于 DPO，还要测试 helpfulness、安全拒答、over-refusal、格式稳定性和能力回归。

对于多模态，还要测试新图片风格、OCR、图表、计数和空间关系。

6.13 最小可运行泛化审计 demo

下面这个 demo 不调用任何机器学习库，只用 toy 数据模拟三类模型：memorizer 记住训练样本、keyword_rule 学到可迁移规则、underfit 基本没学到目标。它同时审计 train / validation / test gap、bias-variance、L2 正则、early stopping、线上分布偏移和训练-评估 exact overlap。

```
from collections import defaultdict
```

```
train = [  
    {"id": "tr_01", "text": "solve math triangle", "label": 1},  
    {"id": "tr_02", "text": "debug code loop", "label": 1},  
    {"id": "tr_03", "text": "reasoning chain puzzle", "label": 1},  
    {"id": "tr_04", "text": "math proof induction", "label": 1},  
    {"id": "tr_05", "text": "code bug stack", "label": 1},  
    {"id": "tr_06", "text": "reasoning compare objects", "label": 1},  
    {"id": "tr_07", "text": "weather small talk", "label": 0},  
    {"id": "tr_08", "text": "movie preference chat", "label": 0},  
    {"id": "tr_09", "text": "music playlist request", "label": 0},  
    {"id": "tr_10", "text": "casual greeting", "label": 0},  
]
```

```
validation = [  
    {"id": "va_01", "text": "math word problem", "label": 1},  
    {"id": "va_02", "text": "code task parsing", "label": 1},  
    {"id": "va_03", "text": "reasoning chain with distractor", "label": 1},  
    {"id": "va_04", "text": "math geometry proof", "label": 1},  
    {"id": "va_05", "text": "weather forecast chat", "label": 0},  
    {"id": "va_06", "text": "recipe suggestion", "label": 0},  
]
```

```
test = [  
    {"id": "te_01", "text": "math contest problem", "label": 1},  
    {"id": "te_02", "text": "code parse json", "label": 1},  
    {"id": "te_03", "text": "reasoning logic grid", "label": 1},  
    {"id": "te_04", "text": "code unit test", "label": 1},  
    {"id": "te_05", "text": "travel planning", "label": 0},  
    {"id": "te_06", "text": "friendly greeting", "label": 0},  
]
```

```
production = [  
    {"id": "pr_01", "text": "math word problem", "label": 1, "slice": "in_domain"},  
    {"id": "pr_02", "text": "code review task", "label": 1, "slice": "in_domain"},  
    {"id": "pr_03", "text": "weather small talk", "label": 0, "slice": "in_domain"},  
    {"id": "pr_04", "text": "algebra equation", "label": 1, "slice": "new_terms"},  
    {"id": "pr_05", "text": "python bug report", "label": 1, "slice": "new_terms"},  
    {"id": "pr_06", "text": "friendly greeting", "label": 0, "slice": "new_terms"},  
]
```

```
def normalize(text):  
    return " ".join(text.lower().split())
```

```
memorized_labels = {normalize(row["text"]): row["label"] for row in train}
```

```
def predict_memorizer(row):  
    return memorized_labels.get(normalize(row["text"]), 0)
```

```

def predict_keyword_rule(row):
    words = set(normalize(row["text"]).split())
    return int(bool(words & {"math", "code", "reasoning"}))

def predict_underfit(row):
    return 0

def accuracy(predict_fn, rows):
    correct = sum(int(predict_fn(row) == row["label"]) for row in rows)
    return correct / len(rows)

def summarize_model(name, predict_fn):
    train_acc = accuracy(predict_fn, train)
    val_acc = accuracy(predict_fn, validation)
    test_acc = accuracy(predict_fn, test)
    return {
        "train": round(train_acc, 3),
        "val": round(val_acc, 3),
        "test": round(test_acc, 3),
        "gap": round(train_acc - val_acc, 3),
    }

models = {
    "memorizer": predict_memorizer,
    "keyword_rule": predict_keyword_rule,
    "underfit": predict_underfit,
}

score_summary = {name: summarize_model(name, fn) for name, fn in models.items()}

diagnosis = {
    "memorizer_overfit": score_summary["memorizer"]["train"] >= 0.9 and score_summary["memorizer"]["gap"] >= 0.25,
    "underfit_detected": score_summary["underfit"]["train"] < 0.7 and score_summary["underfit"]["val"] < 0.7,
    "rule_generalizes": score_summary["keyword_rule"]["val"] >= 0.9 and score_summary["keyword_rule"]["test"] >= 0.9,
}

def bias_variance(predictions, true_value, noise_var):
    mean_pred = sum(predictions) / len(predictions)
    bias_sq = (mean_pred - true_value) ** 2
    variance = sum((p - mean_pred) ** 2 for p in predictions) / len(predictions)
    return {
        "mean_prediction": round(mean_pred, 3),
        "bias_sq": round(bias_sq, 3),
        "variance": round(variance, 3),
        "noise": round(noise_var, 3),
        "expected_error": round(bias_sq + variance + noise_var, 3),
    }

weights = [3.0, 4.0]

```

```

task_loss = 0.42
lambda_l2 = 0.01
l2_penalty = lambda_l2 * sum(w * w for w in weights)
regularized_loss = task_loss + l2_penalty

checkpoints = [
    {"step": 100, "train_loss": 0.82, "val_loss": 0.86},
    {"step": 200, "train_loss": 0.55, "val_loss": 0.61},
    {"step": 300, "train_loss": 0.38, "val_loss": 0.49},
    {"step": 400, "train_loss": 0.25, "val_loss": 0.54},
    {"step": 500, "train_loss": 0.16, "val_loss": 0.67},
]
best_checkpoint = min(checkpoints, key=lambda row: row["val_loss"])

def slice_scores(predict_fn, rows):
    grouped = defaultdict(list)
    for row in rows:
        grouped[row["slice"]].append(row)
    return {name: round(accuracy(predict_fn, subset), 3) for name, subset in sorted(grouped.items())}

prod_acc = accuracy(predict_keyword_rule, production)
shift = {
    "val_acc": score_summary["keyword_rule"]["val"],
    "prod_acc": round(prod_acc, 3),
    "score_drop": round(score_summary["keyword_rule"]["val"] - prod_acc, 3),
    "slice_scores": slice_scores(predict_keyword_rule, production),
}

eval_for_leakage = validation + [{"id": "eval_copy", "text": "math proof induction", "label": 1}]
train_hash_to_id = {normalize(row["text"]): row["id"] for row in train}
exact_overlaps = [
    (row["id"], train_hash_to_id[normalize(row["text"])])
    for row in eval_for_leakage
    if normalize(row["text"]) in train_hash_to_id
]
overlap_rate = len(exact_overlaps) / len(eval_for_leakage)

checks = {
    "memorizer_overfits": diagnosis["memorizer_overfit"],
    "underfit_detected": diagnosis["underfit_detected"],
    "rule_generalizes_clean_sets": diagnosis["rule_generalizes"],
    "early_stopping_before_overfit": best_checkpoint["step"] == 300,
    "shift_detected": shift["score_drop"] > 0.2 and min(shift["slice_scores"].values()) < 0.7,
    "contamination_detected": overlap_rate > 0,
}

print("score_summary=", score_summary)
print("diagnosis=", diagnosis)
print("bias_variance=", bias_variance([1.2, 1.8, 2.4], true_value=2.0, noise_var=0.1))
print("regularization=", {"task_loss": task_loss, "l2_penalty": round(l2_penalty, 3), "total": round(regularized_loss, 3)})
print("best_checkpoint=", best_checkpoint)
print("shift=", shift)
print("exact_overlaps=", exact_overlaps, "overlap_rate=", round(overlap_rate, 3))

```

```
print("checks=", checks)
```

预期输出：

```
score_summary= {'memorizer': {'train': 1.0, 'val': 0.333, 'test': 0.333, 'gap': 0.667}, 'keyword_rule': {'train': 1.0, 'v
diagnosis= {'memorizer_overfit': True, 'underfit_detected': True, 'rule_generalizes': True}
bias_variance= {'mean_prediction': 1.8, 'bias_sq': 0.04, 'variance': 0.24, 'noise': 0.1, 'expected_error': 0.38}
regularization= {'task_loss': 0.42, 'l2_penalty': 0.25, 'total': 0.67}
best_checkpoint= {'step': 300, 'train_loss': 0.38, 'val_loss': 0.49}
shift= {'val_acc': 1.0, 'prod_acc': 0.667, 'score_drop': 0.333, 'slice_scores': {'in_domain': 1.0, 'new_terms': 0.333}}
exact_overlaps= [('eval_copy', 'tr_04')] overlap_rate= 0.143
checks= {'memorizer_overfits': True, 'underfit_detected': True, 'rule_generalizes_clean_sets': True, 'early_stopping_be
```

这个 demo 的重点不是分类器本身，而是诊断流程：

1. memorizer 训练集 100%，验证和测试很差，说明训练集好不等于泛化好。
2. underfit 训练集和验证集都差，说明模型或训练配置还没学到目标。
3. keyword_rule 在干净验证集和测试集好，但遇到 algebra、python 这类新表述就掉分，说明分布偏移仍然存在。
4. Early stopping 选择验证损失最低的 checkpoint，而不是训练损失最低的 checkpoint。
5. 只要 final eval 出现训练样本 exact overlap，就要解释污染来源，不能直接把分数当作泛化能力。

6.14 面试高频问题

6.14.1 大模型是在背诵还是泛化

回答：

两者都有。LLM 会记忆高频、重复或特殊训练样本，也会通过 next-token prediction 学习语言、知识和任务模式，从而在新表述和新

6.14.2 如何判断 SFT 过拟合

回答：

如果 SFT train loss 持续下降，但验证集、人评或真实用户样本变差，可能过拟合。还要看模型是否学到固定模板、回答变机械、对

6.14.3 如何缓解过拟合

回答：

可以从数据、模型、优化和评估四层做。数据上去重、增加多样性和质量过滤；模型上用 LoRA rank、dropout、weight decay 等限制 out、分布外、改写和线上抽样，避免只过拟合固定 benchmark。

6.14.4 Benchmark contamination 为什么危险

回答：

污染会让 benchmark 分数虚高，因为模型可能见过题目、答案或相似模板。这会误导模型比较和研究结论。需要做数据去重、近重复

6.15 常见误区

1. 认为参数多一定过拟合。
2. 认为大模型不会过拟合。
3. 只看训练 loss 判断模型好坏。
4. 把 benchmark 高分等同于真实泛化。
5. 不区分记忆、模式学习和组合泛化。
6. SFT 只看 loss，不看真实用户问题。
7. DPO 只看 win rate，不看 over-refusal 和能力回归。
8. RAG 只看固定问题集，不测试新文档和新领域。

6.16 小练习

1. 用自己的话解释泛化。
2. 举一个 LLM 记忆的例子和一个泛化的例子。
3. 解释 bias 和 variance 的区别。
4. 说明 SFT 过拟合有哪些表现。
5. 设计一个检测 benchmark contamination 的方案。
6. 设计一个组合泛化测试集。
7. 说明 LoRA rank 如何影响过拟合风险。
8. 写出真实风险、经验风险和泛化 gap 的公式。
9. 用 0 依赖 Python 写一个小脚本，比较 memorizer、rule-based model 和 underfit model 的 train / validation / test accuracy。
10. 给定多个 checkpoint 的 train loss 和 validation loss，选择 early stopping step，并解释为什么不能用 test set 选 checkpoint。
11. 构造一个线上分布偏移切片，说明为什么验证集高分仍可能线上低分。

6.17 本章总结

统计学习与泛化帮助我们判断模型是否真的学到了可迁移能力。

本章最重要的结论：

1. 泛化是模型在未见数据上的表现。
2. 训练误差低不代表泛化好。
3. 经验风险是训练集上的平均损失，真实风险是目标分布上的期望损失，二者差距就是泛化问题的核心。
4. Bias 高通常训练集也差，variance 高通常训练集好但验证集差。
5. LLM 的过拟合可能发生在预训练、SFT、DPO、RAG 和评估集上。
6. Regularization 在 LLM 中包括 weight decay、dropout、early stopping、数据去重、LoRA 低秩约束和 KL penalty。
7. 大模型既会记忆，也会泛化。
8. Benchmark contamination 会让公开分数虚高。
9. 泛化评估要包含分布外、扰动、组合和真实用户样本。

面试中讲泛化，不要说“大模型参数多所以会过拟合”或“大模型规模大所以不会过拟合”。成熟回答应该区分训练拟合、记忆、泛化、数据污染、评估设计和真实任务表现。

第七章：贝叶斯与不确定性

贝叶斯思想和不确定性，是理解大模型可靠性的关键。LLM 很会生成流畅文本，但流畅不等于正确，自信不等于可信。很多真实问题不是“模型能不能答”，而是“模型什么时候应该答、什么时候应该检索、什么时候应该承认不知道、什么时候应该交给人工或工具验证”。

概率论告诉我们如何表示不确定性，贝叶斯思想告诉我们如何根据证据更新判断。大模型系统里的 RAG、工具调用、事实核验、置信度校准、安全拒答、人审抽检和上线门禁，都可以用这套直觉理解。

本章重点：先验、似然、后验、MAP、校准、不确定性估计、模型置信度、幻觉、高风险场景决策。

0. 本讲资料边界与第二轮精修口径

第二轮精修时，本章按 WRITING_PLAN.md 先核对 Stanford CS229 关于 Bayes rule、MLE / MAP 和正则化的讲义，scikit-learn 关于概率校准和 calibration curve 的文档，On Calibration of Modern Neural Networks，以及 aleatoric / epistemic uncertainty 的经典深度学习不确定性资料。

本章聚焦大模型算法岗常用的不确定性数学和工程口径：

1. Bayes rule、prior、likelihood、evidence、posterior。
2. MLE、MAP 与 L2 正则的关系。
3. Aleatoric uncertainty、epistemic uncertainty、entropy、self-consistency。

4. Confidence、calibration、ECE、Brier score 和 abstention。
5. RAG / 工具 / verifier / 人审如何作为证据更新可靠性判断。

本章不展开完整贝叶斯统计教材、共轭分布大全、MCMC、变分推断、Gaussian process 或深度贝叶斯神经网络训练细节。面试中更重要的是：能把“模型很自信但可能错”转成可度量的置信度、证据、校准、拒答和高风险门禁问题。

7.1 为什么大模型需要不确定性意识

大模型常见失败不是完全不会，而是“不知道自己不知道”。

典型表现：

1. 对不存在的论文编造引用。
2. 对资料不足的问题给出确定答案。
3. 对高风险医疗、法律、金融建议过度自信。
4. RAG 没检索到证据仍然强行回答。
5. 工具调用失败后编造工具结果。
6. LLM judge 给出分数但缺少校准。

如果模型或系统缺少不确定性意识，就容易把“语言上合理”误当成“事实上可靠”。

面试回答：

LLM 的输出流畅性不等于可靠性。很多高风险错误来自模型无法正确表达不确定性。可靠系统需要让模型在证据不足时检索、调用工具。

7.1.1 关键公式与不确定性速查

Bayes rule:

$$p(H | E) = \frac{p(E | H)p(H)}{p(E)}$$

其中 H 是假设, E 是证据, p(H) 是 prior, p(E|H) 是 likelihood, p(H|E) 是 posterior, p(E) 是 evidence / normalizer。

两个互斥假设下，后验可以直接按归一化写：

$$p(H | E) = \frac{p(E | H)p(H)}{p(E | H)p(H) + p(E | \neg H)p(\neg H)}$$

MLE:

$$\theta_{\text{MLE}} = \arg \max_{\theta} p(D | \theta)$$

MAP:

$$\theta_{\text{MAP}} = \arg \max_{\theta} p(\theta | D) = \arg \max_{\theta} p(D | \theta)p(\theta)$$

取负对数后：

$$\theta_{\text{MAP}} = \arg \min_{\theta} -\log p(D | \theta) - \log p(\theta)$$

如果先验偏好较小参数，例如零均值高斯先验：

$$p(\theta) \propto \exp(-\lambda \|\theta\|_2^2)$$

则 MAP 会出现类似 L2 正则的目标：

$$L_{\text{MAP}}(\theta) = L_{\text{NLL}}(\theta) + \lambda \|\theta\|_2^2$$

Beta-Bernoulli 更新常用于理解 “模型在某类任务上的可靠率”：

$$\alpha' = \alpha + s, \quad \beta' = \beta + f$$

$$\mathbb{E}[q \mid D] = \frac{\alpha'}{\alpha' + \beta'}$$

其中 s 是成功次数， f 是失败次数， q 可以理解为某个切片上的正确率或可靠率。

预测熵：

$$H(p) = - \sum_i p_i \log p_i$$

熵越高，分布越分散；但低熵也不保证事实正确。

置信度：

$$c(x) = \max_y p_\theta(y \mid x)$$

校准的理想性质：

$$p(Y = \hat{Y} \mid c(X) = r) = r$$

Expected Calibration Error:

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{n} |\text{acc}(S_b) - \text{conf}(S_b)|$$

Brier score:

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2$$

其中 p_i 是预测为正类或正确的概率， y_i 是 0/1 标签。

Abstention / selective prediction 可以写成：

$$a(x) = \begin{cases} \text{answer}, & c(x) \geq \tau \wedge r(x) < r_{\text{high}} \\ \text{verify}, & c(x) \geq \tau \wedge r(x) \geq r_{\text{high}} \\ \text{abstain}, & c(x) < \tau \end{cases}$$

这里 $r(x)$ 表示风险等级。高风险任务即使置信度高，也可能需要工具验证或人工复核。

7.2 先验、似然和后验

贝叶斯公式的核心是：根据证据更新信念。

可以直观写成：

posterior is proportional to likelihood times prior

也就是：

$$P(H \mid E) = P(E \mid H) * P(H) / P(E)$$

其中：

1. H 是假设。
2. E 是证据。
3. $P(H)$ 是先验。
4. $P(E \mid H)$ 是似然。
5. $P(H \mid E)$ 是后验。

在大模型系统里，可以这样理解：

1. 模型参数和语言模式提供先验。
2. 用户问题、RAG 文档、工具结果提供证据。
3. 系统最终回答应该基于证据更新后的判断。

如果没有外部证据，模型很容易依赖语言先验猜答案。

7.3 语言先验和证据

LLM 有很强的语言先验。它知道 “某类问题通常应该怎么回答”，但这不代表它知道当前问题的真实答案。

例如用户问：

某公司 2024 年第三季度的内部故障报告结论是什么？

如果模型没有内部文档，它可能根据类似报告的写法编造一个很像真的答案。

这就是语言先验压过证据的问题。

RAG 的作用可以理解为给模型提供证据，让回答从 “凭先验续写” 变成 “基于证据回答”。

面试回答：

LLM 的语言先验很强，能生成看起来合理的答案，但不一定有事实依据。RAG、工具调用和引用约束的作用，就是把外部证据引入上下

7.4 MAP：先验和数据的折中

MAP 是 maximum a posteriori，也就是最大后验估计。

直觉上，MLE 只看数据似然；MAP 同时考虑数据和先验。

可以直观理解为：

MAP = fit the data while respecting the prior

在机器学习中，很多正则化都可以从 MAP 角度理解。

例如 weight decay 可以理解为一种先验：我们不希望参数过大。训练时既要拟合数据，也要避免参数变得过大。

面试回答：

MLE 只最大化数据似然，MAP 会同时考虑先验和数据。很多正则化可以从 MAP 角度理解，例如 weight decay 相当于偏好较小参数的先

7.5 不确定性的两类来源

机器学习中常把不确定性分成两类。

第一类是 aleatoric uncertainty，也就是数据本身的随机性或噪声。

例如：

1. 用户问题含糊。
2. 标注本身有歧义。
3. 同一个 prompt 有多个合理答案。
4. 图像或语音输入质量差。

第二类是 epistemic uncertainty，也就是模型知识不足带来的不确定性。

例如：

1. 模型没见过某领域知识。
2. 当前问题分布外。
3. RAG 没有检索到证据。
4. 模型能力不足以完成推理。

两者处理方式不同。

1. 数据噪声类不确定性，可能需要澄清问题、输出多个可能答案或表达条件。
2. 知识不足类不确定性，可能需要检索、工具、人工或拒答。

面试回答：

不确定性可以分为数据本身噪声和模型知识不足。前者来自输入歧义或任务本身多解，后者来自模型不了解或分布外。LLM 系统要区分

7.6 校准：置信度是否可信

Calibration 关注模型的置信度是否和真实正确率匹配。

如果模型对一批回答都说自己 80% 确定，理想情况下这些回答大约 80% 应该正确。

LLM 常见校准问题：

1. 错了也很自信。
2. 对简单问题过度保守。
3. 对分布外问题不会表达不知道。
4. SFT/RLHF 让语言更自信，但不一定让事实更可靠。
5. Judge model 给分稳定，但和人工不一致。

校准不是只看模型自己说“我有 90% 把握”。语言表达的置信度和真实概率未必一致。

面试回答：

校准衡量模型置信度是否能反映真实正确率。LLM 可能语气自信但事实错误，所以不能直接把自然语言中的自信表达当作可靠概率。高

7.7 如何估计 LLM 的不确定性

LLM 不确定性估计没有一个万能方法。常见信号包括：

1. Token probability 或 log probability。
2. 输出 entropy。
3. 多次采样答案是否一致。
4. Self-consistency。
5. RAG 是否检索到足够证据。
6. 引用是否支持结论。
7. Verifier 或工具检查是否通过。
8. LLM judge 和人工评审是否一致。

这些信号都不是完美的。

例如 token probability 高，不代表事实正确；多次采样一致，也可能一致地错；RAG 检索到文档，也可能文档不支持答案。

成熟做法是多信号结合，并按风险等级决定是否自动回答。

7.8 幻觉和不确定性

幻觉往往发生在模型应该不确定但仍然生成确定答案的时候。

常见原因：

1. Next-token prediction 奖励流畅续写。
2. SFT 数据常包含完整答案，模型学会总要回答。
3. RLHF 可能偏好自信、完整、友好的回答。
4. 模型缺少外部事实验证。
5. Prompt 没有明确要求资料不足时拒答。

降低幻觉的一条主线是：让模型在证据不足时不要假装知道。

方法包括：

1. RAG 和引用约束。
2. 工具验证。
3. 训练不知道和澄清样本。
4. Abstention eval。
5. Unsupported claim 检测。
6. 高风险场景人工复核。

7.9 RAG 中的不确定性处理

RAG 系统中，不确定性可能来自多个环节：

1. 文档是否入库。
2. 检索是否召回正确证据。
3. Reranker 是否排序正确。
4. Context 是否被截断或污染。
5. LLM 是否基于证据回答。
6. 引用是否真的支持结论。

如果证据不足，系统应该允许模型回答“不知道”或“资料中未找到”。

面试回答：

RAG 不是只要检索到文本就回答。系统要判断检索证据是否足够支持答案。如果召回为空、证据冲突或 citation 不支持结论，模型应

7.10 高风险场景如何决策

不确定性在高风险场景尤其重要。

例如：

1. 医疗建议。
2. 法律判断。
3. 金融决策。
4. 安全和危险操作。
5. 企业内部权限数据。

高风险场景不能只看模型是否会回答，还要看：

1. 是否有可靠证据。

2. 是否需要专业人士确认。
3. 是否应该降低细节级别。
4. 是否需要拒答。
5. 是否需要记录审计日志。

面试回答：

高风险场景中，不确定时不应该强答。系统应该按风险等级处理：低风险可以给一般信息，高风险需要引用证据、工具验证、人工复核。

7.11 LLM-as-a-Judge 的校准

LLM judge 也是模型，也有不确定性。

常见偏差：

1. 偏好长答案。
2. 偏好某种格式。
3. 位置偏差。
4. 自我偏好。
5. 对事实错误不敏感。
6. 对高风险样本判断不稳。

校准方法：

1. 用人工 gold set 对齐。
2. 做 pairwise 且随机交换顺序。
3. 要求结构化 rubric。
4. 抽样人工复核。
5. 报告置信区间。
6. 分任务、分风险切片分析。

面试回答：

LLM-as-a-judge 可以规模化评估开放式输出，但它必须校准。要用人工 gold set 检查它和人类一致性，控制位置偏差、长度偏差和格式偏差。

7.12 最小可运行贝叶斯与校准审计 demo

下面这个 demo 不调用模型，也不依赖第三方库。它用 toy 数据演示：Bayes 更新、Beta-Bernoulli 可靠率估计、MAP 相比 MLE 的先验收缩、entropy / self-consistency、ECE / Brier 校准指标，以及按风险等级做 answer / verify / abstain 决策。

```
import math
from collections import Counter
```

```
def bayes_binary(prior_h, likelihood_e_given_h, likelihood_e_given_not_h):
    numerator = likelihood_e_given_h * prior_h
    denominator = numerator + likelihood_e_given_not_h * (1 - prior_h)
    return numerator / denominator
```

```
def beta_update(alpha, beta, successes, failures):
    post_alpha = alpha + successes
    post_beta = beta + failures
    mean = post_alpha / (post_alpha + post_beta)
    map_estimate = (post_alpha - 1) / (post_alpha + post_beta - 2)
    return {"alpha": post_alpha, "beta": post_beta, "mean": mean, "map": map_estimate}
```

```
def gaussian_map_estimate(observations, noise_var, prior_mean, prior_var):
```

```

sample_mean = sum(observations) / len(observations)
mle = sample_mean
precision_data = len(observations) / noise_var
precision_prior = 1 / prior_var
numerator = precision_data * sample_mean + precision_prior * prior_mean
denominator = precision_data + precision_prior
return {"mle": mle, "map": numerator / denominator}

def entropy(probs):
    return -sum(p * math.log(p) for p in probs if p > 0)

def calibration_report(records, num_bins=5):
    bins = [[] for _ in range(num_bins)]
    for conf, correct in records:
        index = min(int(conf * num_bins), num_bins - 1)
        bins[index].append((conf, correct))

    total = len(records)
    ece = 0.0
    summaries = []
    for index, bucket in enumerate(bins):
        if not bucket:
            continue
        avg_conf = sum(conf for conf, _ in bucket) / len(bucket)
        acc = sum(correct for _, correct in bucket) / len(bucket)
        gap = abs(avg_conf - acc)
        ece += len(bucket) / total * gap
        summaries.append({
            "bin": f"{index / num_bins:.1f}-{(index + 1) / num_bins:.1f}",
            "n": len(bucket),
            "avg_conf": round(avg_conf, 3),
            "acc": round(acc, 3),
            "gap": round(gap, 3),
        })

    brier = sum((conf - correct) ** 2 for conf, correct in records) / total
    return {"ece": ece, "brier": brier, "bins": summaries}

def self_consistency(answers):
    counts = Counter(answers)
    total = len(answers)
    probs = [count / total for count in counts.values()]
    top_answer, top_count = counts.most_common(1)[0]
    return {
        "top_answer": top_answer,
        "top_share": top_count / total,
        "entropy": entropy(probs),
    }

def route_decision(posterior_reliable, risk):
    if posterior_reliable < 0.60:

```

```

        return "abstain"
    if risk == "high":
        return "verify"
    return "answer"

prior_reliable = 0.70
posterior_strong = bayes_binary(prior_reliable, 0.90, 0.20)
posterior_weak = bayes_binary(prior_reliable, 0.20, 0.70)

beta_posterior = beta_update(alpha=2, beta=2, successes=8, failures=2)
point_estimates = gaussian_map_estimate(
    observations=[2.4, 2.0, 1.6],
    noise_var=0.25,
    prior_mean=0.0,
    prior_var=1.0,
)

token_distribution = [0.70, 0.20, 0.10]
sampled_answers = ["answer_a", "answer_a", "answer_b", "answer_a", "answer_c"]
uncertainty = {
    "token_entropy": entropy(token_distribution),
    "self_consistency": self_consistency(sampled_answers),
}

calibration_records = [
    (0.95, 1), (0.90, 1), (0.85, 0), (0.80, 1),
    (0.75, 1), (0.70, 0), (0.65, 1), (0.60, 0),
    (0.55, 0), (0.40, 0),
]
calibration = calibration_report(calibration_records)

decision_cases = {
    "low_risk_strong_evidence": route_decision(posterior_strong, "low"),
    "high_risk_strong_evidence": route_decision(posterior_strong, "high"),
    "low_risk_weak_evidence": route_decision(posterior_weak, "low"),
}

checks = {
    "strong_evidence_raises_posterior": posterior_strong > prior_reliable,
    "weak_evidence_lowers_posterior": posterior_weak < prior_reliable,
    "map_shrinks_mle_toward_prior": point_estimates["map"] < point_estimates["mle"],
    "calibration_detects_gap": calibration["ece"] > 0.10,
    "high_risk_requires_verification": decision_cases["high_risk_strong_evidence"] == "verify",
    "weak_evidence_abstains": decision_cases["low_risk_weak_evidence"] == "abstain",
}

print("bayes_update=", {"strong_evidence": round(posterior_strong, 3), "weak_evidence": round(posterior_weak, 3)})
print("beta_posterior=", {k: round(v, 3) for k, v in beta_posterior.items()})
print("mle_map=", {k: round(v, 3) for k, v in point_estimates.items()})
print("uncertainty=", {
    "token_entropy": round(uncertainty["token_entropy"], 3),
    "self_consistency": {
        "top_answer": uncertainty["self_consistency"]["top_answer"],
        "top_share": round(uncertainty["self_consistency"]["top_share"], 3),
    }
})

```

```

        "entropy": round(uncertainty["self_consistency"]["entropy"], 3),
    },
})
print("calibration=", {
    "ece": round(calibration["ece"], 3),
    "brier": round(calibration["brier"], 3),
    "bins": calibration["bins"],
})
print("decision_cases=", decision_cases)
print("checks=", checks)

```

预期输出：

```

bayes_update= {'strong_evidence': 0.913, 'weak_evidence': 0.4}
beta_posterior= {'alpha': 10, 'beta': 4, 'mean': 0.714, 'map': 0.75}
mle_map= {'mle': 2.0, 'map': 1.846}
uncertainty= {'token_entropy': 0.802, 'self_consistency': {'top_answer': 'answer_a', 'top_share': 0.6, 'entropy': 0.95}}
calibration= {'ece': 0.215, 'brier': 0.227, 'bins': [{'bin': '0.4-0.6', 'n': 2, 'avg_conf': 0.475, 'acc': 0.0, 'gap': 0.475}, {'bin': '0.6-0.8', 'n': 4, 'avg_conf': 0.675, 'acc': 0.5, 'gap': 0.175}, {'bin': '0.8-1.0', 'n': 4, 'avg_conf': 0.875, 'acc': 0.75, 'gap': 0.125}]}
decision_cases= {'low_risk_strong_evidence': 'answer', 'high_risk_strong_evidence': 'verify', 'low_risk_weak_evidence': 'verify', 'high_risk_weak_evidence': 'verify'}
checks= {'strong_evidence_raises_posterior': True, 'weak_evidence_lowers_posterior': True, 'map_shrinks_mle_toward_prior': True}

```

这个 demo 对应真实 LLM 系统中的几个面试重点：

1. RAG 证据、工具结果和引用检查都可以看作更新可靠性后验的证据。
2. MAP 不是 “比 MLE 一定更准”，而是把先验约束显式纳入估计。
3. Token 熵和 self-consistency 只是信号，不是事实正确性的证明。
4. ECE 和 Brier 可以发现 “模型概率看起来高，但真实正确率跟不上” 的校准问题。
5. 高风险场景不应只按置信度自动回答，还要引入验证、拒答或人工复核。

7.13 面试高频问题

7.13.1 贝叶斯公式的直觉是什么

回答：

贝叶斯公式描述如何根据证据更新信念。先验是看到证据前的判断，似然表示某个假设下看到证据的可能性，后验是结合证据后的判断。

7.13.2 LLM 为什么会自信地错

回答：

LLM 优化的是 token 条件概率，不是事实验证。SFT/RLHF 还可能强化流畅、完整和自信的回答风格。如果模型缺少证据或处于分布外，模型可能会自信地给出错误答案。

7.13.3 如何让模型知道自己不知道

回答：

需要训练、评估和系统共同处理。训练上加入资料不足、需要澄清和拒答样本；系统上接入 RAG、工具验证和引用检查；评估上看 abstain 比例。

7.13.4 如何校准 LLM judge

回答：

用人工 gold set 校准 judge，比较它和人工标注的一致性；做位置交换和长度偏差检查；固定 judge prompt 和版本；输出结构化 rubric。

7.14 常见误区

1. 把模型语气自信当成真实置信度。

2. 认为 log probability 高就事实正确。
3. 认为 RAG 检索到文档就一定可靠。
4. 只靠 prompt 解决不知道的问题。
5. 把 LLM judge 当作绝对真值。
6. 不区分输入歧义和模型知识不足。
7. 高风险场景仍然追求“尽量回答”。

7.15 小练习

1. 用自己的话解释先验、似然和后验。
2. 举一个 LLM 语言先验压过证据的例子。
3. 解释 calibration 为什么重要。
4. 设计一个检测 unsupported claim 的评估方法。
5. 说明 RAG 中证据不足时系统应该怎么做。
6. 设计一个 LLM judge 校准流程。
7. 说明医疗问答场景中不确定性应该如何处理。
8. 写出 Bayes rule、MAP 和 ECE 的公式，并说明每个量的含义。
9. 用 0 依赖 Python 写一个 Bayes 更新 demo：强证据提高后验，弱证据降低后验。
10. 给一组 confidence / correctness 记录，计算 ECE 和 Brier score。
11. 设计一个 answer / verify / abstain 路由规则，说明高风险场景为什么不能只看置信度。

7.16 本章总结

贝叶斯和不确定性帮助我们“从模型会回答”升级到“模型是否应该回答”。

本章最重要的结论：

1. 贝叶斯思想是根据证据更新信念。
2. LLM 有强语言先验，但先验不等于事实证据。
3. MAP 可以理解为数据和先验的折中。
4. 不确定性既可能来自数据歧义，也可能来自模型知识不足。
5. Calibration 衡量置信度是否可信。
6. ECE 和 Brier score 可以把校准问题变成可审计指标。
7. 幻觉常来自模型在不确定时仍自信生成。
8. RAG、工具、引用和人审都是管理不确定性的系统手段。
9. 高风险场景需要 answer / verify / abstain 决策，而不是一律自动回答。
10. LLM judge 也需要校准和人工复核。

面试中讲不确定性，不要只说“让模型说不知道”。要能讲清先验、证据、校准、评估、系统防护和高风险场景决策。

第八章：强化学习数学

强化学习数学是理解 RLHF、PPO、reward model、policy、value、advantage 和 KL penalty 的基础。大模型后训练里，很多候选人知道“RLHF 是用人类反馈训练模型”，但讲不清为什么它是强化学习问题、PPO 在更新什么、value model 有什么用、advantage 为什么能降低方差、为什么要加 KL 约束。

本章不追求完整 RL 教材，而是只讲大模型算法岗最需要的强化学习数学直觉：MDP、policy、reward、return、value、advantage、policy gradient、PPO，以及这些概念如何映射到 RLHF。

0. 本讲资料边界与第二轮精修口径

第二轮精修时，本章按 WRITING_PLAN.md 先核对 Sutton & Barto 强化学习教材中的 MDP、return、value、policy gradient 基础，PPO 原论文，InstructGPT / RLHF 论文，以及 DPO 论文的偏好优化推导。

本章聚焦大模型算法岗最常用的强化学习数学：

1. MDP、state、action、transition、reward、policy 和 discount。
2. Return、value function、Q function、advantage 和 baseline。
3. Policy gradient 为什么提高高 advantage 行为的概率。
4. PPO clipped objective、KL penalty 和 RLHF 中的 reference model。
5. Reward model pairwise loss、DPO loss 与带 KL 约束偏好优化的联系。

本章不展开完整动态规划、Bellman 收敛证明、Q-learning / Actor-Critic 全族算法、GAE 完整推导、多智能体 RL、在线环境探索或生产级 RLHF 系统实现。面试中更重要的是：能把 LLM 生成映射成 policy / trajectory / reward，并能解释 PPO、value、advantage、KL 和 reward hacking 的数学角色。

8.1 为什么 RLHF 需要强化学习

SFT 让模型模仿高质量示范回答，但示范回答不一定覆盖所有可能输出。偏好数据告诉我们：同一个 prompt 下，哪个回答更好。

RLHF 的目标是让模型生成更高 reward 的回答。

这很像强化学习：

1. 模型是 policy。
2. Prompt 是初始状态或上下文。
3. 生成 token 是 action。
4. 完整回答形成 trajectory。
5. Reward model 给完整回答打分。
6. PPO 等算法根据 reward 更新 policy。

面试回答：

RLHF 可以看成语言模型上的强化学习。Policy 是当前语言模型，action 是生成 token，trajectory 是完整回答，reward 来自 reward model。

8.1.1 关键公式与 RLHF 数学速查

MDP 可以写成：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$$

其中 $\mathcal{S}$ 是状态集合， $\mathcal{A}$ 是动作集合， $P(s'|s, a)$ 是转移概率， $r(s, a)$ 是奖励， γ 是折扣因子。

Policy：

$$\pi_{\theta}(a | s)$$

在 LLM 中， s_t 是 prompt 加已生成 token， a_t 是下一个 token。

Trajectory：

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$$

折扣回报：

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$

State value：

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$$

Action value:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Advantage:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

实际 PPO / RLHF 中常用估计值:

$$\hat{A}_t = \hat{G}_t - V_\phi(s_t)$$

Policy gradient 直觉:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t]$$

也就是: 正 advantage 提高对应动作概率, 负 advantage 降低对应动作概率。

PPO 概率比:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

PPO clipped surrogate:

$$L_{\text{clip}}(\theta) = \mathbb{E}_t[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

RLHF 中常见 KL 约束目标:

$$J_{\text{RLHF}}(\theta) = \mathbb{E}_{x, y \sim \pi_\theta}[r_\psi(x, y) - \beta D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))]$$

Reward model pairwise loss:

$$L_{\text{RM}} = -\log \sigma(r_\psi(x, y_w) - r_\psi(x, y_l))$$

其中 y_w 是 chosen / winner, y_l 是 rejected / loser。

DPO loss:

$$L_{\text{DPO}} = -\log \sigma(\beta[\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x) - \log \pi_{\text{ref}}(y_w \mid x) + \log \pi_{\text{ref}}(y_l \mid x)])$$

DPO 不显式训练 reward model, 也不跑 PPO, 但仍然通过 reference model 和 **beta** 控制 policy 偏离程度。

8.2 MDP 的基本组成

MDP 是 Markov Decision Process，强化学习中描述决策问题的基本框架。

它通常包含：

1. State: 状态。
2. Action: 动作。
3. Transition: 状态转移。
4. Reward: 奖励。
5. Policy: 策略。
6. Discount: 折扣因子。

在游戏或机器人中，状态可能是环境观测，动作可能是移动或控制指令。

在语言模型中，可以粗略映射为：

1. State: 当前 prompt 加已经生成的 tokens。
2. Action: 下一个 token。
3. Transition: 把 token 追加到上下文。
4. Reward: 完整回答由 reward model 打分。
5. Policy: 语言模型的 token 分布。

注意：LLM 的 RLHF 和传统 RL 不完全一样。它通常没有复杂外部环境交互，reward 多数在完整回答后给出，而且 policy 已经来自预训练和 SFT，不是从零学习。

8.3 Policy：模型如何选择动作

Policy 表示在某个状态下选择动作的概率分布。

在 LLM 中，policy 就是：

$\pi(\text{next token} \mid \text{current context})$

每一步生成时，模型根据当前上下文输出词表概率分布。

如果 policy 改变，模型的生成行为也会改变。例如 RLHF 希望 policy 更倾向于生成 helpful、honest、harmless 的回答。

面试回答：

Policy 是状态到动作分布的映射。在 LLM 中，状态是当前上下文，动作是下一个 token，policy 就是语言模型输出的 token 概率分布。

8.4 Reward 和 Return

Reward 是某一步或某段 trajectory 的奖励。

Return 是从当前开始累计得到的奖励。

传统 RL 中，每一步都可能有 reward。LLM RLHF 中，reward 通常在完整回答后给出，例如 reward model 对 prompt+response 打一个分数。

这带来两个问题：

1. Reward 很稀疏，只有完整回答后才知道好坏。
2. 很难判断每个 token 对最终 reward 的贡献。

这就是为什么需要 value model 和 advantage 来辅助训练。

8.5 Value function：估计未来能拿多少 reward

Value function 用来估计某个状态下未来能获得多少 reward。

直觉上：

$\text{value}(\text{state}) = \text{expected future return from this state}$

在 PPO/RLHF 中，value model 可以帮助估计当前生成过程的基线，降低 policy gradient 的方差。

如果没有 value baseline，单次 reward 的噪声会很大，policy 更新不稳定。

面试回答：

Value function 估计某个状态未来能获得的期望回报。在 RLHF 中，reward 往往只在完整回答后给出，噪声较大。Value model 提供

8.6 Advantage：比预期好多少

Advantage 衡量一个 action 或 trajectory 比当前预期好多少。

直觉：

$\text{advantage} = \text{actual return} - \text{expected value}$

如果 advantage 为正，说明这个动作比预期好，应该提高它的概率。

如果 advantage 为负，说明比预期差，应该降低它的概率。

面试回答：

Advantage 表示实际回报相对 value baseline 好多少。正 advantage 的 token 或行为应该被增强，负 advantage 应该被抑制。相比重

8.7 Policy gradient 的直觉

Policy gradient 的目标是直接优化 policy，让高 reward 的行为概率变大。

直觉更新方向：

increase log probability of actions with positive advantage

decrease log probability of actions with negative advantage

在语言模型里，如果某个回答得到高 reward，那么生成这个回答的 token 序列概率会被整体提高。

但如果只这样做，模型可能出现以下问题：

1. 为了 reward 牺牲语言质量。
2. 学会 reward hacking。
3. 输出变得重复或极端。
4. 偏离 SFT 模型太远。

因此 RLHF 通常加入 KL 约束。

8.8 PPO 的核心直觉

PPO 是 Proximal Policy Optimization，核心是不要让 policy 一次更新太大。

它通过比较新旧 policy 对同一 action 的概率比值，限制更新幅度。

直觉：

1. 如果新 policy 和旧 policy 差不多，可以更新。
2. 如果新 policy 变化太大，就 clip 或惩罚。
3. 这样能避免一次更新把模型行为推得太远。

面试回答：

PPO 的核心思想是限制 policy 每次更新幅度。它用新旧 policy 的概率比值衡量变化，如果变化太大就 clip，避免训练不稳定。在 R

8.9 KL penalty 在 RLHF 中的作用

如果只最大化 reward，模型可能利用 reward model 漏洞。

常见风险：

1. Reward hacking。
2. 输出模板化。
3. 语言质量退化。
4. 过度拒答。
5. 偏离原模型能力。

KL penalty 的作用是让 policy 不要偏离 reference model 太远。

直觉目标：

$\text{maximize reward} - \beta * \text{KL}(\text{policy} || \text{reference})$

β 控制约束强度。

1. β 太小，模型可能乱跑。
2. β 太大，模型学不动。

面试回答：

RLHF 中 KL penalty 用来约束 policy 接近 reference model。Reward model 只是人类偏好的 proxy，如果无约束最大化 reward，模型

8.10 RLHF 完整流程的数学视角

RLHF 通常包括：

1. 训练或准备 SFT model。
2. 收集同一 prompt 下多个回答的人类偏好。
3. 训练 reward model，让 chosen 得分高于 rejected。
4. 用 policy 生成回答。
5. Reward model 给回答打分。
6. 加上 KL penalty 得到最终训练信号。
7. 用 PPO 更新 policy。
8. 用人工评估和安全评估检查效果。

数学视角看，RLHF 不是“人类实时给奖励”，而是用偏好数据训练一个 reward proxy，再优化 policy。

这也解释了 RLHF 的风险：如果 reward proxy 错了，policy 会放大这个错误。

8.11 Reward hacking

Reward hacking 指模型找到提高 reward 的捷径，但输出不符合真实人类意图。

LLM 中可能表现为：

1. 输出冗长但空泛。
2. 迎合 reward model 喜欢的格式。
3. 看似安全但过度拒答。
4. 编造引用但语气可信。
5. 利用 judge 或 reward model 的盲区。

缓解方法：

1. 高质量偏好数据。
2. Reward model holdout。
3. KL 约束。
4. 人工评估。
5. 安全和能力回归。

6. Red teaming。

面试回答：

Reward hacking 是模型优化 reward proxy 时找到捷径，但真实质量没有提升。RLHF 必须配合 KL 约束、holdout 偏好集、人工评估、

8.12 DPO 和 RLHF 的关系

DPO 不显式训练 reward model，也不跑 PPO，但它和带 KL 约束的偏好优化有理论联系。

RLHF：

preference data -> reward model -> PPO optimize policy

DPO：

preference data -> directly optimize policy

DPO 直接利用 chosen/rejected 的 log probability 构造 loss，让 policy 相比 reference 更偏好 chosen。

面试回答：

DPO 可以看作把带 KL 约束的偏好优化改写成直接基于 preference pair 的监督式目标。它省掉 reward model 和 PPO loop，训练更简

8.13 最小可运行强化学习数学审计 demo

下面这个 demo 不调用模型，也不依赖第三方库。它用 toy 数值演示 return、advantage、PPO clipping、KL penalty、reward model pairwise loss 和 DPO loss。重点是让读者能把 RLHF 的几个数学对象放到同一张表里，而不是只背流程。

```
import math
```

```
def discounted_returns(rewards, gamma):
    returns = []
    running = 0.0
    for reward in reversed(rewards):
        running = reward + gamma * running
        returns.append(running)
    return list(reversed(returns))
```

```
def clip(x, low, high):
    return max(low, min(high, x))
```

```
def ppo_terms(old_probs, new_probs, advantages, epsilon):
    rows = []
    objective = 0.0
    for old_p, new_p, adv in zip(old_probs, new_probs, advantages):
        ratio = new_p / old_p
        unclipped = ratio * adv
        clipped = clip(ratio, 1 - epsilon, 1 + epsilon) * adv
        surrogate = min(unclipped, clipped)
        objective += surrogate
        rows.append({
            "ratio": ratio,
            "adv": adv,
            "unclipped": unclipped,
            "clipped": clipped,
```

```

        "surrogate": surrogate,
    })
    return rows, objective / len(rows)

def kl_divergence(policy, reference):
    return sum(p * math.log(p / q) for p, q in zip(policy, reference) if p > 0)

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def neg_log_sigmoid(x):
    return -math.log(sigmoid(x))

def reward_model_loss(chosen_score, rejected_score):
    return neg_log_sigmoid(chosen_score - rejected_score)

def dpo_loss(policy_chosen_logp, policy_rejected_logp, ref_chosen_logp, ref_rejected_logp, beta):
    policy_gap = policy_chosen_logp - policy_rejected_logp
    ref_gap = ref_chosen_logp - ref_rejected_logp
    margin = beta * (policy_gap - ref_gap)
    return neg_log_sigmoid(margin), margin

rewards = [0.0, 0.2, 1.0]
values = [0.70, 1.25, 0.20]
gamma = 0.90
returns = discounted_returns(rewards, gamma)
advantages = [g - v for g, v in zip(returns, values)]

old_probs = [0.50, 0.40, 0.25]
new_probs = [0.60, 0.30, 0.40]
ppo_rows, ppo_objective = ppo_terms(old_probs, new_probs, advantages, epsilon=0.20)

policy_dist = [0.60, 0.25, 0.15]
reference_dist = [0.50, 0.30, 0.20]
kl_policy_ref = kl_divergence(policy_dist, reference_dist)
reward_score = 1.40
beta_kl = 0.30
rlhf_objective = reward_score - beta_kl * kl_policy_ref

rm_loss = reward_model_loss(chosen_score=1.8, rejected_score=0.4)
dpo, dpo_margin = dpo_loss(
    policy_chosen_logp=-1.1,
    policy_rejected_logp=-2.0,
    ref_chosen_logp=-1.3,
    ref_rejected_logp=-1.7,
    beta=0.5,
)

checks = {

```

```

    "returns_include_future_reward": returns[0] > rewards[0] and returns[1] > rewards[1],
    "positive_advantage_increases_ratio": ppo_rows[0]["ratio"] > 1 and advantages[0] > 0,
    "negative_advantage_decreases_ratio": ppo_rows[1]["ratio"] < 1 and advantages[1] < 0,
    "ppo_clip_limits_large_ratio": ppo_rows[2]["clipped"] < ppo_rows[2]["unclipped"],
    "kl_penalizes_reward": rlhf_objective < reward_score,
    "dpo_prefers_chosen": dpo_margin > 0 and dpo < 0.70,
}

print("returns=", [round(x, 3) for x in returns])
print("advantages=", [round(x, 3) for x in advantages])
print("ppo_rows=", [
    {key: round(value, 3) for key, value in row.items()}
    for row in ppo_rows
])
print("ppo_objective=", round(ppo_objective, 3))
print("kl_policy_ref=", round(kl_policy_ref, 3), "rlhf_objective=", round(rlhf_objective, 3))
print("rm_loss=", round(rm_loss, 3))
print("dpo=", {"loss": round(dpo, 3), "margin": round(dpo_margin, 3)})
print("checks=", checks)

```

预期输出:

```

returns= [0.99, 1.1, 1.0]
advantages= [0.29, -0.15, 0.8]
ppo_rows= [{'ratio': 1.2, 'adv': 0.29, 'unclipped': 0.348, 'clipped': 0.348, 'surrogate': 0.348}, {'ratio': 0.75, 'adv': 0.15, 'unclipped': -0.112, 'clipped': -0.12, 'surrogate': -0.12}, {'ratio': 1.6, 'adv': 0.8, 'unclipped': 1.28, 'clipped': 1.28, 'surrogate': 1.28}]
ppo_objective= 0.396
kl_policy_ref= 0.021 rlhf_objective= 1.394
rm_loss= 0.22
dpo= {'loss': 0.576, 'margin': 0.25}
checks= {'returns_include_future_reward': True, 'positive_advantage_increases_ratio': True, 'negative_advantage_decreases_ratio': True, 'ppo_clip_limits_large_ratio': True, 'kl_penalizes_reward': True, 'dpo_prefers_chosen': True}

```

注意这个 demo 也暴露两个常见误区:

1. Discounted return 不是 “当前 reward”，它包含未来 reward。
2. PPO 的 probability ratio 来自新旧 policy，不是由 advantage 自动决定的；训练目标会推动正 advantage 的动作概率上升、负 advantage 的动作概率下降，但一次实际更新后的 ratio 还取决于优化器、batch 和约束。

真实训练时，更重要的是看 PPO objective、KL、reward-human gap、长度偏置、安全回归和人工评估，而不是只盯 reward score。

8.14 RLHF 和普通 RL 的区别

RLHF 和传统 RL 有明显不同:

1. Policy 不是从零开始，而是预训练/SFT 后的语言模型。
2. 环境通常不是复杂物理环境，而是 token 生成过程。
3. Reward 来自 learned reward model，不是真实环境奖励。
4. Reward 通常在完整回答后给出。
5. 需要强 KL 约束保持语言能力。
6. 评估不能只看 reward，还要看人类偏好、安全和能力回归。

面试回答:

RLHF 不是传统意义上从零探索环境。它是在已有 SFT 模型上，用人类偏好训练 reward model，再用 PPO 等方法微调 policy。Reward

8.15 面试高频问题

8.15.1 PPO 在 RLHF 中做什么

回答：

PPO 用 reward model 给出的奖励更新 policy model，让模型更倾向于生成高 reward 回答。它通过限制新旧 policy 的变化幅度提高训练稳定性。

8.15.2 Value model 有什么用

回答：

Value model 估计当前状态未来能获得的期望 reward，作为 baseline 帮助计算 advantage。这样可以降低 policy gradient 的方差，提高训练稳定性。

8.15.3 Advantage 是什么

回答：

Advantage 表示实际回报比当前 value 预期好多少。正 advantage 表示行为比预期好，应该提高概率；负 advantage 表示比预期差，应该降低概率。

8.15.4 为什么 RLHF 不能只看 reward score

回答：

Reward model 只是人类偏好的 proxy，可能有偏差和漏洞。Policy 如果过度优化 reward，可能 reward hacking，导致分数升高但真实质量下降。

8.16 常见误区

1. 认为 RLHF 是人类实时给模型奖励。
2. 认为 PPO 是训练 reward model 的方法。
3. 只说 PPO，不讲 policy、reward、value、advantage。
4. 认为 reward score 越高模型越好。
5. 忽略 KL penalty。
6. 把 DPO 说成完全没有约束。
7. 不知道 RLHF 通常发生在 SFT 之后。

8.17 小练习

1. 把 LLM 生成过程映射成 state、action、policy 和 reward。
2. 用自己的话解释 value function。
3. 用自己的话解释 advantage。
4. 说明 PPO 为什么要限制 policy 更新幅度。
5. 举 3 个 reward hacking 的例子。
6. 解释 RLHF 为什么需要 KL penalty。
7. 比较 RLHF 和 DPO 的训练流程。
8. 写出 return、value、Q function 和 advantage 的公式。
9. 手算一个三步 trajectory 的 discounted return 和 advantage。
10. 给定 old policy prob、new policy prob 和 advantage，计算 PPO ratio、clipped surrogate 和平均 objective。
11. 给定 policy / reference 分布，计算 KL penalty 后的 RLHF objective。
12. 给定 chosen / rejected 的 reward score，计算 reward model pairwise loss。
13. 给定 policy 和 reference 的 chosen / rejected log probability，计算 DPO margin 和 DPO loss。

8.18 本章总结

强化学习数学帮助我们理解 RLHF 背后的优化结构。

本章最重要的结论：

1. LLM 的 policy 是 token 分布。
2. 生成 token 可以看作 action。
3. 完整回答可以看作 trajectory。
4. Reward model 给回答提供奖励信号。
5. Value model 估计期望回报。
6. Advantage 衡量比预期好多少。
7. Policy gradient 增强高 advantage 行为。
8. PPO 限制 policy 更新幅度。
9. KL penalty 防止 policy 偏离 reference。
10. DPO 直接优化 preference pair，但仍依赖 reference model 和 beta。
11. RLHF 不能只看 reward score，必须防 reward hacking。

面试中讲 RLHF，不要只背流程。要能把 policy、reward、value、advantage、PPO、KL 和 reward hacking 串成一个完整优化故事。

第九章：评估统计与显著性

大模型评估最危险的习惯，是看到一个分数变高就认为模型变好了。真实情况是：评估集是样本，用户分布在变化，LLM judge 有偏差，human eval 有噪声，线上 A/B 有随机波动，多次试验还会制造偶然显著。统计显著性和实验设计的作用，就是帮你判断观察到的提升是否可信，是否值得上线。

本章不是完整统计学教材，而是面向大模型面试讲清：假设检验、p-value、置信区间、bootstrap、样本量、A/B test、pairwise eval 和 ablation 的核心直觉。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修时，参考了 NIST 工程统计手册中关于均值、标准误、置信区间和假设检验的基础口径，Efron bootstrap 的经典论文，SciPy bootstrap 文档中对重采样置信区间的说明，statsmodels mcnemar 文档中对配对二分类检验的实现口径，以及 Benjamini-Hochberg FDR 论文的多重比较控制思想。

本章只服务于大模型评估和面试表达：把有限评估集上的分数、差异、置信区间、paired test、McNemar、bootstrap、样本量和多重比较讲清楚。它不展开完整数理统计教材、严格渐近证明、复杂贝叶斯实验设计、生产级 A/B 平台实现，也不把任何单一显著性规则写成上线模型的充分条件。

9.1 为什么模型评估需要统计意识

假设两个模型结果是：

Model A accuracy = 82.1%

Model B accuracy = 82.6%

这 0.5% 的提升可能是真提升，也可能只是随机波动。

大模型评估的不确定性来自：

1. 测试集抽样噪声。
2. Decoding 随机性。
3. Human eval 标注噪声。
4. LLM-as-a-judge 偏差。
5. 离线和线上分布不一致。
6. 多次实验带来的偶然发现。

面试回答：

模型评估不能只比较两个平均分。测试集只是总体分布的样本，评估过程也有标注、采样和 judge 噪声。统计意识的核心是判断观察

9.1.1 关键公式与评估统计速查

设评估样本有 n 条，每条样本的指标值是 x_i ，例如 0/1 correctness、pairwise win、judge score 或人工评分。
样本均值：

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

样本方差：

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

样本均值的标准误：

$$SE(\bar{x}) = \frac{s}{\sqrt{n}}$$

常用正态近似 95% 置信区间：

$$\bar{x} \pm 1.96SE(\bar{x})$$

成对评估中，新旧模型在同一条样本上的差异：

$$d_i = m_i^{new} - m_i^{old}$$

成对差异均值和标准误：

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad SE(\bar{d}) = \frac{s_d}{\sqrt{n}}$$

二分类 paired evaluation 的 McNemar discordant counts：

$$b = \sum_i \mathbf{1}[old_i = 0, new_i = 1], \quad c = \sum_i \mathbf{1}[old_i = 1, new_i = 0]$$

连续性校正后的 McNemar 统计量：

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

Bootstrap percentile 置信区间：

$$CI_{boot} = [q_{\alpha/2}(T_b^*), q_{1-\alpha/2}(T_b^*)]$$

双比例实验的粗略单组样本量估计：

$$n_{\text{each}} \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 p(1-p)}{\Delta^2}$$

Bonferroni 校正：

$$p_i \leq \frac{\alpha}{m}$$

Benjamini-Hochberg FDR 判定：

$$p_{(i)} \leq \frac{iq}{m}$$

这些公式的共同作用不是“机械给出能不能上线”，而是把点估计、随机波动、样本量、复杂指标、paired 对照和多重尝试放进同一个评估决策框架。

9.2 随机变量、均值和方差

评估分数可以看成随机变量。

我们真正关心的是模型在真实任务分布上的期望表现，但只能用有限样本估计。

均值表示平均表现，方差表示波动程度。

如果样本量小，均值不稳定；如果任务差异大，方差也会大。

这解释了为什么同一个模型在不同 benchmark、不同抽样、不同 judge prompt 下分数可能不同。

在 LLM 评估里， x_i 可以是很多东西：单题 accuracy、pairwise win、人工 rubric score、RAG faithfulness、latency、cost 或 safety violation indicator。均值告诉你总体水平，方差告诉你不同样本之间波动多大。开放生成和长尾用户任务的方差通常更大，所以同样 200 条样本，开放问答评估的不确定性可能远高于固定答案分类题。

9.3 标准误和样本量

标准误衡量样本均值的不确定性。

直觉上：

1. 样本量越大，均值越稳定。
2. 样本方差越大，均值越不稳定。
3. 长尾任务、开放生成和人工评分通常方差更大。

大模型评估中，样本量不足会导致：

1. 置信区间很宽。
2. 小提升无法判断。
3. 分层分析不可靠。
4. 容易被少数 outlier 影响。

如果目标是比较两个比例类指标，例如 accuracy、win rate 或任务成功率，可以用上面的粗略样本量公式做一阶估算。这里 p 是基线比例的保守估计， Δ 是最小可检测提升， $z_{1-\alpha/2}$ 控制显著性水平， $z_{1-\beta}$ 控制统计功效。实际项目中还要考虑 paired design、分层流量、重复用户、序贯查看、护栏指标和流量可用性。

面试回答：

样本量决定评估结果的不确定性。样本越少，均值估计越不稳定，置信区间越宽。大模型评估里不能只报告平均分，还要报告样本量、

9.4 假设检验和 p-value

假设检验用于判断观察到的差异是否可能由随机波动产生。

零假设通常是：

H0: 新模型和旧模型没有真实差异

备择假设是:

H1: 新模型和旧模型存在真实差异

p-value 的直觉是: 如果零假设成立, 观察到当前或更极端结果的概率。

常见误区: p-value 不是 “新模型无效的概率”, 也不是 “新模型更好的概率”。

对大模型评估来说, p-value 还必须和实验设计绑定理解。未提前定义主指标、反复调 prompt、频繁中途看结果、从很多分层里挑显著提升, 都会让 p-value 比看起来更乐观。p-value 小只能说明 “在当前检验口径下结果不像纯随机波动”, 不能替代 effect size、置信区间、样本代表性、污染检查和护栏指标。

面试回答:

p-value 表示在零假设成立时, 观察到当前或更极端结果的概率。它不是新模型无效的概率。假设检验只能说明差异是否不像随机噪声

9.5 置信区间: 比单个分数更有用

置信区间给出效果大小的不确定范围。

例如:

win rate lift = +2.0%
95% CI = [+0.6%, +3.4%]

这比只说 “提升 2%” 更有信息。

如果置信区间很宽, 说明不确定性大; 如果区间跨过 0, 说明提升可能不稳定。

置信区间比单个 p-value 更适合面试表达, 因为它同时告诉你 “提升可能有多大” 和 “最坏合理情况可能有多差”。例如 +2.0% 的 lift, 如果 95% CI 是 [+0.1%, +3.9%], 说明方向大致正但下界很小; 如果 CI 是 [-1.0%, +5.0%], 说明当前实验还不足以确认新模型真的优于旧模型。

面试回答:

置信区间能表达效果大小和不确定性。相比只报告 p-value, 我更希望看到点估计、置信区间、样本量和实验设计。如果区间很宽或跨

9.6 统计显著和实际显著

统计显著不等于值得上线。

例如:

准确率提升 0.1%, p-value < 0.05
推理成本增加 40%

这可能统计显著, 但没有实际价值。

上线决策还要看:

1. 效果大小。
2. 成本。
3. 延迟。
4. 安全回归。
5. 分层用户影响。
6. 失败样本类型。

面试回答:

统计显著回答差异是否不像随机噪声, 实际显著回答差异是否值得行动。大模型上线不能只看 p-value, 还要看效果大小、成本、延迟

9.7 Paired evaluation

比较两个模型时，最好让它们在上一批样本上评估。这叫 paired evaluation。

好处是控制样本难度。

例如同一道题：

Model A: wrong

Model B: correct

这种差异比两个模型分别在不同样本上的平均分更可靠。

Pairwise human eval 也属于常见 paired 形式：同一个 prompt 下比较 A/B 两个回答，让标注员判断谁更好或 tie。

成对评估的核心统计对象不是两个模型各自的平均分，而是每条样本上的差异 d_i 。这能抵消样本难度差异：难题对两个模型都难，简单题对两个模型都简单，真正关心的是同一题上谁改善了。

如果是二分类正确 / 错误结果，尤其适合看 discordant pairs：

1. b: 旧模型错、新模型对。
2. c: 旧模型对、新模型错。

McNemar 检验只看这两类“不一致样本”，因为两个模型都对或都错的样本不能说明新旧模型谁更好。样本很小时，更稳妥的做法是用 exact binomial 口径；样本较大时可以用卡方近似。

面试回答：

模型比较最好做 paired evaluation，让两个模型在同一批样本上比较。这样能控制样本难度，降低方差。Pairwise human eval 中通

9.8 Bootstrap

Bootstrap 是估计指标不确定性的实用方法。

基本做法：

1. 有一批评估样本。
2. 从中有放回抽样，形成新的样本集。
3. 计算指标。
4. 重复很多次。
5. 用指标分布估计置信区间。

Bootstrap 适合大模型评估，因为很多指标很复杂，很难写出解析方差公式。

例如：

1. Pairwise win rate。
2. LLM judge score。
3. RAG faithfulness。
4. 多维 rubric 人评分。
5. 长文本摘要质量。

对 paired evaluation，bootstrap 时应以“样本”为单位重采样，并在每次重采样里同时取旧模型和新模型在该样本上的结果，然后重新计算差异。不要分别对新旧模型独立抽样，否则会破坏配对结构，方差估计会偏离真实比较方式。

面试回答：

Bootstrap 通过重复有放回采样估计指标分布，从而得到置信区间。它特别适合复杂指标，比如 pairwise win rate、LLM judge 分数

9.9 最小可运行评估统计审计 demo

下面这个 0 依赖 demo 用同一批样本比较旧模型和新模型，覆盖 accuracy、成对差异、标准误、正态近似置信区间、paired bootstrap、McNemar exact p-value、样本量估计、多重比较校正和上线门禁。

```

from math import comb, sqrt
import random

def mean(values):
    return sum(values) / len(values)

def sample_variance(values):
    avg = mean(values)
    return sum((x - avg) ** 2 for x in values) / (len(values) - 1)

def standard_error(values):
    return sqrt(sample_variance(values) / len(values))

def normal_ci(values, z=1.96):
    avg = mean(values)
    se = standard_error(values)
    return avg - z * se, avg + z * se

def percentile(sorted_values, q):
    idx = int(q * (len(sorted_values) - 1))
    return sorted_values[idx]

def paired_bootstrap_ci(old, new, n_bootstrap=2000, alpha=0.05, seed=13):
    rng = random.Random(seed)
    n = len(old)
    scores = []

    for _ in range(n_bootstrap):
        indices = [rng.randrange(n) for _ in range(n)]
        old_score = mean([old[i] for i in indices])
        new_score = mean([new[i] for i in indices])
        scores.append(new_score - old_score)

    scores.sort()
    return percentile(scores, alpha / 2), percentile(scores, 1 - alpha / 2)

def mcnemar_counts(old, new):
    b = sum(1 for o, n in zip(old, new) if o == 0 and n == 1)
    c = sum(1 for o, n in zip(old, new) if o == 1 and n == 0)
    return b, c

def mcnemar_exact_pvalue(b, c):
    # Under H0, discordant improvements follow Binomial(b+c, 0.5).
    n = b + c
    tail = sum(comb(n, k) for k in range(0, min(b, c) + 1)) / (2 ** n)
    return min(1.0, 2 * tail)

```

```

def mcnemar_chi2_cc(b, c):
    if b + c == 0:
        return 0.0
    return ((abs(b - c) - 1) ** 2) / (b + c)

def two_proportion_sample_size(p=0.5, delta=0.05, z_alpha=1.96, z_power=0.84):
    return int((2 * (z_alpha + z_power) ** 2 * p * (1 - p)) / (delta ** 2) + 0.999)

def bonferroni(pvalues, alpha=0.05):
    cutoff = alpha / len(pvalues)
    return [p for p in pvalues if p <= cutoff]

def benjamini_hochberg(pvalues, q=0.05):
    ordered = sorted(pvalues)
    m = len(ordered)
    k = 0
    for i, p in enumerate(ordered, start=1):
        if p <= i * q / m:
            k = i
    return ordered[:k]

old = [1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0]
new = [1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1]
diffs = [n - o for o, n in zip(old, new)]

old_acc = mean(old)
new_acc = mean(new)
mean_diff = mean(diffs)
normal_low, normal_high = normal_ci(diffs)
boot_low, boot_high = paired_bootstrap_ci(old, new)
b, c = mcnemar_counts(old, new)
exact_p = mcnemar_exact_pvalue(b, c)
chi2_cc = mcnemar_chi2_cc(b, c)
sample_size = two_proportion_sample_size()
pvalues = [0.004, 0.018, 0.031, 0.049, 0.20]
bonferroni_pass = bonferroni(pvalues)
bh_pass = benjamini_hochberg(pvalues)

latency_p95_ms = 820
cost_multiplier = 1.25
safety_regression = False
primary_decision_ready = boot_low > 0 and exact_p < 0.05
guardrails_ok = latency_p95_ms <= 900 and cost_multiplier <= 1.30 and not safety_regression
ship = primary_decision_ready and guardrails_ok

report = {
    "accuracy": {"old": round(old_acc, 3), "new": round(new_acc, 3)},
    "paired_diff": {
        "diffs": diffs,
        "mean": round(mean_diff, 3),
    },
}

```

```

    "se": round(standard_error(diffs), 3),
    "normal_ci": (round(normal_low, 3), round(normal_high, 3)),
    "bootstrap_ci": (round(boot_low, 3), round(boot_high, 3)),
},
"mcnemar": {
    "old_wrong_new_correct_b": b,
    "old_correct_new_wrong_c": c,
    "chi2_cc": round(chi2_cc, 3),
    "exact_p": round(exact_p, 5),
},
"sample_size": {"n_each_for_5pct_mde": sample_size},
"multiple_comparison": {
    "bonferroni_pass": bonferroni_pass,
    "bh_pass": bh_pass,
},
"decision": {
    "primary_decision_ready": primary_decision_ready,
    "guardrails_ok": guardrails_ok,
    "ship": ship,
},
"checks": {
    "bootstrap_ci_computed": boot_low <= mean_diff <= boot_high,
    "mcnemar_counts_match": b == 5 and c == 1,
    "sample_size_positive": sample_size > 0,
    "bonferroni_stricter_than_bh": len(bonferroni_pass) <= len(bh_pass),
    "decision_blocks_weak_evidence": ship is False,
},
}

```

`print(report)`

这段代码的关键输出含义：

1. `paired_diff` 先报告新旧模型问题差异，再给出标准误、正态近似 CI 和 paired bootstrap CI。
2. `mcnemar` 只统计旧错新对的 `b` 和旧对新错的 `c`，再给出 exact p-value。
3. `sample_size` 给出双比例实验的粗略单组样本量，帮助判断当前实验是不是太小。
4. `multiple_comparison` 展示 Bonferroni 比 Benjamini-Hochberg 更保守。
5. `decision` 说明即使平均 accuracy 提升，若 bootstrap CI 包含 0 或 exact p-value 不显著，也不应直接上线。

9.10 A/B test

线上 A/B test 用真实用户流量比较两个版本。

它比离线评估更接近真实体验，但也更复杂。

需要提前定义：

1. 主指标。
2. 护栏指标。
3. 样本量和实验周期。
4. 分流方式。
5. 停止规则。
6. 是否允许中途查看。

样本量要在实验前估算，而不是看到趋势后临时决定。中途反复查看并在结果好看时停止，会提高假阳性风险。真实 A/B 平台通常还要处理重复用户、流量污染、实验互相干扰、分层不均衡、节假日效应和 guardrail 回滚。

大模型 A/B test 常见主指标：

1. 任务成功率。
2. 用户满意度。
3. 采纳率。
4. 留存。
5. 人工转接率。

护栏指标：

1. 延迟。
2. 成本。
3. 安全拦截。
4. 误拒率。
5. 投诉率。
6. 错误率。

面试回答：

线上 A/B test 要提前定义主指标、护栏指标、样本量和停止规则。大模型实验不能只看满意度提升，还要看延迟、成本、安全、误拒

9.11 多重比较问题

如果你试了很多模型、prompt、数据配比、指标和分层，总会有一些结果偶然显著。

这就是多重比较问题。

大模型项目中常见来源：

1. 同时看很多 benchmark。
2. 多个 prompt 版本反复试。
3. 多个 judge prompt。
4. 多个用户分层。
5. 多个安全指标。
6. 多次中途看实验结果。

缓解方法：

1. 预先定义主指标。
2. 保留独立 holdout。
3. 区分探索性分析和确认性实验。
4. 对多重检验做校正。
5. 不只报告变好的指标。

Bonferroni 控制 family-wise error rate, 简单但保守；Benjamini-Hochberg 控制 false discovery rate, 更适合探索性分析中同时看很多候选发现。面试时不用把校正方法讲成万能规则，重点是说明：主指标必须预注册，探索性结果要独立验证，不能从几十个指标里挑一个好看的结论当作确认性结论。

面试回答：

多重比较会制造偶然显著。大模型实验通常会试很多模型、prompt、数据和指标，所以要提前定义主指标，用独立 holdout 验证，并

9.12 Ablation 和 controlled experiment

Ablation study 用于判断某个组件是否真的有贡献。

例如系统包含：

RAG + reranker + citation + safety filter

可以分别去掉 reranker、换 embedding model、关闭 citation，观察指标变化。

好的 ablation 必须控制变量。

如果你同时换了模型、prompt、数据和检索器，就无法知道提升来自哪里。

面试回答：

Ablation 的目标是判断某个组件是否真正贡献了提升。关键是控制其他变量，只改变一个因素。否则如果多个因素同时变化，就无法

9.13 LLM-as-a-Judge 的统计问题

LLM judge 不是绝对真值，它也有噪声和偏差。

需要关注：

1. Judge-human agreement。
2. Position bias。
3. Verbosity bias。
4. Self-preference。
5. Judge prompt 版本。
6. 置信区间。
7. 高风险样本人审复核。

如果报告只说 “GPT judge 认为 B 更好”，但没有说明校准、样本量、置信区间和偏差控制，这个结论不可靠。

9.14 面试高频问题

9.14.1 p-value 是什么

回答：

p-value 是在零假设成立时，观察到当前或更极端结果的概率。它不是新模型无效的概率，也不是新模型更好的概率。解释 p-value 时要结合 effect size、置信区间和实验设计。

9.14.2 为什么要报告置信区间

回答：

置信区间能表达效果大小和不确定性。只看点估计容易误判，尤其是样本量小或指标噪声大时。置信区间很宽说明结论不稳定，跨过

9.14.3 Bootstrap 在大模型评估中怎么用

回答：

Bootstrap 通过对评估样本有放回重采样，重复计算指标，得到指标分布和置信区间。它适合 pairwise win rate、LLM judge 分数、F

9.14.4 如何判断新模型是否值得上线

回答：

我会同时看统计显著性和实际意义。统计上看主指标、置信区间、样本量和分层结果；业务上看效果大小、成本、延迟、安全、误拒和

9.15 常见误区

1. 看到分数高就认为模型更好。
2. 把 p-value 理解成模型无效的概率。
3. 只报告平均分，不报告样本量和置信区间。
4. 统计显著就直接上线。
5. 多次试验后只汇报变好的结果。
6. Ablation 同时改多个变量。
7. 把 LLM judge 当作绝对真值。

9.16 小练习

1. 用自己的话解释 p-value。
2. 解释统计显著和实际显著的区别。
3. 给一个 pairwise win rate 设计 bootstrap 置信区间方案。
4. 设计一个 ChatGPT 服务新模型上线的 A/B test。
5. 说明如何校准 LLM judge。
6. 设计一个 RAG reranker 的 ablation。
7. 举例说明多重比较问题。

9.17 本章总结

评估统计帮助我们判断模型提升是否可信。

本章最重要的结论：

1. 评估分数是有不确定性的估计。
2. p-value 不是模型有效或无效的概率。
3. 置信区间比单点分数更有决策价值。
4. 统计显著不等于实际值得上线。
5. Paired evaluation 能降低样本难度差异带来的方差。
6. Bootstrap 适合复杂大模型指标的不确定性估计。
7. A/B test 要有主指标和护栏指标。
8. 多重比较会制造偶然显著。
9. Ablation 必须控制变量。
10. LLM judge 必须校准。

面试中讲评估统计，不要只说“看 p-value”。成熟回答应该同时覆盖效果大小、置信区间、样本量、实验设计、分层结果和上线风险。

第十章：数学面试题

这一章是第十三册的收束章节。前面九章讲了线性代数、概率、信息论、优化、低秩、统计学习、贝叶斯、强化学习和评估统计。本章把这些内容整理成大模型算法岗面试中最常见的数学问题。

面试中数学题通常不是为了考你背公式，而是看你能不能把公式、模型训练、工程现象和实验判断连起来。一个好的回答应该包含三层：直觉、公式、LLM 场景。

0. 本讲资料边界与第二轮精修口径

本讲第二轮精修时，参考了前九章已经核验过的线性代数、概率、信息论、优化、低秩、统计学习、贝叶斯、强化学习和评估统计资料，并额外核对了 Stanford CS229 关于 MLE / MAP / 正则化的基础口径，PyTorch CrossEntropyLoss 和 AdamW 文档，Hugging Face perplexity 评估说明，Transformer 原论文中的 scaled dot-product attention，AdamW、LoRA、PPO 和 DPO 论文，以及 NIST 工程统计手册关于置信区间和显著性检验的口径。

本章是第十三册的收束章，目标不是新增一套数学体系，而是把前九章的公式、直觉、代码题和面试表达压缩成可复盘、可手算、可追问的答题框架。它不展开完整证明，不替代线性代数 / 概率统计 / 优化教材，也不把某个面试模板写成所有公司的固定标准。

10.1 面试回答数学题的基本模板

推荐结构：

1. 先给直觉。
2. 再给核心公式。
3. 说明在 LLM 中怎么用。

4. 说一个常见误区或工程细节。

例如问 “交叉熵和 KL 的关系是什么”，不要只写公式。更好的回答是：

交叉熵衡量用模型分布 q 去编码真实分布 p 的平均代价。它可以分解成真实分布自身的熵 $H(p)$ 和 $KL(p||q)$ 。训练时 p 固定，所以最 token prediction 的 cross entropy loss 就是在让模型分布接近数据分布。

10.1.1 关键公式与数学面试总复盘

数学面试里，公式不是背诵目标，而是用来支撑判断。下面这组公式覆盖第十三册最常被追问的主线。

信息论和语言模型 loss：

$$H(p) = - \sum_x p(x) \log p(x)$$

$$H(p, q) = - \sum_x p(x) \log q(x)$$

$$D_{KL}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

$$H(p, q) = H(p) + D_{KL}(p||q)$$

自回归语言模型：

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{1:t-1})$$

$$L_{NLL} = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{1:t-1})$$

$$PPL = \exp(L_{avg})$$

Softmax 与交叉熵梯度：

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

$$\frac{\partial L}{\partial z_i} = p_i - y_i$$

优化与稳定性：

$$\theta_{t+1} = \theta_t - \eta g_t$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t$$

$$g' = \min(1, c/\|g\|_2)g$$

低秩与 LoRA:

$$W = U \Sigma V^T$$

$$W_r = U_r \Sigma_r V_r^T$$

$$W' = W + \Delta W, \quad \Delta W = BA$$

$$N_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}})$$

Attention:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

泛化和正则:

$$R(f) = E[\ell(f(x), y)], \quad \hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i)$$

$$E[(y-\hat{f}(x))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$$

贝叶斯、MLE 和 MAP:

$$p(H|E) = \frac{p(E|H)p(H)}{p(E)}$$

$$\theta_{\text{MLE}} = \arg \max_{\theta} p(D|\theta)$$

$$\theta_{\text{MAP}} = \arg \max_{\theta} p(D|\theta)p(\theta)$$

RLHF / PPO / DPO:

$$A_t = G_t - V(s_t)$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

$$L_{\text{clip}} = E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

$$L_{\text{RM}} = -\log \sigma(r^+ - r^-)$$

$$\Delta_{\text{DPO}} = \beta[(\log \pi_{\theta}(y^+|x) - \log \pi_{\text{ref}}(y^+|x)) - (\log \pi_{\theta}(y^-|x) - \log \pi_{\text{ref}}(y^-|x))]$$

$$L_{\text{DPO}} = -\log \sigma(\Delta_{\text{DPO}})$$

评估统计：

$$\bar{x} = \frac{1}{n} \sum_i x_i, \quad SE(\bar{x}) = \frac{s}{\sqrt{n}}$$

$$CI_{95} \approx \bar{x} \pm 1.96SE(\bar{x})$$

$$d_i = m_i^{new} - m_i^{old}$$

$$\chi^2_{\text{McNemar}} = \frac{(|b - c| - 1)^2}{b + c}$$

面试中使用这些公式时，优先说明变量含义、适用条件和常见误解。例如“CE 等价于 KL”要补充 p 固定，“PPL 低”不等于对齐好，“DPO beta”是控制偏好 margin 和 reference 偏离的超参，具体解释要看实现和推导约定。

10.2 交叉熵和 KL 的关系是什么

核心公式：

$$H(p, q) = H(p) + KL(p || q)$$

其中：

$$H(p, q) = - \sum_x p(x) \log q(x)$$

$$H(p) = - \sum_x p(x) \log p(x)$$

$$KL(p || q) = \sum_x p(x) \log(p(x) / q(x))$$

直觉：

1. 熵 $H(p)$ 是真实分布自身的不确定性。
2. 交叉熵 $H(p, q)$ 是用 q 表示 p 的代价。
3. KL 是由于 q 不等于 p 造成的额外代价。

在监督训练中，数据分布 p 固定，所以 $H(p)$ 是常数，最小化交叉熵等价于最小化 $KL(p||q)$ 。

LLM 场景：

1. 预训练 next-token loss 是 token 级交叉熵。
2. Perplexity 是交叉熵的指数形式。
3. SFT 也是在示范数据上最小化交叉熵。
4. Distillation 常用 KL 或软标签交叉熵。

面试回答：

交叉熵可以分解为真实分布的熵加 KL 散度。由于训练数据分布固定，真实熵不随模型变化，所以最小化交叉熵等价于最小化 $KL(p||q)$ 。
token cross entropy 就是在让模型预测分布靠近数据中的下一个 token 分布。

常见误区：

1. 把交叉熵和 KL 当成完全一样。
2. 忘记 $H(p)$ 是否为常数取决于优化对象。
3. 不区分 $KL(p||q)$ 和 $KL(q||p)$ 。

10.3 $KL(p||q)$ 和 $KL(q||p)$ 有什么区别

KL 是不对称的。

$$KL(p || q) \neq KL(q || p)$$

$KL(p||q)$ 的惩罚来自 p 高而 q 低的地方。如果真实分布认为某个事件重要，但模型 q 给了很小概率，惩罚会很大。

$KL(q||p)$ 更关心模型 q 放了概率但真实 p 不支持的地方。

直觉上：

1. Forward KL, $KL(p||q)$, 倾向于覆盖 p 的所有模式。
2. Reverse KL, $KL(q||p)$, 倾向于选择一个高概率模式，可能 mode-seeking。

LLM 场景：

1. 最大似然训练对应 $KL(data||model)$ 。
2. RLHF 中 KL penalty 常约束新 policy 不要离 reference 太远。
3. DPO 中隐含了 policy 相对 reference 的对数概率差。

面试回答：

KL 是不对称的。 $KL(p||q)$ 对 p 有质量但 q 给低概率的位置惩罚很重，倾向于覆盖真实分布； $KL(q||p)$ 对 q 放到 p 不支持区域的概率 seeking。语言模型 MLE 更接近最小化 $KL(data||model)$ ，而 RLHF 常用 KL 约束防止 policy 偏离 reference。

10.4 Perplexity 如何理解

Perplexity 可以理解为模型平均每一步有多少 “困惑度”。

如果 token 级交叉熵是：

$$CE = - \text{average} \log p(x_t | x_{<t})$$

那么：

$$PPL = \exp(CE)$$

如果使用 log base 2, 则：

$$PPL = 2^{CE}$$

直觉：PPL 越低，模型给真实下一个 token 的概率越高。

LLM 场景：

1. PPL 适合衡量语言建模能力。
2. PPL 不等于任务能力。
3. 低 PPL 不一定代表 instruction following 更好。
4. 不同 tokenizer 或不同数据集上的 PPL 不能随便横向比较。

面试回答：

Perplexity 是交叉熵的指数形式，可以理解为模型平均每个 token 的困惑度。PPL 越低说明模型给真实 token 的概率越高。但 PPL 主

10.5 Softmax 为什么常和交叉熵一起用

Softmax 把 logits 转成概率分布：

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

交叉熵衡量预测分布和目标分布的差异。

对 one-hot 标签，softmax cross entropy 的梯度非常简洁：

$$dL/dz_i = p_i - y_i$$

这意味着：

1. 正确类别概率低，就强烈提高它的 logit。
2. 错误类别概率高，就降低它的 logit。
3. 梯度形式稳定且易于优化。

LLM 场景：每个位置输出词表 logits，经 softmax 得到 next-token 概率，再用目标 token 的负对数似然作为 loss。

面试回答：

Softmax 把 logits 变成概率分布，交叉熵衡量预测概率和目标分布的差距。二者结合后，对 logits 的梯度是 $p - y$ ，形式简单稳定。语言模型每个 token 位置都会对词表 logits 做 softmax，然后用真实下一个 token 计算 cross entropy loss。

10.6 为什么训练语言模型等价于最大似然估计

给定序列：

$x_1, x_2, \dots, x_T$

自回归语言模型分解为：

$$p(x_1, \dots, x_T) = \text{product}_t p(x_t \mid x_{<t})$$

最大化数据似然：

$$\max \sum_t \log p(x_t \mid x_{<t})$$

等价于最小化负对数似然：

$$\min - \sum_t \log p(x_t \mid x_{<t})$$

这就是 token 级交叉熵损失。

面试回答：

自回归语言模型把序列概率分解成每个 token 的条件概率乘积。最大似然就是最大化训练语料中真实 token 的条件概率，取负号后就 token cross entropy。因此预训练本质上是对语料分布做最大似然估计。

10.7 Adam 和 SGD 的区别是什么

SGD 使用当前梯度更新参数：

$$\theta = \theta - lr * grad$$

Adam 使用一阶矩和二阶矩自适应调整每个参数的步长：

$$\begin{aligned} m_t &= \beta_1 * m_{t-1} + (1 - \beta_1) * g_t \\ v_t &= \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \\ \theta &= \theta - lr * m_{hat} / (\sqrt{v_{hat}} + \epsilon) \end{aligned}$$

直觉：

1. m 是动量，平滑梯度方向。
2. v 是梯度平方的估计，用来缩放学习率。
3. 梯度经常很大的参数步子变小，梯度小的参数相对步子变大。

LLM 场景：Adam/AdamW 是大模型训练和微调中的常用优化器，因为 Transformer 参数规模大、不同参数梯度尺度差异明显。

面试回答：

SGD 基本按当前梯度统一更新，Adam 会维护梯度一阶矩和二阶矩，对不同参数使用自适应步长。一阶矩类似动量，二阶矩用于按梯度缩放。

10.8 AdamW 为什么不同于 Adam

Adam 中如果把 L2 正则直接加到 loss，正则项会进入梯度，并被 Adam 的自适应二阶矩缩放。

AdamW 的核心是 decoupled weight decay:

```
theta = theta - lr * adam_update - lr * weight_decay * theta
```

也就是权重衰减不再混入梯度自适应缩放，而是直接对参数做衰减。

为什么重要：

1. Weight decay 的含义更接近真正的参数收缩。
2. 避免不同参数因为自适应缩放导致正则强度不一致。
3. 大模型训练中通常比 Adam + L2 更稳定。

面试回答：

AdamW 和 Adam 的关键区别是 weight decay 解耦。Adam 里 L2 正则项会进入梯度，然后被自适应二阶矩缩放；AdamW 直接对参数做衰减，

10.9 学习率 warmup 为什么有用

训练初期参数、梯度统计和 optimizer state 都不稳定。如果一开始使用很大学习率，容易 loss spike 或训练发散。

Warmup 的做法是前若干 step 从小学习率逐步升到目标学习率。

LLM 中 warmup 有几个作用：

1. 让 Adam 的一阶、二阶矩估计先稳定。
2. 避免初期大步更新破坏预训练表示。
3. 在大 batch 和 mixed precision 下提高稳定性。
4. 微调时减少灾难性遗忘风险。

面试回答：

Warmup 是为了提高训练初期稳定性。刚开始 optimizer 的动量和二阶矩估计还不可靠，直接使用大学习率容易 loss spike。Warmup 让

10.10 梯度裁剪为什么能稳定训练

梯度裁剪限制梯度范数：

```
if ||g|| > c:  
    g = c * g / ||g||
```

它不会改变小梯度，只在梯度过大时缩放。

作用：

1. 防止极端 batch 导致参数大幅跳动。
2. 缓解 exploding gradients。
3. 提高 mixed precision 训练稳定性。
4. 在 RLHF 等高噪声阶段尤其有用。

面试回答：

梯度裁剪通过限制梯度范数，避免少数异常 batch 造成过大的参数更新。它不是让模型学得更快，而是减少训练不稳定和 loss spike

10.11 为什么归一化有助于训练

归一化的作用是控制激活或特征尺度，使优化更稳定。

Transformer 中常见 LayerNorm 或 RMSNorm。

归一化带来的好处：

1. 降低不同层激活尺度漂移。
2. 改善梯度传播。
3. 减少学习率敏感性。
4. 让深层网络更容易训练。

Pre-LN Transformer 把 norm 放在子层之前，通常比 Post-LN 更稳定，尤其适合深层大模型。

面试回答：

归一化通过控制激活尺度，让深层网络的优化更稳定。Transformer 中 LayerNorm 或 RMSNorm 能减少层间尺度漂移，改善梯度传播。LN 或 RMSNorm，因为深层训练更稳定，对学习率也不那么敏感。

10.12 SVD 和低秩近似是什么

SVD 将矩阵分解为：

$$W = U \Sigma V^T$$

如果只保留最大的 r 个奇异值，可以得到低秩近似：

$$W_r = U_r \Sigma_r V_r^T$$

直觉：矩阵中很多变化方向不一样重要，最大奇异值对应最主要的方向。

低秩近似的意义：

1. 压缩参数。
2. 提取主要结构。
3. 降低计算和存储。
4. 解释 LoRA 等参数高效微调方法。

面试回答：

SVD 把矩阵分解成正交方向和奇异值。保留最大的 r 个奇异值，就得到最佳低秩近似之一。直觉是矩阵变化主要集中在少数重要方向。

10.13 LoRA 为什么是低秩

LoRA 不直接全量更新权重矩阵 W ，而是学习一个低秩增量：

$$W' = W + \Delta W$$

$$\Delta W = B A$$

其中：

$$B \in \mathbb{R}^{d_{\text{out}} \times r}$$

$$A \in \mathbb{R}^{r \times d_{\text{in}}}$$

$$r \ll \min(d_{\text{in}}, d_{\text{out}})$$

所以 ΔW 的 rank 不超过 r 。

为什么有效：

1. 下游任务需要的更新可能集中在少数方向。
2. 预训练模型已经有强表达能力，不需要全量改动。
3. 低秩更新减少显存和可训练参数。
4. 多任务适配时可以保存不同 LoRA adapter。

面试回答：

LoRA 低秩是因为它把权重增量写成两个小矩阵的乘积 $\Delta W = BA$ ，其中中间维度 r 很小，所以增量矩阵的秩最多是 r 。它假设微调所追问：为什么不是所有任务都适合很小的 r ？

回答：

如果任务和预训练分布差异很大，或者需要改变很多能力方向，低秩子空间可能不够表达， r 太小会欠拟合。实际要根据任务复杂度、 off 调参。

10.14 Attention 中点积为什么要除以 $\sqrt{d}$

Attention score 是：

$$\text{score} = QK^T / \sqrt{d_k}$$

如果 q 和 k 的每个维度方差近似为 1，那么点积 $q \cdot k$ 的方差大约随 d_k 增长。

维度越高，点积数值越大，softmax 更容易饱和，导致梯度变小。

除以 $\sqrt{d_k}$ 可以让 score 的尺度更稳定。

面试回答：

Q 和 K 的点积会随着维度增加而方差变大。如果不缩放，softmax 输入过大，分布容易变得极端，梯度变小。除以 $\sqrt{d_k}$ 是为了

10.15 为什么残差连接有助于深层网络

残差连接形式：

$$y = x + F(x)$$

作用：

1. 提供信息直通路径。
2. 改善梯度传播。
3. 让网络更容易学习增量变换。
4. 缓解深层退化问题。

在 Transformer 中，每个 attention block 和 MLP block 周围都有 residual connection。

面试回答：

残差连接让层学习相对输入的增量，而不是从零学习完整映射。它提供了信息和梯度的直通路径，使深层网络更容易优化。Transform

10.16 Bias-variance trade-off 如何理解

Bias 是模型假设和真实规律之间的系统性偏差。

Variance 是模型对训练数据扰动的敏感程度。

直觉：

1. 模型太简单，bias 高，容易欠拟合。
2. 模型太复杂且数据不足，variance 高，容易过拟合。
3. 正则化、更多数据、早停和数据增强可以影响二者平衡。

LLM 场景：

1. 预训练大模型容量很大，通常靠海量数据和正则化控制泛化。
2. 小数据 SFT 容易过拟合风格和格式。
3. RLHF 过强可能导致 reward hacking 或分布变窄。

面试回答：

Bias-variance trade-off 描述欠拟合和过拟合之间的平衡。Bias 高说明模型表达不足，variance 高说明模型对训练样本过敏。大模

10.17 为什么正则化能缓解过拟合

正则化限制模型复杂度或参数规模。

常见方式：

1. Weight decay。
2. Dropout。
3. Early stopping。
4. Data augmentation。
5. Label smoothing。
6. KL penalty。

数学直觉：正则化相当于在优化目标中加入约束或先验，使模型不要为了拟合训练集噪声而走到过于复杂的解。

LLM 场景：

1. SFT 数据少时可能背答案或过拟合格式。
2. RLHF 用 KL penalty 约束 policy 不偏离 reference。
3. DPO 中 beta 控制偏好优化强度。

面试回答：

正则化通过限制参数规模、输出变化或策略偏移，降低模型对训练噪声的拟合。它可以看成对解空间加约束，也可以看成引入先验。

10.18 Bayes 公式在大模型中有什么用

Bayes 公式：

$$P(A \mid B) = P(B \mid A) P(A) / P(B)$$

直觉：看到证据 B 后，更新对假设 A 的信念。

LLM 相关场景：

1. 生成模型可以看成根据上下文更新下一个 token 分布。
2. RAG 中根据 query 和检索证据更新答案可信度。
3. 分类任务中 posterior 取决于 likelihood 和 prior。
4. 不确定性估计中关注模型 confidence 是否校准。

面试回答：

Bayes 公式描述如何用新证据更新信念。它由先验、似然和后验组成。大模型中，虽然 Transformer 不一定显式做 Bayes 推断，但很

10.19 MLE 和 MAP 有什么区别

MLE 最大化数据似然：

$$\text{theta_MLE} = \text{argmax_theta } P(D \mid \text{theta})$$

MAP 最大化后验：

$$\text{theta_MAP} = \text{argmax_theta } P(D \mid \text{theta}) P(\text{theta})$$

区别：MAP 多了参数先验。

如果取 log：

$$\log P(D \mid \text{theta}) + \log P(\text{theta})$$

其中 $\log P(\text{theta})$ 常可理解为正则项。

面试回答：

MLE 只最大化数据似然，MAP 最大化后验，相当于在似然之外加入参数先验。取 log 后，先验项常表现为正则化。例如高斯先验对应

10.20 DPO loss 如何理解

DPO 用偏好数据直接优化 policy，不显式训练 reward model 后再跑 PPO。

对于同一个 prompt，有 preferred response y_w 和 rejected response y_l 。

DPO 关注 policy 相对 reference 的 log probability 差：

$$\log \pi(y_w|x) - \log \pi_{\text{ref}}(y_w|x)$$

和：

$$\log \pi(y_l|x) - \log \pi_{\text{ref}}(y_l|x)$$

目标是让 preferred 的相对优势高于 rejected。

简化管理解：

increase relative probability of chosen answer
decrease relative probability of rejected answer
stay close to reference through implicit KL control

面试回答：

DPO 可以理解为直接从偏好对中学习 policy。它比较 chosen 和 rejected response 在当前 policy 下相对 reference policy 的 log

追问：DPO 里的 beta 控制什么？

回答：

beta 会缩放 chosen/rejected 的 log-ratio margin，并间接影响 policy 相对 reference 的偏离强度。不同推导里它也常被解释成 KL

10.21 PPO 中 advantage 是什么

Advantage 衡量某个动作或轨迹比预期好多少：

$$A = \text{return} - \text{value}$$

如果 $A > 0$ ，说明比 baseline 好，应提高概率。

如果 $A < 0$ ，说明比 baseline 差，应降低概率。

在 RLHF 中：

1. Policy 是语言模型。
2. Action 是 token。
3. Reward 来自 reward model 或规则。
4. Value model 估计期望回报。
5. Advantage 指导 policy 更新方向。

面试回答：

Advantage 表示实际回报相对 value baseline 好多少。它的作用是降低 policy gradient 方差，并告诉模型哪些 token 或回答应该提

10.22 PPO 为什么需要 clipping

PPO 使用概率比值：

$$r_t = \pi_{\text{new}}(a_t|s_t) / \pi_{\text{old}}(a_t|s_t)$$

如果 policy 一次更新太大，训练会不稳定。

Clipping 限制 r_t 的范围：

$$\text{clip}(r_t, 1 - \text{eps}, 1 + \text{eps})$$

直觉：不要让新 policy 相对旧 policy 改得太猛。

LLM 场景：RLHF 中如果 policy 偏离 reference 或 old policy 太快，可能出现胡说、格式崩坏、reward hacking 或安全退化。

面试回答：

PP0 clipping 是为了限制 policy 每次更新幅度。它通过裁剪新旧 policy 概率比，避免某些高 reward 样本导致模型概率突变。在 RL

10.23 统计显著和实际显著有什么区别

统计显著回答：观察到的差异是否不像随机噪声。

实际显著回答：这个差异是否值得行动。

例如：

win rate +0.2%, p-value < 0.05

latency +30%, cost +25%

这可能统计显著，但未必值得上线。

面试回答：

统计显著说明差异不太像随机波动，实际显著说明差异是否有业务和工程价值。大模型上线不能只看 p-value，还要看 effect size、

10.24 p-value 和置信区间怎么解释

p-value：在零假设成立时，观察到当前或更极端结果的概率。

置信区间：指标估计的不确定范围。

常见错误：

1. p-value 不是“新模型无效的概率”。
2. p-value 不是“新模型更好的概率”。
3. 95% CI 不是说某个固定真值有 95% 概率落在这个具体区间。
4. 显著不等于一定能上线。

面试回答：

p-value 表示零假设成立时，观察到当前或更极端结果的概率。置信区间表达效果大小的不确定范围。模型评估时我不会只看 p-value，而会同时报告点估计、置信区间、样本量、effect size 和实验设计。

10.25 Bootstrap 为什么适合大模型评估

大模型评估指标常常复杂，很难写解析方差。

例如：

1. Pairwise win rate。
2. LLM judge score。
3. RAG faithfulness。
4. Human eval rubric。
5. Safety violation rate。

Bootstrap 通过重复有放回抽样估计指标分布。

面试回答：

Bootstrap 适合大模型评估，因为很多指标不是简单均值，解析方差不好推。它通过对评估样本有放回重采样，重复计算指标，得到抽

10.26 为什么 pairwise eval 常比单独打分更稳

Pairwise eval 让两个模型在同一个 prompt 下直接比较。

优点：

1. 控制样本难度。
2. 标注者更容易判断相对好坏。
3. 比绝对分数更稳定。
4. 适合开放生成任务。

缺点：

1. Tie 处理会影响结果。
2. 顺序偏差需要随机化。
3. 需要做显著性分析。
4. Judge 仍可能有 verbosity bias。

面试回答：

Pairwise eval 在同一 prompt 下比较两个回答，能控制样本难度，通常比单独打绝对分更稳定。它适合开放生成任务，因为标注者判

10.27 如果模型分数提升 0.5%，你如何判断是否可信

回答框架：

1. 看样本量和置信区间。
2. 做 paired evaluation。
3. 做显著性检验或 bootstrap。
4. 看分层结果。
5. 检查是否多重比较。
6. 做错误分析。
7. 看成本、延迟和安全护栏。
8. 必要时做线上 A/B。

面试回答：

我不会只看 0.5% 的点估计。首先看评估集大小、方差和置信区间，最好用 paired evaluation 或 paired bootstrap 判断差异是否稳

10.28 如何解释 embedding 相似度

Embedding 把文本映射到向量空间。相似度常用 cosine similarity：

$$\cos(a, b) = a \cdot b / (||a|| \cdot ||b||)$$

直觉：

1. 方向越接近，语义越相似。
2. Cosine 关注方向，不直接关注向量长度。
3. Dot product 同时受方向和长度影响。

LLM 场景：

1. RAG 检索。
2. 去重。
3. 聚类。
4. 语义匹配。
5. 评估答案相似性。

面试回答：

Embedding 相似度通常用向量夹角衡量，cosine similarity 越高表示方向越接近，语义上往往越相似。它适合 RAG 检索和语义匹配。

10.29 为什么高维空间中的距离会失效

高维空间有一些反直觉现象：

1. 点与点之间距离趋于接近。
2. 最近邻和最远邻差距变小。
3. 数据变稀疏。
4. 噪声维度会干扰相似度。

这叫 curse of dimensionality 的一部分。

LLM 场景：embedding 检索不能只依赖粗糙向量距离，通常还需要：

1. 更好的 embedding model。
2. 归一化。
3. reranker。
4. metadata filter。
5. chunk 策略。
6. 人工评估和线上指标。

面试回答：

高维空间中数据稀疏，距离容易集中，最近邻不一定真正语义相关。Embedding 检索虽然有效，但不能迷信向量距离。实际 RAG 系统

10.30 面试中的数学题答题清单

遇到数学题，可以检查自己是否覆盖了这些点：

1. 有没有先讲直觉。
2. 有没有写出关键公式。
3. 有没有解释公式中每个量的含义。
4. 有没有连接到 LLM 训练、微调、对齐、评估或检索。
5. 有没有说明边界条件。
6. 有没有指出常见误区。
7. 有没有体现工程 trade-off。

10.31 高频快问快答

10.31.1 Cross entropy 为什么能训练分类模型

回答：

Cross entropy 等价于最大化正确标签的 log likelihood。对 softmax logits 的梯度是 $p-y$ ，会提高正确类别概率、降低错误类别概率 token prediction。

10.31.2 KL 为什么非负

回答：

KL 非负来自 Gibbs inequality。直觉上，用错误分布 q 编码真实分布 p ，不会比用 p 自己编码更省。KL 为 0 当且仅当两个分布相同。

10.31.3 为什么 log probability 要相加

回答：

序列概率是条件概率的乘积，取 log 后乘积变求和，数值更稳定，也方便优化。语言模型训练最大化 $\sum \log p(x_t|x_{<t})$ 。

10.31.4 Temperature 对 softmax 有什么影响

回答：

Temperature 会缩放 logits。温度低时分布更尖锐，模型更确定；温度高时分布更平滑，输出更多样。采样时 temperature 影响多样性。

10.31.5 Label smoothing 有什么作用

回答：

Label smoothing 把 one-hot 标签变得更平滑，避免模型对训练标签过度自信。它能改善校准和泛化，但在语言模型中要谨慎，因为过平滑可能降低性能。

10.31.6 Weight decay 和 L2 正则一样吗

回答：

在普通 SGD 下二者常等价，但在 Adam 这类自适应优化器中不完全等价。AdamW 使用 decoupled weight decay，避免 weight decay 被学习率影响。

10.31.7 为什么 batch size 会影响训练

回答：

Batch size 影响梯度估计噪声。小 batch 噪声大，可能有正则化效果但不稳定；大 batch 梯度更稳定、吞吐更高，但可能需要调学习率。

10.31.8 为什么要做 learning rate decay

回答：

训练后期需要更小步长来收敛到更稳定的区域。学习率衰减可以减少参数震荡，提高最终 loss 和泛化表现。大模型常用 cosine decay。

10.31.9 为什么 checkpoint averaging 或 EMA 有时有效

回答：

它们可以看成在参数空间做平滑，减少单个 checkpoint 的随机波动。EMA 对近期参数赋更大权重，可能得到更稳定的模型。但大模型中效果可能不明显。

10.31.10 为什么 LLM judge 需要校准

回答：

LLM judge 有位置偏差、长度偏差、自偏好和 prompt 敏感性。校准的目标是确认 judge 分数和人类判断一致，并控制偏差。通常要做温度校准和提示工程。

10.32 常见追问陷阱

10.32.1 “最小化 CE 等价于最小化 KL” 总是对吗

不总是。它依赖 p 是否固定。如果真实分布 p 固定，则 $H(p)$ 是常数，最小化 $H(p,q)$ 等价于最小化 $KL(p||q)$ 。如果 p 也在变，就不能这样简单说。

10.32.2 “PPL 低就一定模型好” 对吗

不对。PPL 衡量预测下一个 token 的能力，不直接等价于推理、对齐、安全、工具使用 and 用户满意度。

10.32.3 “LoRA 参数少所以一定不如全参微调” 对吗

不一定。很多下游任务需要的更新可能低秩，LoRA 可以达到接近全参微调的效果，并显著降低成本。但对于分布差异很大或需要深度改变能力的任务，全参或更高 rank 可能更好。

10.32.4 “p-value 小就一定能上线” 对吗

不对。上线还要看实际收益、成本、延迟、安全、用户分层和长期影响。

10.32.5 “DPO 一定比 PPO 好” 对吗

不对。DPO 更简单稳定，不需要在线 RL rollout，但也依赖偏好数据质量和 reference。PPO 在某些需要显式 reward 优化和复杂反馈的场景仍有价值。

10.33 最小可运行数学面试复盘 demo

下面这个 0 依赖 demo 做两件事：第一，手算一组面试高频公式；第二，用一个 toy rubric 检查数学面试回答是否同时覆盖公式、直觉、LLM 场景、边界和代码 demo。

```
from math import exp, log, sqrt
```

```
def softmax(logits, temperature=1.0):
    scaled = [z / temperature for z in logits]
    shift = max(scaled)
    weights = [exp(z - shift) for z in scaled]
    total = sum(weights)
    return [w / total for w in weights]

def entropy(probs):
    return -sum(p * log(p) for p in probs if p > 0)

def cross_entropy(p, q):
    return -sum(pi * log(qi) for pi, qi in zip(p, q) if pi > 0)

def kl_divergence(p, q):
    return sum(pi * log(pi / qi) for pi, qi in zip(p, q) if pi > 0)

def sigmoid(x):
    return 1.0 / (1.0 + exp(-x))

def mean(values):
    return sum(values) / len(values)

def l2_norm(values):
    return sqrt(sum(v * v for v in values))

def global_clip(grad, max_norm):
    norm = l2_norm(grad)
    scale = min(1.0, max_norm / norm)
    return [g * scale for g in grad], norm, scale
```

```
p_true = [0.7, 0.2, 0.1]
```

```

q_model = [0.6, 0.25, 0.15]
h = entropy(p_true)
ce = cross_entropy(p_true, q_model)
kl = kl_divergence(p_true, q_model)

logits = [2.0, 1.0, 0.0]
probs = softmax(logits)
one_hot = [1.0, 0.0, 0.0]
ce_grad = [round(pi - yi, 3) for pi, yi in zip(probs, one_hot)]

seq_logprobs = [-0.2, -0.5, -1.1, -0.7]
avg_nll = -sum(seq_logprobs) / len(seq_logprobs)
ppl = exp(avg_nll)

clipped_grad, grad_norm, clip_scale = global_clip([3.0, 4.0, 12.0], 5.0)

full_params = 4096 * 4096
lora_params = 8 * (4096 + 4096)
lora_ratio = lora_params / full_params
attention_scale = sqrt(64)

policy_chosen = -1.2
policy_rejected = -2.0
ref_chosen = -1.5
ref_rejected = -1.7
beta = 0.5
dpo_margin = beta * (
    (policy_chosen - ref_chosen) - (policy_rejected - ref_rejected)
)
dpo_loss = -log(sigmoid(dpo_margin))

old = [1, 0, 1, 0, 1, 0, 1, 0]
new = [1, 1, 1, 1, 0, 0, 1, 1]
paired_diff = [n - o for o, n in zip(old, new)]
win_lift = mean(paired_diff)

required_topics = {
    "linear_algebra",
    "probability",
    "information",
    "optimization",
    "low_rank",
    "generalization",
    "bayes",
    "rlhf_math",
    "eval_stats",
}
answers = [
    {
        "qid": "ce_kl",
        "topics": {"information", "probability"},
        "formula": 1.0,
        "intuition": 1,
        "llm": 1,
        "caveat": 1,
    }

```

```

        "demo": 1,
    },
    {
        "qid": "adamw",
        "topics": {"optimization"},
        "formula": 0.8,
        "intuition": 1,
        "llm": 1,
        "caveat": 1,
        "demo": 1,
    },
    {
        "qid": "lora",
        "topics": {"linear_algebra", "low_rank"},
        "formula": 1.0,
        "intuition": 1,
        "llm": 1,
        "caveat": 1,
        "demo": 1,
    },
    {
        "qid": "dpo_ppo",
        "topics": {"rlhf_math"},
        "formula": 0.7,
        "intuition": 1,
        "llm": 1,
        "caveat": 0,
        "demo": 1,
    },
    {
        "qid": "eval_stats",
        "topics": {"eval_stats"},
        "formula": 0.9,
        "intuition": 1,
        "llm": 1,
        "caveat": 1,
        "demo": 1,
    },
    {
        "qid": "generalization",
        "topics": {"generalization"},
        "formula": 0.4,
        "intuition": 1,
        "llm": 1,
        "caveat": 0,
        "demo": 0,
    },
}
]

```

```

covered_topics = set()
for ans in answers:
    covered_topics.update(ans["topics"])

```

```

def answer_score(ans):

```

```

return (
    2 * ans["formula"]
    + ans["intuition"]
    + ans["llm"]
    + ans["caveat"]
    + ans["demo"]
) / 6

scores = {ans["qid"]: round(answer_score(ans), 3) for ans in answers}
weak_questions = [qid for qid, score in scores.items() if score < 0.8]
missing_topics = sorted(required_topics - covered_topics)
topic_coverage = len(covered_topics) / len(required_topics)
formula_accuracy = mean([ans["formula"] for ans in answers])
demo_coverage = mean([ans["demo"] for ans in answers])
readiness_gate = (
    topic_coverage >= 0.9
    and formula_accuracy >= 0.85
    and demo_coverage >= 0.8
    and not weak_questions
)

report = {
    "formula_checks": {
        "entropy": round(h, 3),
        "cross_entropy": round(ce, 3),
        "kl": round(kl, 3),
        "ce_minus_h": round(ce - h, 3),
        "ppl": round(ppl, 3),
        "softmax_probs": [round(x, 3) for x in probs],
        "ce_grad": ce_grad,
        "clip": {
            "norm": round(grad_norm, 3),
            "scale": round(clip_scale, 3),
            "clipped": [round(x, 3) for x in clipped_grad],
        },
    },
    "lora": {
        "full": full_params,
        "rank8": lora_params,
        "ratio": round(lora_ratio, 6),
    },
    "attention_scale_sqrt64": round(attention_scale, 3),
    "dpo": {"margin": round(dpo_margin, 3), "loss": round(dpo_loss, 3)},
    "paired_eval_lift": round(win_lift, 3),
},
    "interview_scores": scores,
    "readiness": {
        "topic_coverage": round(topic_coverage, 3),
        "formula_accuracy": round(formula_accuracy, 3),
        "demo_coverage": round(demo_coverage, 3),
        "missing_topics": missing_topics,
        "weak_questions": weak_questions,
        "ready": readiness_gate,
        "revision_plan": [
            "  补一个 Bayes / MAP 手算题",

```

```

        " 把 DPO / PPO 回答补上 beta 与 reference 边界",
        " 给泛化题增加 bias-variance 或 early stopping demo",
    ],
},
"checks": {
    "ce_kl_decomposition": round(ce - h, 6) == round(kl, 6),
    "softmax_grad_sums_zero": abs(
        sum([pi - yi for pi, yi in zip(probs, one_hot)])
    ) < 1e-9,
    "lora_ratio_small": lora_ratio < 0.01,
    "readiness_blocks_missing_topics": readiness_gate is False
    and "bayes" in missing_topics,
},
}

```

print(report)

这段 demo 的输出应该体现三件事：

1. formula_checks 能验证 CE/KL 分解、PPL、softmax 梯度、gradient clipping、LoRA 参数比例、attention 缩放、DPO loss 和 paired eval lift。
2. interview_scores 把每道题拆成公式、直觉、LLM 场景、边界和 demo 五个维度，而不是只看会不会背定义。
3. readiness 会暴露缺失主题和薄弱题。例如缺 Bayes / MAP，或 DPO / PPO 边界不清，就不能认为数学面试已经准备好。

10.34 小练习

1. 用 3 句话解释交叉熵和 KL 的关系。
2. 用公式说明 AdamW 和 Adam + L2 的区别。
3. 解释 LoRA 的低秩增量为什么 rank 不超过 r 。
4. 说明 attention score 为什么要除以 $\sqrt{d}$ 。
5. 解释 DPO loss 中 reference model 的作用。
6. 设计一个判断 0.5% win rate 提升是否可信的实验流程。
7. 说明为什么 PPL 不能直接代表 instruction following 能力。
8. 解释 pairwise eval 为什么常比单独打分稳定。
9. 说明 temperature 如何影响生成分布。
10. 解释 MLE、MAP 和正则化之间的关系。
11. 用 0 依赖 Python 写一个数学面试复盘 demo，至少覆盖 CE/KL、PPL、softmax gradient、LoRA 参数量、DPO loss、paired eval 和 readiness rubric。

10.35 本章总结

大模型算法岗的数学面试，重点不是炫技推导，而是把数学概念讲到模型行为和工程决策上。

本章最重要的结论：

1. 交叉熵等于真实熵加 KL，训练中常等价于最小化 $KL(\text{data}||\text{model})$ 。
2. KL 不对称，方向不同会带来不同优化行为。
3. PPL 是交叉熵的指数形式，但不等于综合能力。
4. Softmax cross entropy 的梯度是 $p-y$ ，适合分类和 token prediction。
5. AdamW 的关键是 decoupled weight decay。
6. Warmup、梯度裁剪、归一化和残差都是训练稳定性工具。
7. SVD 和低秩近似是理解 LoRA 的数学基础。
8. Attention 除以 $\sqrt{d}$ 是为了控制点积尺度。
9. DPO 可以理解为带 reference 约束的偏好学习目标。
10. 模型评估要同时看显著性、置信区间、effect size 和上线风险。

如果只能记住一句话：数学面试的好答案不是“公式正确”，而是“公式、直觉、LLM 场景和工程取舍同时正确”。